

**Neurapy**  
*Release v5.0.8*

**Neura Robotics GmbH**

**Jul 11, 2025**

# NEURAPY

|          |                                                 |            |
|----------|-------------------------------------------------|------------|
| <b>1</b> | <b>Neurapy</b>                                  | <b>1</b>   |
| <b>2</b> | <b>Offline Simulation Setup</b>                 | <b>21</b>  |
| <b>3</b> | <b>API Installation</b>                         | <b>34</b>  |
| <b>4</b> | <b>API Reference</b>                            | <b>40</b>  |
| <b>5</b> | <b>Examples</b>                                 | <b>171</b> |
| <b>6</b> | <b>Sample Applications</b>                      | <b>201</b> |
| <b>7</b> | <b>TCP-flange connection pin outs for LARA</b>  | <b>225</b> |
| <b>8</b> | <b>TCP-flange connection pin outs for Maira</b> | <b>227</b> |
|          | <b>Python Module Index</b>                      | <b>229</b> |
|          | <b>Index</b>                                    | <b>230</b> |

## NEURAPY

NeuraPy is an easy-to-use, extendable, robot-agnostic Python API for Neura's control software, designed for application developers.

## 1.1 Robot Class

NeuraPy provides a *Robot* class that allows access to all the functionalities and static parameters of the robot. These can be accessed as methods and attributes of an instantiation of the *Robot* class.

## 1.2 Parameters

- *robot\_name*
- *dof*
- *payload*
- *controller\_ip*
- *version*

**Warning:**

Before running any move function, *init\_program* must be called, and after motion execution, *stop* should be used. Example:

```
from neurapy.robot import Robot
r = Robot()
r.init_program()
r.move_joint('Home')
r.move_joint('Parking')
r.stop()
```

## 1.3 Functionalities

### Warning:

The following functions have been removed from the Robot class. These functions were deprecated in previous versions and are no longer available in the current release of Neurapy. Use the corresponding functions below for similar functionality:

- `get_current_cartesian_pose` → `get_tcp_pose_quaternion`
- `get_current_cartesian_pose_with_timestamp` → `get_tcp_pose_quaternion_with_timestamp`
- `get_mode` →
  - `is_robot_in_teach_mode`
  - `is_robot_in_automatic_mode`
  - `is_robot_in_semi_automatic_mode`
- `get_sim_or_real` → `is_robot_in_simulation`
- `get_tool_flange_pose` → `get_flange_pose`
- `get_zerog_status` → `is_free_drive_mode_enabled`
- `ik_fk` →
  - `compute_forward_kinematics`
  - `compute_inverse_kinematics`
- `override` →
  - `get_override`
  - `set_override`
- `power` →
  - `power_on`
  - `power_off`
- `record_path` → `move_recorded_path`
- `robot_status` →
  - `get_current_joint_angles`
  - `get_current_joint_angles_with_timestamp`
  - `get_tcp_pose_quaternion`
  - `get_tcp_pose_quaternion_with_timestamp`
  - `get_current_joint_torques`
  - `get_current_joint_torques_with_timestamp`
  - `get_current_joint_velocities_with_timestamp`
  - `get_current_load_side_encoder_values`
  - `get_load_side_encoder_values_with_timestamp`
  - `get_current_motor_side_encoder_values`
  - `get_motor_side_encoder_values_with_timestamp`
- `set_mode` →
  - `switch_to_teach_mode`
  - `switch_to_automatic_mode`
  - `switch_to_semi_automatic_mode`
- `set_sim_real` →
  - `switch_to_real`
  - `switch_to_simulation`
- `start_external_interface` →
  - `activate_servo_interface`
  - `activate_ros_interface`
  - `activate_ethernetIP_interface`
  - `activate_opcua_interface`
  - `activate_ethercat_interface`
  - `activate_profinet_interface`

- *stop\_external\_interface* →
  - *deactivate\_servo\_interface*
  - *deactivate\_ros\_interface*
  - *deactivate\_ethernetIP\_interface*
  - *deactivate\_opcua\_interface*
  - *deactivate\_ethercat\_interface*
  - *deactivate\_profinet\_interface*
- *zero\_g* →
  - *turn\_on\_free\_drive\_mode*
  - *turn\_off\_free\_drive\_mode*

Refer to the updated API documentation for details on these functions.

| S.No. | Functionality                             | Action                                                                                        |
|-------|-------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1.    | <i>activate_ethercat_interface</i>        | Activate the Ethercat/et6695 interface.                                                       |
| 2.    | <i>activate_ethernetIP_interface</i>      | Activate the Ethernet/IP interface.                                                           |
| 3.    | <i>activate_modbus_interface</i>          | Activate the Modbus interface.                                                                |
| 4.    | <i>activate_opcua_interface</i>           | Activate the OPC UA interface.                                                                |
| 5.    | <i>activate_profinet_interface</i>        | Activate the Profinet interface.                                                              |
| 6.    | <i>activate_ros_interface</i>             | Activate the ROS interface.                                                                   |
| 7.    | <i>activate_servo_interface</i>           | Activate the Servo interface.                                                                 |
| 8.    | <i>associate_pose_to_new_parent_frame</i> | Method which shall change the association of the input pose(point/frame) to a new frame.      |
| 9.    | <i>compute_forward_kinematics</i>         | Compute forward kinematics (end-effector's Cartesian pose) for the given joint configuration. |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                           | Action                                                                                                 |
|-------|-----------------------------------------|--------------------------------------------------------------------------------------------------------|
| 10.   | <i>compute_inverse_kinematics</i>       | Compute inverse kinematics(joint configuration) for the given end-effector's cartesian pose.           |
| 11.   | <i>convert_euler_to_quaternion_pose</i> | Convert a pose with Euler angles to quaternions.                                                       |
| 12.   | <i>convert_quaternion_to_euler_pose</i> | Convert a pose with quaternion representation to a pose with Euler angles.                             |
| 13.   | <i>create_external_source_app</i>       | Creates an instance of the external source, returning an object of the connected external source type. |
| 14.   | <i>create_point</i>                     | Store a point in the database.                                                                         |
| 15.   | <i>create_tool</i>                      | Create a new tool in the robot's database using the provided tool data.                                |
| 16.   | <i>deactivate_ethercat_interface</i>    | Deactivate the Ethercat interface.                                                                     |
| 17.   | <i>deactivate_ethernetIP_interface</i>  | Deactivate the Ethernet/IP interface.                                                                  |
| 18.   | <i>deactivate_modbus_interface</i>      | Deactivate the Modbus interface.                                                                       |
| 19.   | <i>deactivate_opcua_interface</i>       | Deactivate the OPCUA interface.                                                                        |
| 20.   | <i>deactivate_profinet_interface</i>    | Deactivate the Profinet interface.                                                                     |
| 21.   | <i>deactivate_ros_interface</i>         | Deactivate the ROS interface.                                                                          |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                               | Action                                                                                                            |
|-------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 22.   | <i>deactivate_servo_interface</i>           | Deactivate the Servo interface.                                                                                   |
| 23.   | <i>delete_pose_in_DB</i>                    | Deletes an existing pose in the DB. If the pose is a frame with children, then the children are properly handled. |
| 24.   | <i>disable_collision_detection</i>          | Disable collision detection on the robot.                                                                         |
| 25.   | <i>disable_friction_mode</i>                | To turn off friction mode (smooth free drive mode).                                                               |
| 26.   | <i>disable_reflex</i>                       | Disable reflex after collision.                                                                                   |
| 27.   | <i>enable_collision_detection</i>           | Enable collision detection on the robot.                                                                          |
| 28.   | <i>enable_friction_mode</i>                 | To turn on friction mode (smooth free drive mode). Free drive mode needs to be activated beforehand.              |
| 29.   | <i>enable_reflex</i>                        | Enable reflex after collision.                                                                                    |
| 30.   | <i>encoder2rad</i>                          | Convert encoder impulses to joint angles in radians.                                                              |
| 31.   | <i>execute_closest_orthogonal_tcp_align</i> | Execute the closest orthogonal axes-aligned TCP alignment motion.                                                 |
| 32.   | <i>execute_exact_tcp_align</i>              | Execute the exact axes-aligned TCP alignment motion.                                                              |
| 33.   | <i>execute_program</i>                      | To run a program created from teach pendant's tree program creator                                                |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                      | Action                                                                  |
|-------|----------------------------------------------------|-------------------------------------------------------------------------|
| 34.   | <i>execute_z_axis_tcp_align</i>                    | Execute the z-axis-aligned TCP alignment motion.                        |
| 35.   | <i>executor</i>                                    | Execute already planned motion IDs.                                     |
| 36.   | <i>get_analog_input</i>                            | Retrieve the value of an analog input.                                  |
| 37.   | <i>get_analog_output</i>                           | Retrieve the value of an analog output.                                 |
| 38.   | <i>get_current_joint_angles</i>                    | Get current joint angles of the robot.                                  |
| 39.   | <i>get_current_joint_angles_with_timestamp</i>     | Get the current joint angles of the robot along with UTC timestamp.     |
| 40.   | <i>get_current_joint_torques</i>                   | Get the current joint torques of the robot.                             |
| 41.   | <i>get_current_joint_torques_with_timestamp</i>    | Get the current joint torques of the robot along with UTC timestamp.    |
| 42.   | <i>get_current_joint_velocities</i>                | Get current joint velocities of the robot.                              |
| 43.   | <i>get_current_joint_velocities_with_timestamp</i> | Get the current joint velocities of the robot along with UTC timestamp. |
| 44.   | <i>get_current_load_side_encoder_values</i>        | Get the current load side encoder ticks of the robot.                   |
| 45.   | <i>get_current_motor_side_encoder_values</i>       | Get the current motor side encoder ticks of the robot .                 |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                               | Action                                                                                                                  |
|-------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 46.   | <i>get_current_tool_cogs</i>                | Get the current tool center of gravity (COG) in XYZ directions(measured from robot's flange frame/tool mounting point). |
| 47.   | <i>get_current_tool_inertias</i>            | Get the current tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.                                                         |
| 48.   | <i>get_current_tool_mass</i>                | Get the current tool mass.                                                                                              |
| 49.   | <i>get_current_tool_properties</i>          | Get the current tool properties.                                                                                        |
| 50.   | <i>get_current_tool_rpy_offsets</i>         | Get the current tool roll, pitch, and yaw (RPY) offsets in radians.                                                     |
| 51.   | <i>get_current_tool_translation_offsets</i> | Get the current tool translation offsets in XYZ directions.                                                             |
| 52.   | <i>get_diagnostics</i>                      | Method to query current robot diagnostics                                                                               |
| 53.   | <i>get_digital_input</i>                    | Get the value of a digital input.                                                                                       |
| 54.   | <i>get_digital_output</i>                   | Get the value of a digital output.                                                                                      |
| 55.   | <i>get_doc</i>                              | Get the description and sample usage of the specified function.                                                         |
| 56.   | <i>get_encoder_offsets</i>                  | To retrieve the encoder offsets of a robot.                                                                             |
| 57.   | <i>get_errors</i>                           | Query the list of errors present on the robot.                                                                          |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                    | Action                                                                |
|-------|--------------------------------------------------|-----------------------------------------------------------------------|
| 58.   | <i>get_ext_digital_input</i>                     | Get the value of an external digital input.                           |
| 59.   | <i>get_ext_digital_output</i>                    | Get the value of an external digital output.                          |
| 60.   | <i>get_flange_pose</i>                           | Get the current flange pose in XYZRPY format.                         |
| 61.   | <i>get_flange_pose_quaternion</i>                | Get the current flange pose in XYZwxyz format.                        |
| 62.   | <i>get_flange_pose_quaternion_with_timestamp</i> | Get the current flange pose in XYZwxyz format with UTC timestamp.     |
| 63.   | <i>get_flange_pose_with_timestamp</i>            | Get the current tool flange pose in XYZRPY format with UTC timestamp. |
| 64.   | <i>get_io_configuration</i>                      | Get the number of available IOs.                                      |
| 65.   | <i>get_io_status</i>                             | Get all the available IOs and it's values                             |
| 66.   | <i>get_joint_acceleration</i>                    | Get the current acceleration value for joint motion.                  |
| 67.   | <i>get_joint_speed</i>                           | Get the current speed value for joint motion.                         |
| 68.   | <i>get_linear_acceleration</i>                   | Get the current acceleration value for linear motion.                 |
| 69.   | <i>get_linear_speed</i>                          | Get the current speed value for linear motion.                        |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                       | Action                                                                                                                        |
|-------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 70.   | <i>get_load_side_encoder_values_with_timestamp</i>  | Get the current load side encoder ticks of the robot along with UTC timestamp.                                                |
| 71.   | <i>get_motor_side_encoder_values_with_timestamp</i> | Get the current motor side encoder ticks of the robot along with UTC timestamp.                                               |
| 72.   | <i>get_override</i>                                 | Get the current override value.                                                                                               |
| 73.   | <i>get_point</i>                                    | Retrieves a point from the database based on the given name.                                                                  |
| 74.   | <i>get_point_names</i>                              | Retrieves a list of names of all saved points in the database.                                                                |
| 75.   | <i>get_pose_id_from_name</i>                        | Get the id of the pose in DB from the corresponding name saved.                                                               |
| 76.   | <i>get_pose_in_parent_frame_from_DB</i>             | Retrieves the pose for the input_str and converts it from world frame to its parent frame which is returned by this function. |
| 77.   | <i>get_pose_in_reference_frame_from_DB</i>          | Gets the pose for the {pose_id} from DB, which is saved in world frame, and transforms it w.r.t the {reference_frame_id}.     |
| 78.   | <i>get_pose_in_world_frame_from_DB</i>              | Gets the pose in world frame for the input_str from DB.                                                                       |
| 79.   | <i>get_program_names</i>                            | Retrieves list of program names from the robot's database                                                                     |
| 80.   | <i>get_reference_frame</i>                          | Retrieve the coordinates and orientations of a user-defined reference frame.                                                  |
| 81.   | <i>get_reference_frame_with_offset</i>              | Retrieve a reference frame with an applied offset to its position.                                                            |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                 | Action                                                                                                         |
|-------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 82.   | <i>get_register_bytes</i>                     | Returns a python “bytes” object from a T2O or O2T register specified by its tag name and index range           |
| 83.   | <i>get_register_values</i>                    | Returns values from a T2O or O2T register specified by its tag name and index range from first to last element |
| 84.   | <i>get_safety</i>                             | No description!                                                                                                |
| 85.   | <i>get_selected_fts_data</i>                  | Retrieves the selected external fts from the robot’s database.                                                 |
| 86.   | <i>get_selected_tool_data</i>                 | Retrieves the selected tool data from the robot’s database.                                                    |
| 87.   | <i>get_selected_tool_name</i>                 | Retrieves the selected tool name from the robot’s database.                                                    |
| 88.   | <i>get_servo_trajectory_scaling_factor</i>    | Method to get the trajectory scaling factor from ServoJ                                                        |
| 89.   | <i>get_tcp_force</i>                          | Get the current TCP (Tool Center Point) force values of the robot.                                             |
| 90.   | <i>get_tcp_pose</i>                           | to get the current tcp pose in XYZRPY format                                                                   |
| 91.   | <i>get_tcp_pose_quaternion</i>                | to get the current tcp pose in XYZwxyz format                                                                  |
| 92.   | <i>get_tcp_pose_quaternion_with_timestamp</i> | Get the current tcp pose in XYZwxyz format along with UTC timestamp                                            |
| 93.   | <i>get_tcp_pose_with_timestamp</i>            | Get the current robot’s TCP pose in XYZRPY format along with UTC timestamp.                                    |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                  | Action                                                                                                                |
|-------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 94.   | <i>get_tcp_speed</i>           | Get the current speed of the robot's TCP (Tool Center Point).                                                         |
| 95.   | <i>get_tcp_torque</i>          | Get the current TCP (Tool Center Point) torque values of the robot.                                                   |
| 96.   | <i>get_tcp_wrench</i>          | Get the current TCP (Tool Center Point) wrench values,                                                                |
| 97.   | <i>get_tool_analog_input</i>   | Get the value of a tool analog input.                                                                                 |
| 98.   | <i>get_tool_digital_input</i>  | Get the value of a tool digital input.                                                                                |
| 99.   | <i>get_tool_digital_output</i> | Get the value of a tool digital output.                                                                               |
| 100.  | <i>get_tools</i>               | Retrieves list of tools from the robot's database.                                                                    |
| 101.  | <i>grasp</i>                   | To grasp the object with the attached gripper                                                                         |
| 102.  | <i>gripper</i>                 | Method to control the gripper attached to the robot                                                                   |
| 103.  | <i>init_program</i>            | To allow running motion commands, without this, the planning of motion will be blocked and no motion will take place. |
| 104.  | <i>io</i>                      | Access and manipulate (output) I/O values on the robot.                                                               |
| 105.  | <i>is_collision_enabled</i>    | Check if reflex after collision is enabled for the robot.                                                             |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                          | Action                                                                                                                     |
|-------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 106.  | <i>is_free_drive_mode_enabled</i>      | Get the status of the robot free drive mode.                                                                               |
| 107.  | <i>is_reflex_enabled</i>               | Check if reflex after collision is enabled for the robot.                                                                  |
| 108.  | <i>is_robot_in_automatic_mode</i>      | Method to check whether the robot is in automatic mode or not                                                              |
| 109.  | <i>is_robot_in_collision</i>           | Check if the robot is currently in collision.                                                                              |
| 110.  | <i>is_robot_in_semi_automatic_mode</i> | Method to check whether the robot is in semi automatic mode or not                                                         |
| 111.  | <i>is_robot_in_simulation</i>          | Method to check whether the robot is in simulation or not                                                                  |
| 112.  | <i>is_robot_in_teach_mode</i>          | Method to check whether the robot is in teach mode or not                                                                  |
| 113.  | <i>jog</i>                             | Method to jog programatically. This needs to be used in conjunction with <i>turn_on_jog</i> , <i>turn_off_jog</i> methods. |
| 114.  | <i>list_methods</i>                    | To list the available functions in the API                                                                                 |
| 115.  | <i>list_recorded_path</i>              | To list the recorded path using ZeroG.                                                                                     |
| 116.  | <i>lock_cartesian_axes</i>             | Lock specific Cartesian axes while in free drive mode.                                                                     |
| 117.  | <i>lock_joint_axes</i>                 | Lock specific joint axes while in free drive mode.                                                                         |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                        | Action                                                                                                                        |
|-------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 118.  | <i>motion_status</i>                 | Query the motion status of the robot.                                                                                         |
| 119.  | <i>move_circular</i>                 | To move the robot in a circular path across the given poses.                                                                  |
| 120.  | <i>move_composite</i>                | To move the robot in the specified linear and circular motion combinations.                                                   |
| 121.  | <i>move_joint</i>                    | To move the robot to specified joint configuration in joint space.                                                            |
| 122.  | <i>move_joint_relative</i>           | To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.     |
| 123.  | <i>move_linear</i>                   | To move the robot to specified poses in Cartesian/Task space.                                                                 |
| 124.  | <i>move_linear_relative</i>          | To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion. |
| 125.  | <i>move_pose_to_new_parent_frame</i> | Method to move the input pose(point/frame), which is defined with respect to a parent frame, to a new parent frame.           |
| 126.  | <i>move_recorded_path</i>            | To move the robot in a pre-recorded path.                                                                                     |
| 127.  | <i>movelinear_online</i>             | Method to do Servo in Cartesian Space                                                                                         |
| 128.  | <i>notify_error</i>                  | To notify an error message to teach pendant .                                                                                 |
| 129.  | <i>notify_info</i>                   | To notify a info message to teach pendant.                                                                                    |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                  | Action                                                                                                                          |
|-------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 130.  | <i>pause</i>                   | To pause the robot's motion.                                                                                                    |
| 131.  | <i>plan_move_circular</i>      | Plan circular motion.                                                                                                           |
| 132.  | <i>plan_move_composite</i>     | This method takes the following arguments/keyword arguments:                                                                    |
| 133.  | <i>plan_move_joint</i>         | This method takes the following arguments/keyword arguments:                                                                    |
| 134.  | <i>plan_move_linear</i>        | To plan a move linear path across the given poses.                                                                              |
| 135.  | <i>plan_move_recorded_path</i> | To plan the given pre-recorded path.                                                                                            |
| 136.  | <i>plan_realtime_wait</i>      | This function can be used to add a wait time to the motion buffer. This is at this time planned but not added to the execution. |
| 137.  | <i>power_off</i>               | To power off the robot.                                                                                                         |
| 138.  | <i>power_on</i>                | To power on the robot.                                                                                                          |
| 139.  | <i>program_status</i>          | Query the program status of the robot.                                                                                          |
| 140.  | <i>quaternion_to_rpy</i>       | Convert a quaternion representation to roll-pitch-yaw (RPY) angles.                                                             |
| 141.  | <i>rad2encoder</i>             | Convert joint angles in radians to encoder impulses.                                                                            |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                        | Action                                                                                                                   |
|-------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 142.  | <i>read_safeio</i>                                   | Read the values of configurable/safe Input/Outputs.                                                                      |
| 143.  | <i>realtime_wait</i>                                 | This function can be used, to add a wait time to the motion buffer.                                                      |
| 144.  | <i>release</i>                                       | To release the grasped object                                                                                            |
| 145.  | <i>remove_load</i>                                   | Remove the load which was set in set_load by setting the load to NoLoad.                                                 |
| 146.  | <i>replicate_list_poses_in_new_parent_frame</i>      | Method to replicate the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame. |
| 147.  | <i>replicate_pose_in_new_parent_frame</i>            | Method to replicate the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame. |
| 148.  | <i>replicate_pose_in_new_parent_frame_and_delete</i> | Replicates the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame.          |
| 149.  | <i>reset_axes_lock</i>                               | Reset all joint and Cartesian axis locks in free drive mode.                                                             |
| 150.  | <i>reset_collision</i>                               | Reset the collision state of the robot.                                                                                  |
| 151.  | <i>reset_control</i>                                 | To restart the control software running on the robot. Equivalent to reset control option from Teach pendant              |
| 152.  | <i>reset_errors</i>                                  | Method to reset the errors on the robot                                                                                  |
| 153.  | <i>reset_led_mode</i>                                | Reset the LED mode to its default state.                                                                                 |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                       | Action                                                                                                                                |
|-------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 154.  | <i>rpy_to_quaternion</i>            | Convert roll-pitch-yaw (RPY) angles to a quaternion representation.                                                                   |
| 155.  | <i>save_new_pose_in_DB</i>          | Saves a new pose in world frame in DB. Converts the input pose that is in parent to world frame prior to saving.                      |
| 156.  | <i>save_or_update_pose_in_DB</i>    | Checks if a pose with the same name as the input already exists in DB. If exists, then update the existing pose else create new pose. |
| 157.  | <i>servo_j</i>                      | Method to do Servo in Joint Space                                                                                                     |
| 158.  | <i>servo_x</i>                      | Method to do Servo in Cartesian Space                                                                                                 |
| 159.  | <i>set_analog_output</i>            | Set the analog outputs of the control box.                                                                                            |
| 160.  | <i>set_current_position_as_zero</i> | Sets the current joint position of the given axis to zero.                                                                            |
| 161.  | <i>set_dblout_register</i>          | Writes values into an element range of the DBLOUT register                                                                            |
| 162.  | <i>set_digital_output</i>           | Set the digital outputs of the control box.                                                                                           |
| 163.  | <i>set_ext_digital_output</i>       | Set the digital outputs of the external I/O module.                                                                                   |
| 164.  | <i>set_intout_register</i>          | Writes values or octets into an element range of the INTOUT register                                                                  |
| 165.  | <i>set_joint_acceleration</i>       | Set the acceleration value for joint motion.                                                                                          |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                   | Action                                                                                                                                                       |
|-------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 166.  | <i>set_joint_speed</i>          | Set the speed value for joint motion.                                                                                                                        |
| 167.  | <i>set_led_mode</i>             | Turn on the LED of the robot using the specified mode, a list of colors, and frequency.                                                                      |
| 168.  | <i>set_linear_acceleration</i>  | Set the acceleration value for linear motion.                                                                                                                |
| 169.  | <i>set_linear_speed</i>         | Set the speed value for linear motion.                                                                                                                       |
| 170.  | <i>set_load</i>                 |                                                                                                                                                              |
| 171.  | <i>set_opcua_msg</i>            | Send a message to the OPC UA server running on the control box.                                                                                              |
| 172.  | <i>set_override</i>             | Set the override value.                                                                                                                                      |
| 173.  | <i>set_safety</i>               | No description!                                                                                                                                              |
| 174.  | <i>set_tool</i>                 | Notify the gripper/tool change to software components after the physical change.                                                                             |
| 175.  | <i>set_tool_digital_output</i>  | Set the target value on the specified tool digital output.                                                                                                   |
| 176.  | <i>set_tool_digital_outputs</i> | Set digital outputs of the tool.                                                                                                                             |
| 177.  | <i>stop</i>                     | To stop the robot's motion and to terminate the script execution. This needs to be added at the end of the script(only once) for a proper script termination |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                        | Action                                                                                                                                                              |
|-------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 178.  | <i>stop_movelinear_online</i>        | Method to stop MoveLinearOnline motion if running                                                                                                                   |
| 179.  | <i>switch_to_automatic_mode</i>      | Switch the robot to Automatic mode.                                                                                                                                 |
| 180.  | <i>switch_to_real</i>                | Switches the robot's running context to real mode.                                                                                                                  |
| 181.  | <i>switch_to_semi_automatic_mode</i> | Switch the robot to Semi Automatic mode.                                                                                                                            |
| 182.  | <i>switch_to_simulation</i>          | Switches the robot's running context to simulation mode.                                                                                                            |
| 183.  | <i>switch_to_teach_mode</i>          | Switch the robot to Teach mode.                                                                                                                                     |
| 184.  | <i>tare_fts</i>                      | Set the property for taring force torque sensor property to true when called and clears the                                                                         |
| 185.  | <i>transform_pose_to_reference</i>   | Transforms the pose data in {pose_json_in_parent}, which is in its parent frame {old_reference_id}, to a target reference frame contained in {pose_json_in_parent}. |
| 186.  | <i>turn_off_free_drive_mode</i>      | To turn off free drive mode                                                                                                                                         |
| 187.  | <i>turn_off_jog</i>                  | Method to disable jogging programatically. This needs to be used in conjunction with jog, turn_on_jog methods.                                                      |
| 188.  | <i>turn_on_free_drive_mode</i>       | To turn on free drive mode                                                                                                                                          |
| 189.  | <i>turn_on_jog</i>                   | Method to enable jogging programatically. This needs to be used in conjunction with jog, turn_off_jog methods.                                                      |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                      | Action                                                                                                                                         |
|-------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 190.  | <i>unpause</i>                                     | To unpause the robot's motion.                                                                                                                 |
| 191.  | <i>update_current_tool_parameters</i>              | Update the current gripper/tool parameters in the tool list and the selected tool properties.                                                  |
| 192.  | <i>update_pose_in_DB</i>                           | Updates an existing pose in DB by converting the input which is defined in its parent frame to world frame. This method shall take care of the |
| 193.  | <i>update_tool_parameters</i>                      | Update the gripper/tool parameters in the tool list with a given tool_name.                                                                    |
| 194.  | <i>wait</i>                                        | Wait for a specified signal.                                                                                                                   |
| 195.  | <i>wait_for_analog_input</i>                       | Wait for an analog input signal to match the expected value.                                                                                   |
| 196.  | <i>wait_for_digital_input</i>                      | Wait for a digital input signal to match the expected value.                                                                                   |
| 197.  | <i>wait_for_digital_input_timer_off_delay</i>      | Wait for a given delay and returns after the specified digital input signal reaches low.                                                       |
| 198.  | <i>wait_for_digital_input_timer_on_delay</i>       | Wait for a given delay and returns after the specified digital input signal reaches high.                                                      |
| 199.  | <i>wait_for_tool_analog_input</i>                  | Wait for a tool-specific analog input signal to match the expected value.                                                                      |
| 200.  | <i>wait_for_tool_digital_input</i>                 | Wait for a tool-specific digital input signal to match the expected value.                                                                     |
| 201.  | <i>wait_for_tool_digital_input_timer_off_delay</i> | Wait for a given delay and return after the specified tool digital input signal reaches low.                                                   |

continues on next page

Table 1 – continued from previous page

| S.No. | Functionality                                     | Action                                                                                        |
|-------|---------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 202.  | <i>wait_for_tool_digital_input_timer_on_delay</i> | Wait for a given delay and return after the specified tool digital input signal reaches high. |

## OFFLINE SIMULATION SETUP

The offline simulation setup enables offline programming and simulation of robot programs.

The offline simulation software is provided as a virtual machine (VM) image, tested exclusively on VirtualBox 7.0 and above.

Using this VM image on other virtualization software may cause unexpected issues.

### 2.1 System Requirements

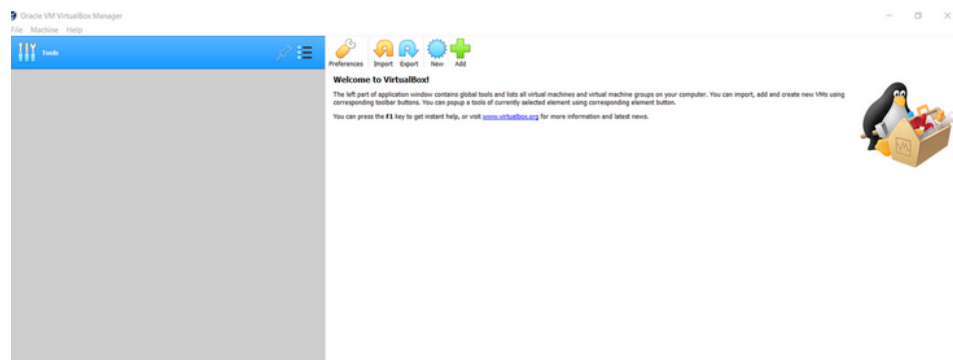
To ensure optimal performance, the following system requirements are recommended:

- **Hard Drive Space:** A minimum of 30GB of free space on your hard drive is required.
- **RAM:** You should have at least 10GB of RAM to ensure smooth performance.
- **Processor:** An 8-12 core processor is recommended to handle the virtual machine efficiently.

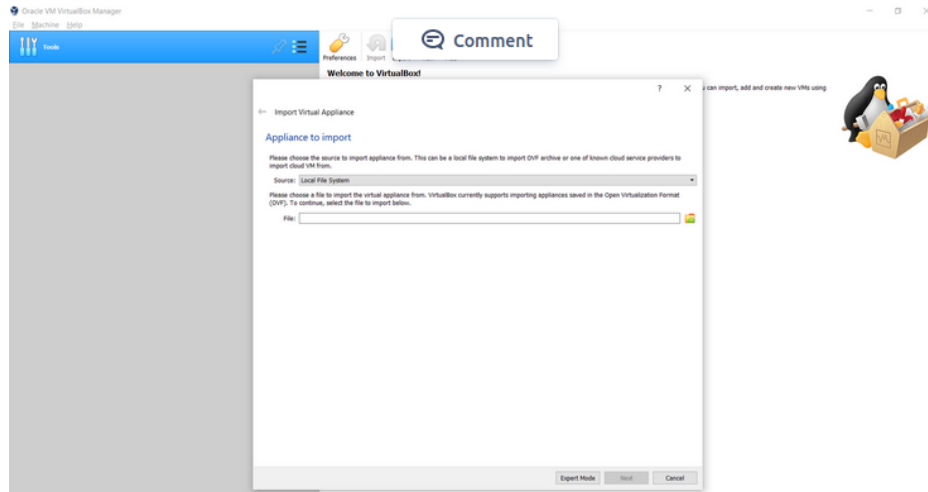
Meeting these system requirements will help ensure that your virtual machine runs smoothly without performance issues.

### 2.2 Installation Guidelines for Windows

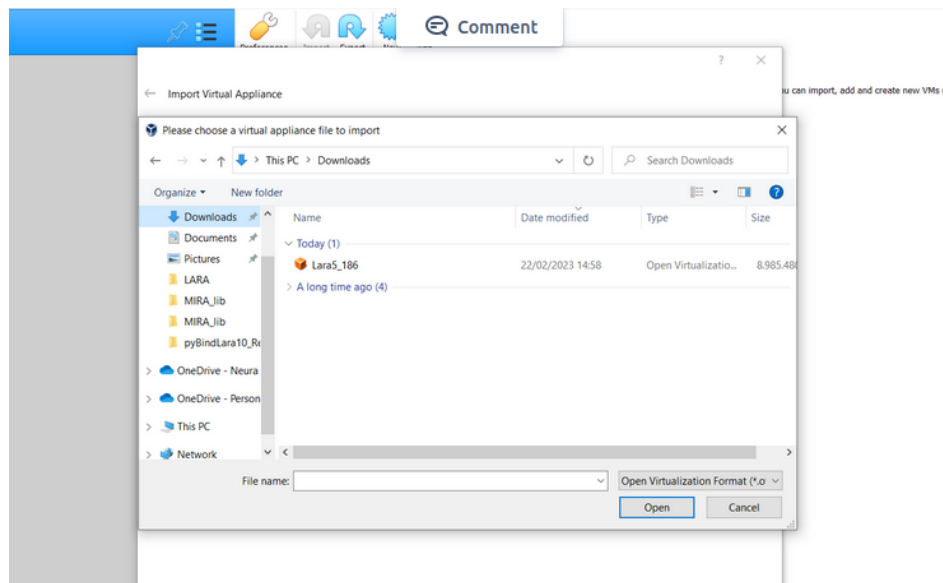
1. Download the Windows-compatible version of VirtualBox from [virtualbox.org](https://www.virtualbox.org) and install it.
2. Download the offline simulation software VM image (LARA5).
3. Open the VirtualBox GUI by navigating to the start menu. You should see a window similar to the one below:



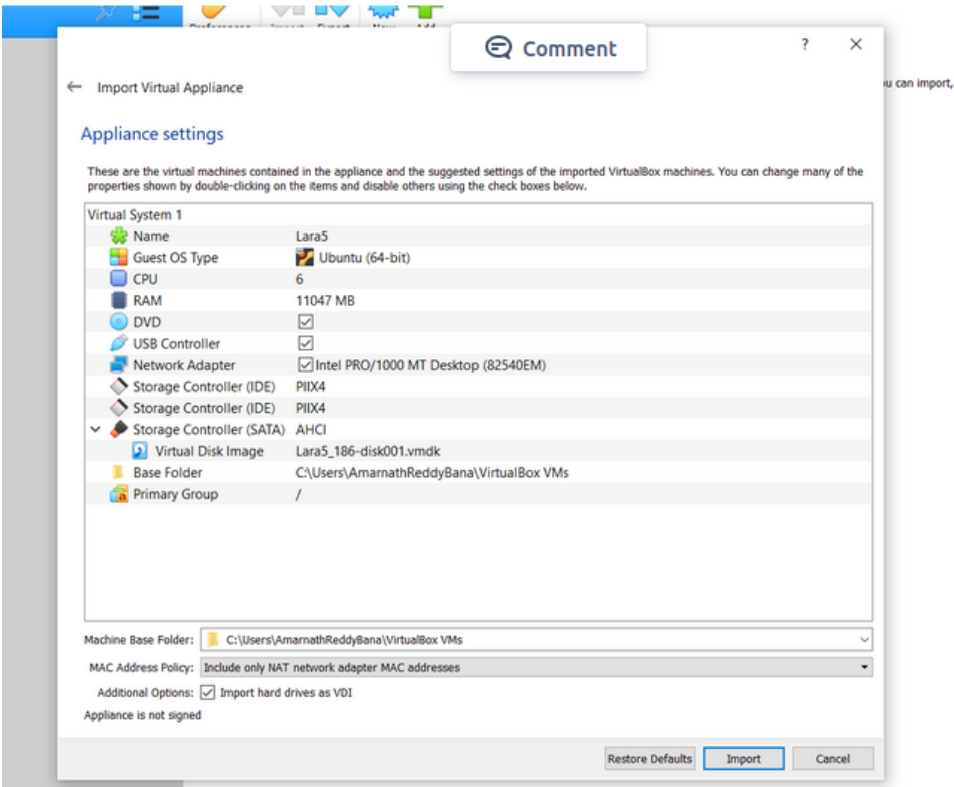
4. Click on the Import Icon. This action will open a second window for importing the Virtual Appliance.



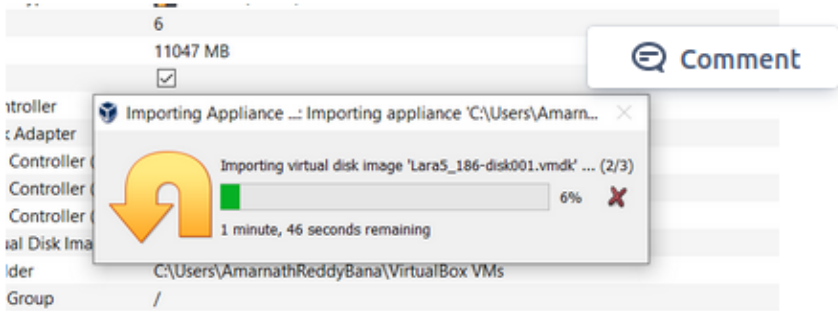
- Click the folder icon beside **File** to open a browser window. Navigate to the location of the downloaded .ova file (e.g., *Lara5\_186.ova*), select the file, and click **Open**.



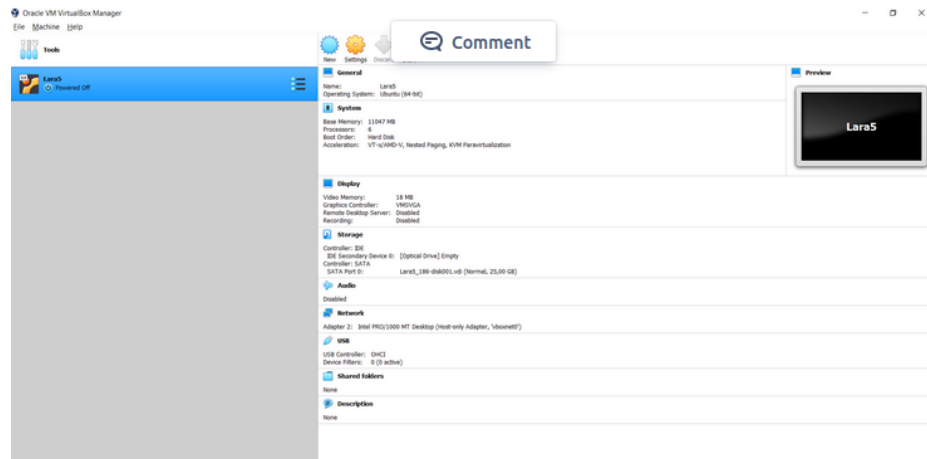
- Click **Next** to open the configuration settings. It is recommended to retain the default configuration settings. Press **Restore Defaults** before proceeding with the import.



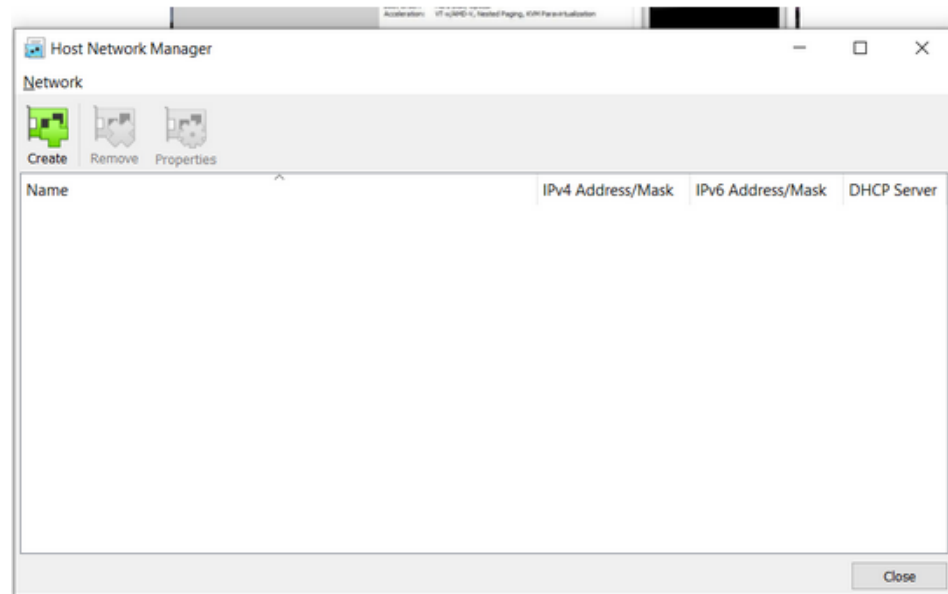
7. Click **Import**, and an acknowledgment window displaying the elapsed time will appear. Allow the process to complete.



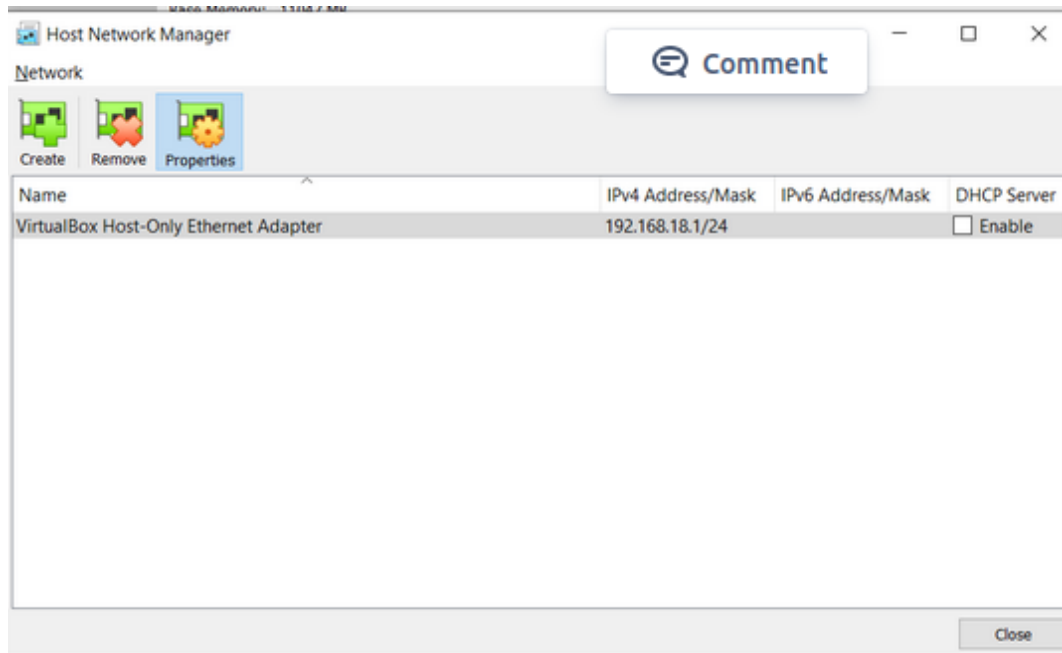
8. After this process is complete, the VirtualBox GUI should resemble the image below:



9. To configure the network adapter, navigate to **File > Host Network Manager**. A window will appear; click **Create**.



10. Once the adapter is created, it should display **VirtualBox Host-Only Ethernet Adapter**. Enable the DHCP Server by clicking the **Enable** checkbox under **DHCP Server** in the top right corner. Next, click on **Properties**.

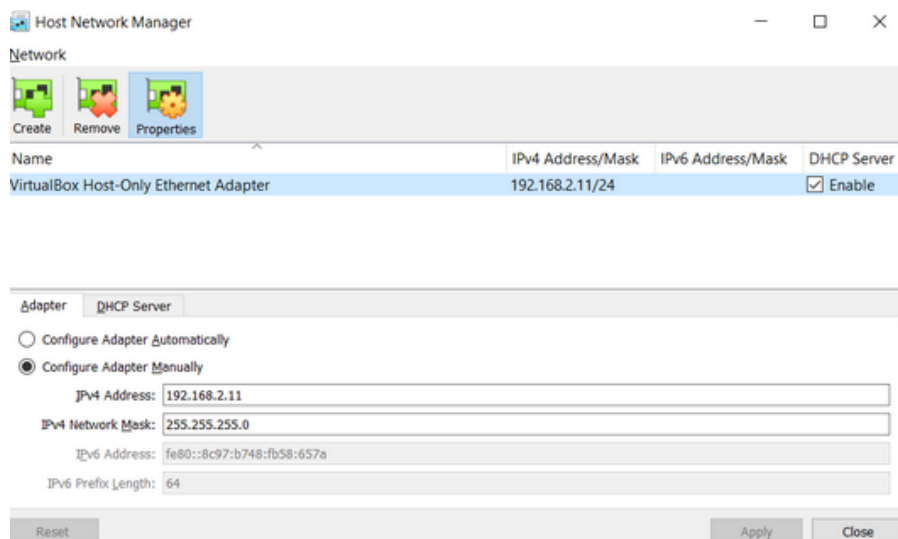


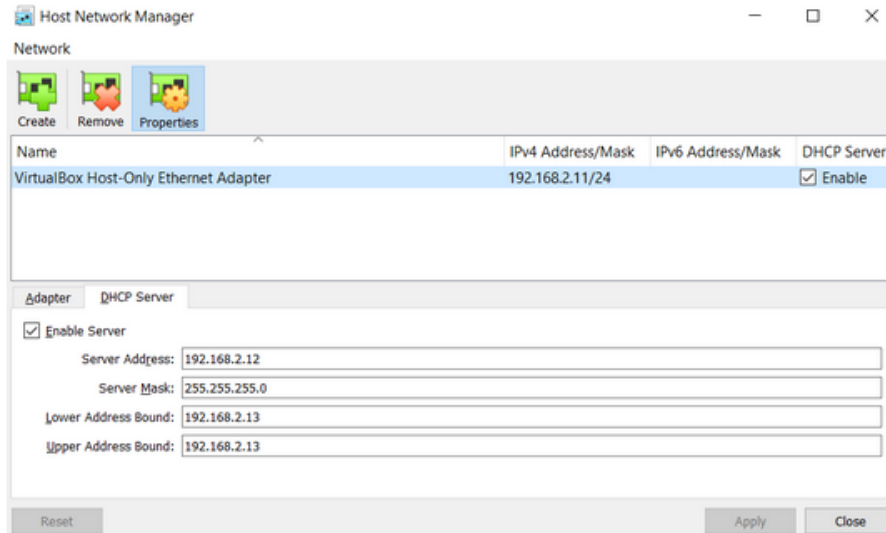
11. Choose the **Configure Adapter Manually** option and adjust the fields as follows:

- **IPv4 Address:** 192.168.2.11
- **IPv4 Network Mask:** 255.255.255.0

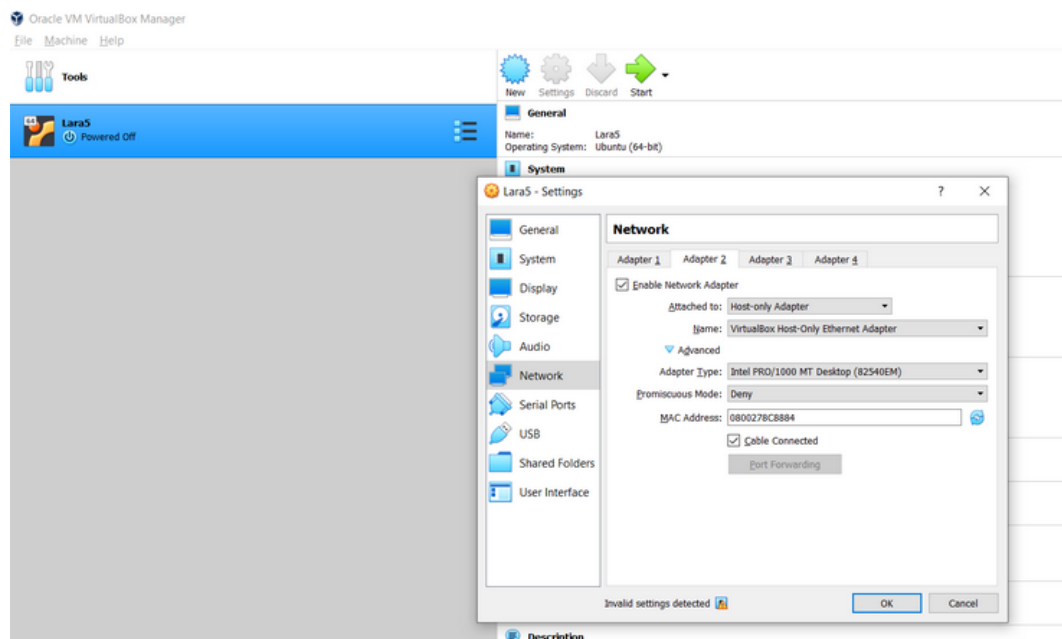
In the DHCP Server settings, configure the following:

- Click on **Enable Server**.
- **Server Address:** 192.168.2.12
- **Server Mask:** 255.255.255.0
- **Lower Address Bound:** 192.168.2.13
- **Upper Address Bound:** 192.168.2.13

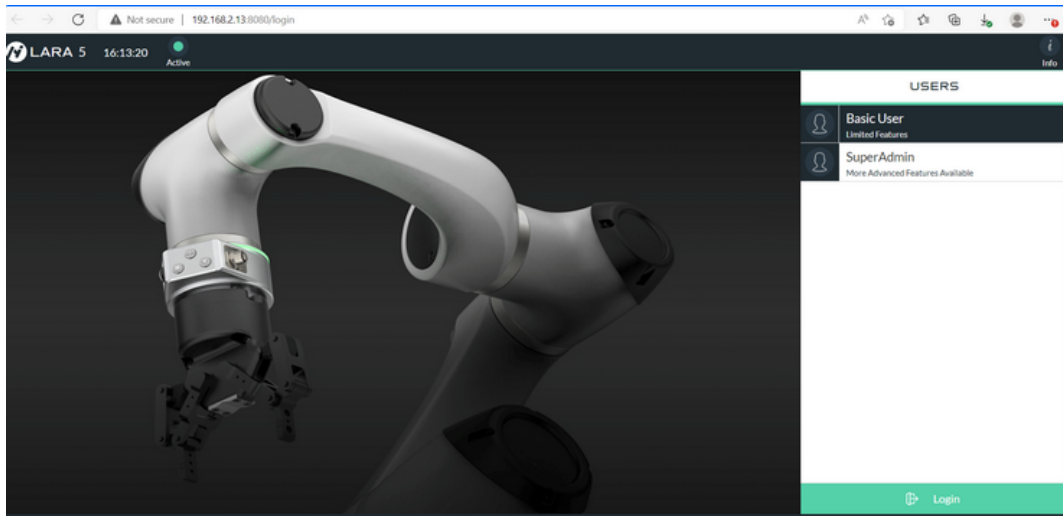




12. Click on **Settings** -> **Network** -> **Adapter 1**. If **Enable Network Adapter** is checked, uncheck it first. Then, click on **Adapter 2** and check **Enable Network Adapter**. From the drop-down menu of **Attached to**, select **Host-only Adapter**. In the **Name** field, choose **VirtualBox Host-Only Ethernet Adapter**. The settings should appear as follows:

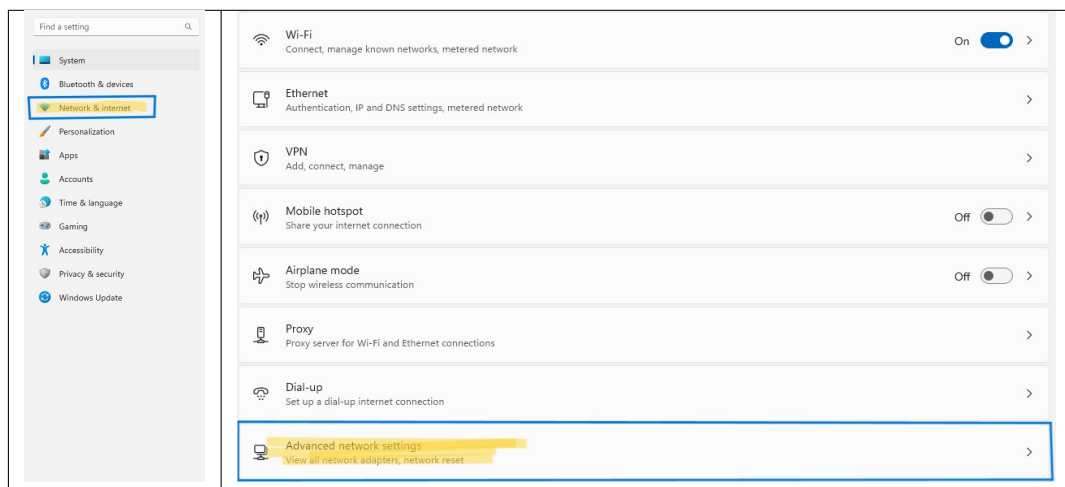


13. The configuration of the offline simulation VM image is complete. Start **Lara5** and run it for 30 seconds. To access the GUI, open Chrome and enter <http://192.168.2.13:8080/login>. If all processes were successful, this address should load the GUI interface.

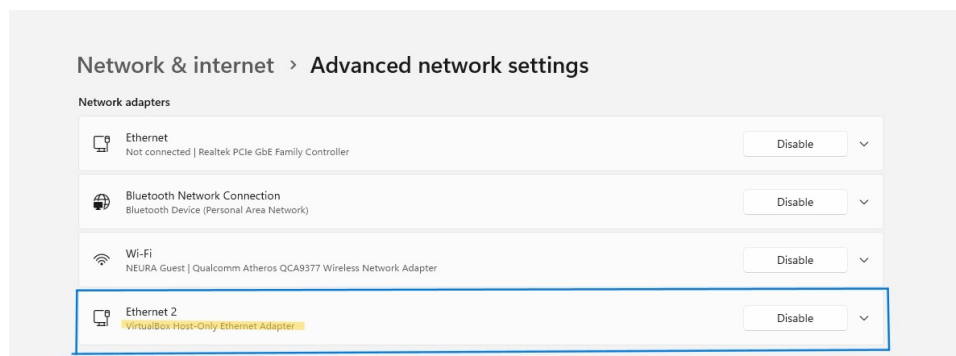


## 2.3 Troubleshooting Network

1. On your Windows PC, click on **Settings** → **Network & Internet** → **Advanced network settings**.



2. Click on **Advanced network settings**.



3. Verify the status of the Ethernet Adapter for VirtualBox in the **Advanced network settings**. If it is currently

enabled, click the **Enable** button to confirm. If it is already enabled by default, click the **Enable** button, wait for it to disable, pause for 3 seconds, and then re-enable it.

## 2.4 Installation Guidelines for Ubuntu

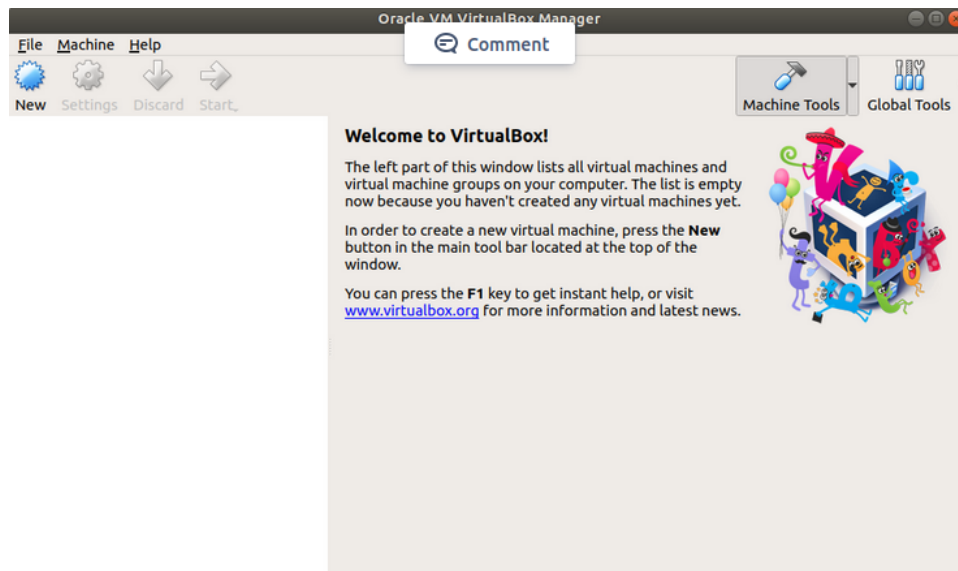
1. Install VirtualBox on Ubuntu using the following commands from the apt repositories:

```
sudo dpkg -i ~/path/virtualbox*deb
```

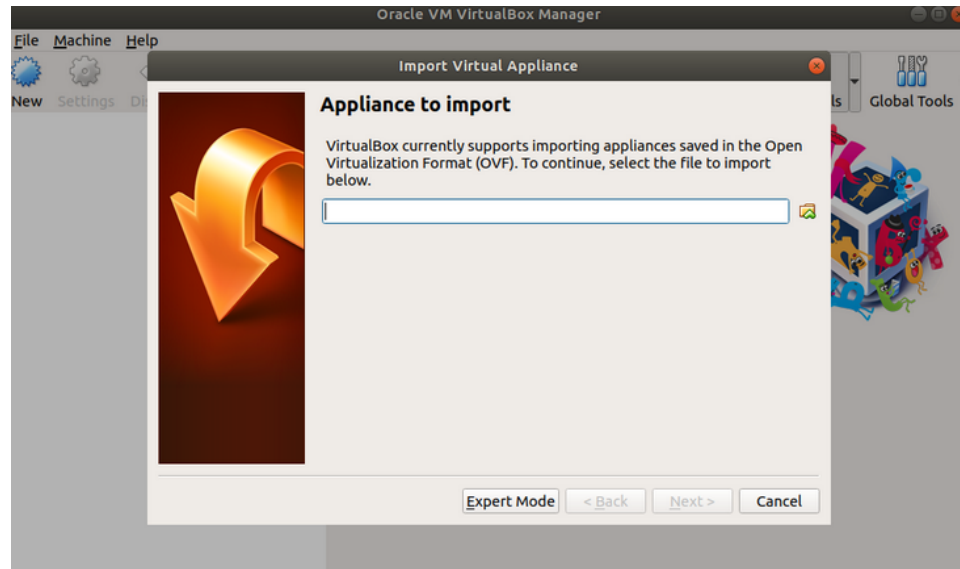
2. Download the offline simulation software VM image (LARA5).
3. To enable static IP allotment for VirtualBox in a custom subnet, execute the following commands:

```
sudo mkdir /etc/vbox  
sudo touch /etc/vbox/networks.conf  
echo "* 10.0.0.0/8 192.168.0.0/16" | sudo tee -a /etc/vbox/networks.conf
```

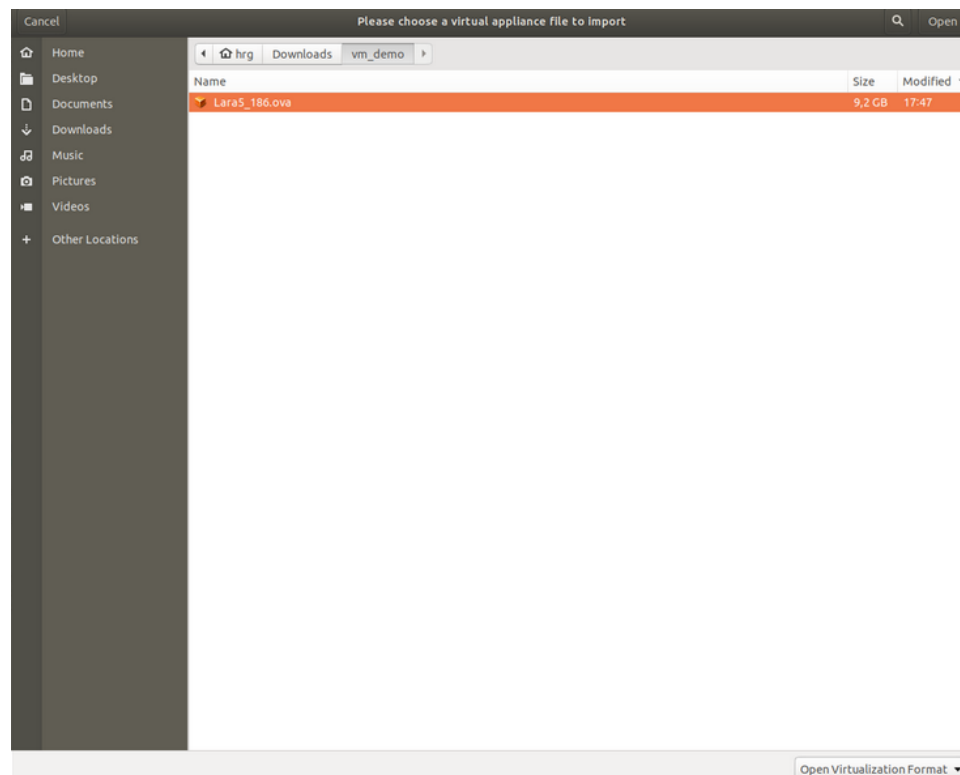
4. Open VirtualBox from the terminal. A window similar to the one below should appear:



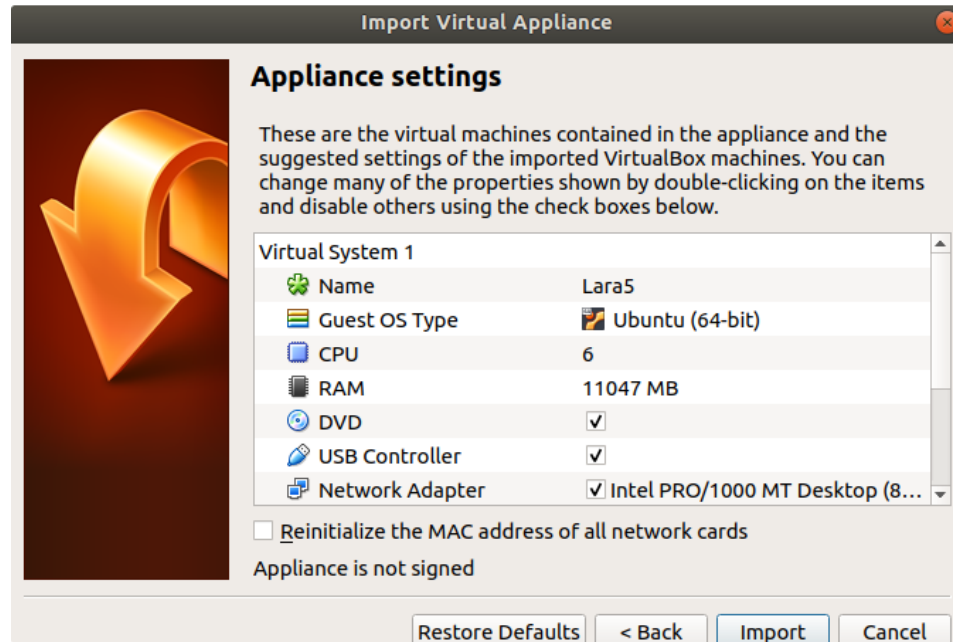
5. Click on the **Import Virtual Appliance from File** tab, and a second window for importing the Virtual Appliance will appear.



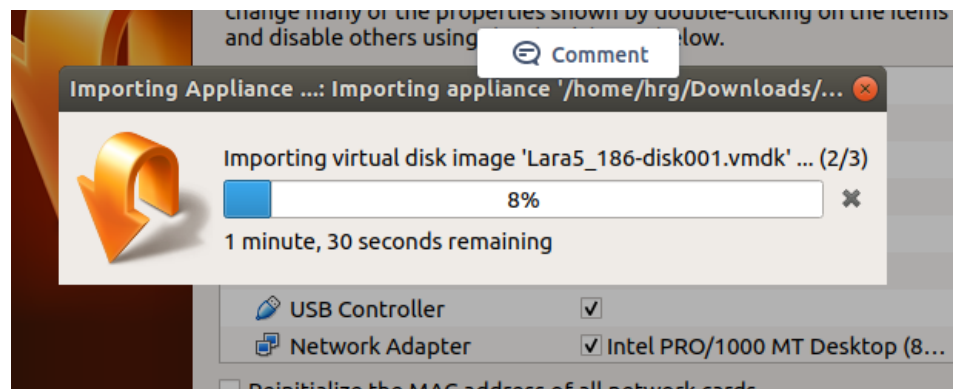
6. Click the folder icon beside **File**. This will open a browser window. Specify the location of the downloaded *.ova* file (e.g., *Lara5\_186.ova*), select the file, and click **Open**.



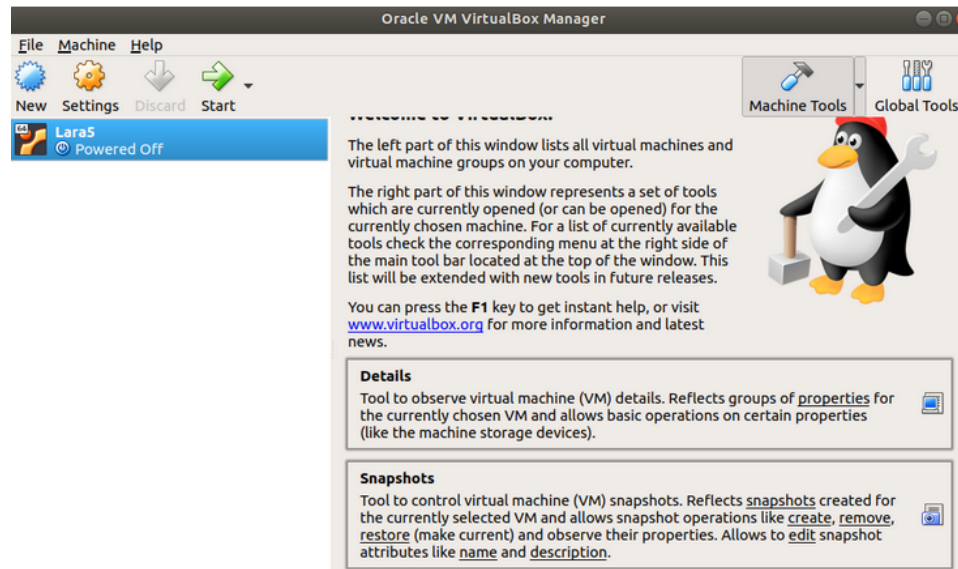
7. Click **Next**, and the configuration settings will appear. It is recommended to retain the default configuration settings. Press **Restore Defaults** before proceeding with the import



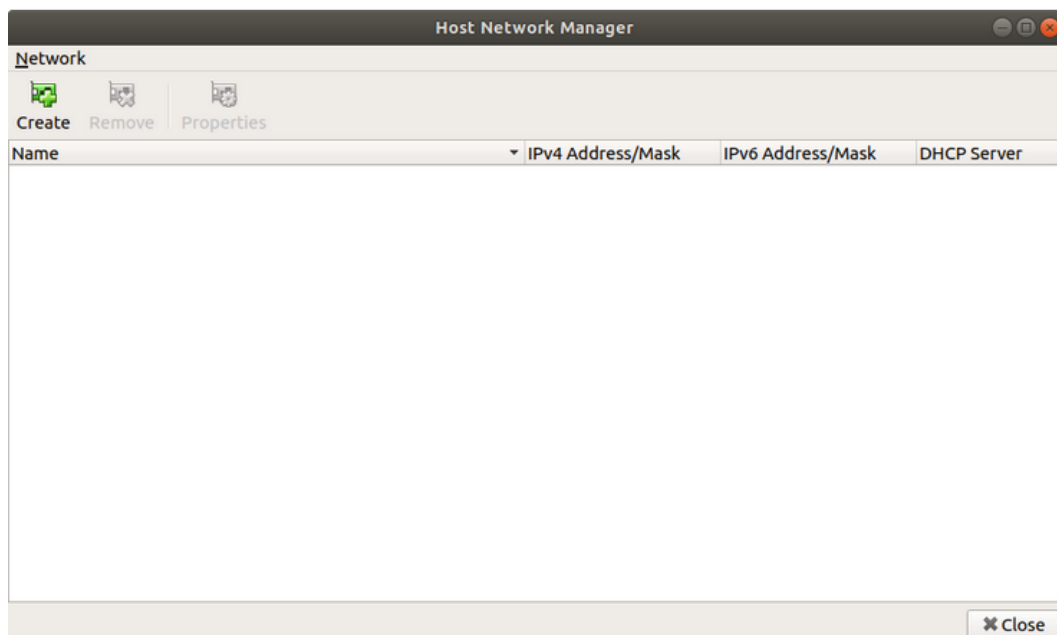
8. Click **Import**, and an acknowledgment window displaying the elapsed time will appear. Allow the process to complete.



9. After this process is complete, the VirtualBox GUI should resemble the image below:

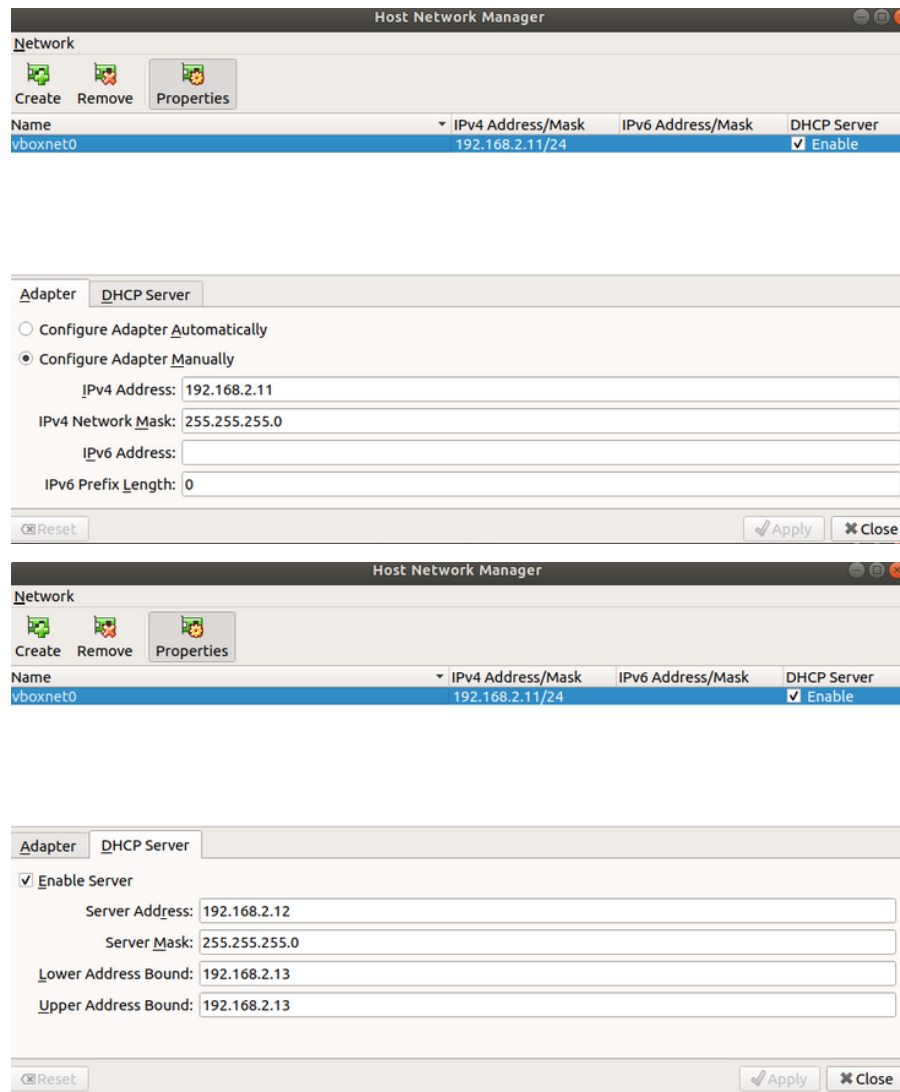


10. To configure the network adapter, navigate to **File > Host Network Manager**. A window will appear; click **Create**.

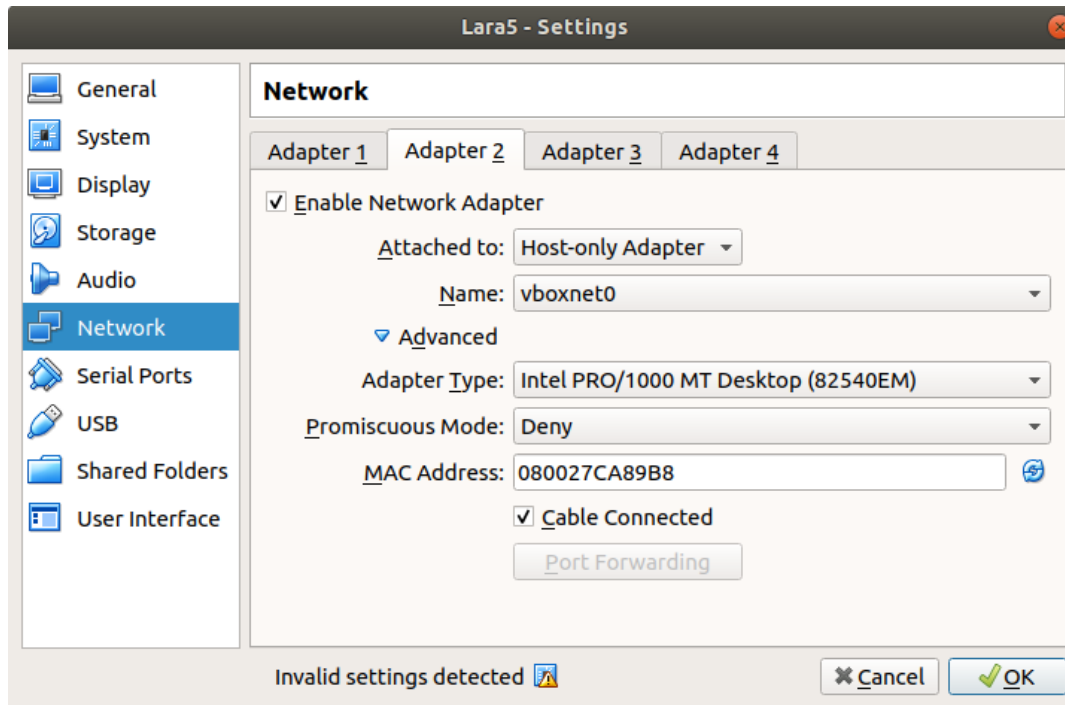


11. Once the adapter is created, it should display **vboxnet0**. Enable the DHCP Server by clicking the **Enable** checkbox under **DHCP Server** in the top right corner. Next, click on **Properties**.
12. Choose the **Configure Adapter Manually** option and adjust the fields as follows:
- **IPv4 Address:** 192.168.2.11
  - **IPv4 Network Mask:** 255.255.255.0
- In the DHCP Server settings, configure the following:
- **Server Address:** 192.168.2.12
  - **Server Mask:** 255.255.255.0
  - **Lower Address Bound:** 192.168.2.13

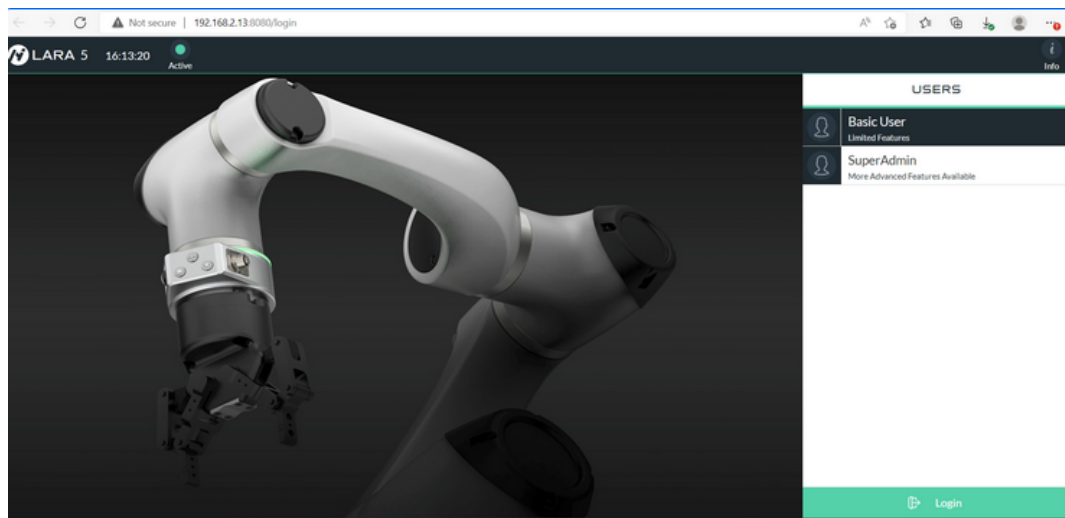
- **Upper Address Bound:** 192.168.2.13



- Click on **Settings** → **Network** → **Adapter 1**. If **Enable Network Adapter** is checked, uncheck it first. Then, click on **Adapter 2** and check **Enable Network Adapter**. From the drop-down menu of **Attached to**, select **Host-only Adapter**. In the **Name** field, select **VirtualBox Host-Only Ethernet Adapter**. The configuration should appear as follows:



14. The configuration of **Lara5** is complete. Start **Ubuntu Lara5** and run it for 30 seconds. To access the GUI, open Chrome and enter <http://192.168.2.13:8080/login>. If all processes were successful, this address should load the GUI interface.



## API INSTALLATION

### 3.1 Installation Instructions for Windows OS

#### 3.1.1 Python 3.8 Installation

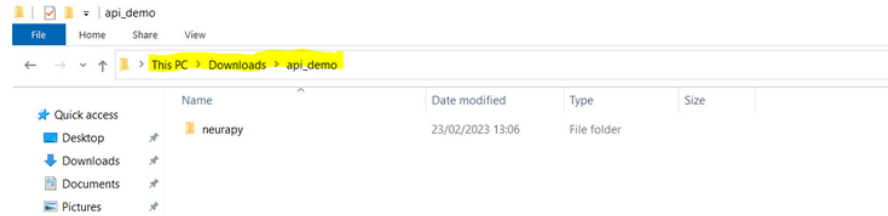
1. Download the Python 3.8 executable (.exe) file from the [official Python website](#).
2. Run the downloaded executable to begin the installation.
3. Ensure that “Add Python 3.8 to PATH” is checked before proceeding.
4. Select “Install Now” to complete the installation.



5. Close the setup once the installation is complete.

### 3.1.2 NeuraPy Installation

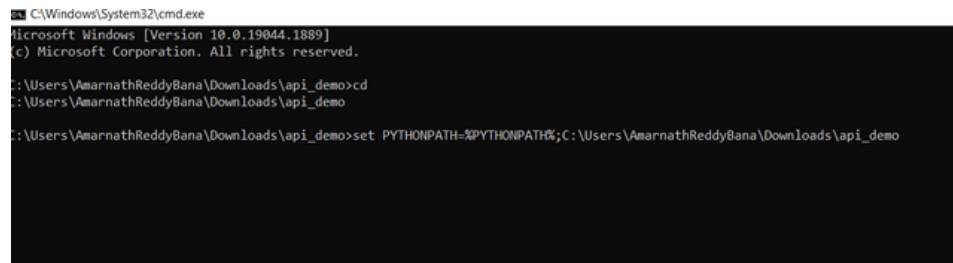
1. Download NeuraPy for Windows and navigate to the directory where it is saved.
2. Open the command prompt from this directory. You can do this by typing “cmd” in the address bar or pressing Alt+D, typing “cmd”, and then pressing Enter.



3. Install the required dependencies, *pywin32* and *prettytable*, using pip. Ensure that you have Internet connectivity for this step.

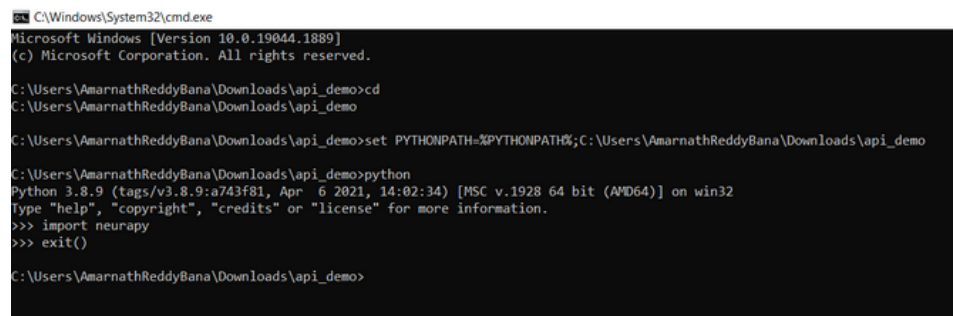
```
pip install pywin32 prettytable
```

4. Use the *cd* command in the command prompt to obtain the current location and append it to the Python path.



```
# open cmd from address bar
cd # to get the current location
set PYTHONPATH=%PYTHONPATH%;<location obtained from previous command>
```

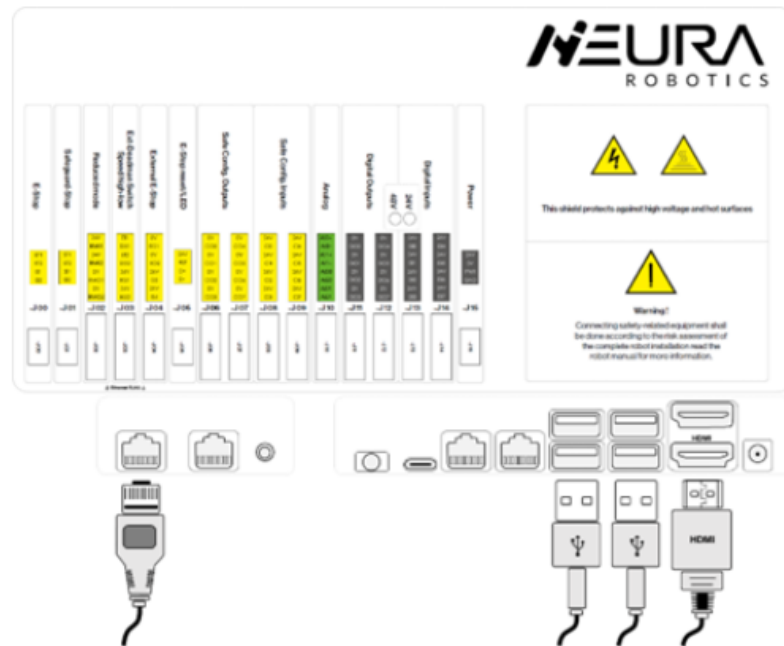
5. Open the python terminal from command prompt after setting the python path. The API can now be imported in the python console.



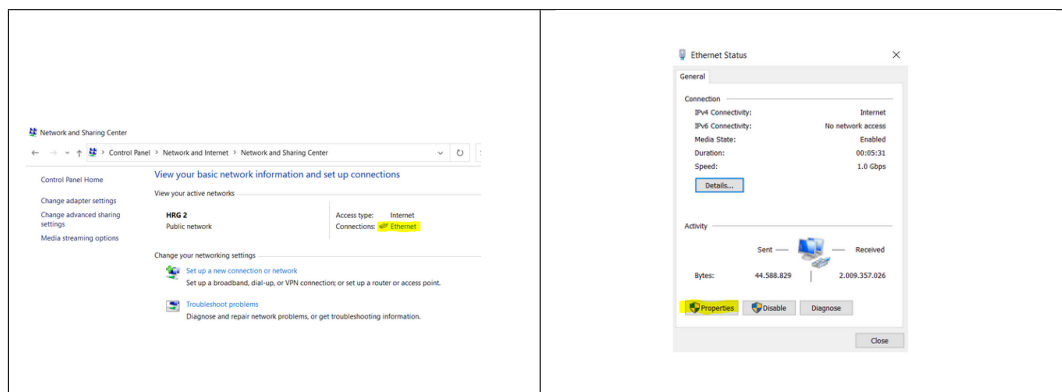
### 3.1.3 Network Configuration

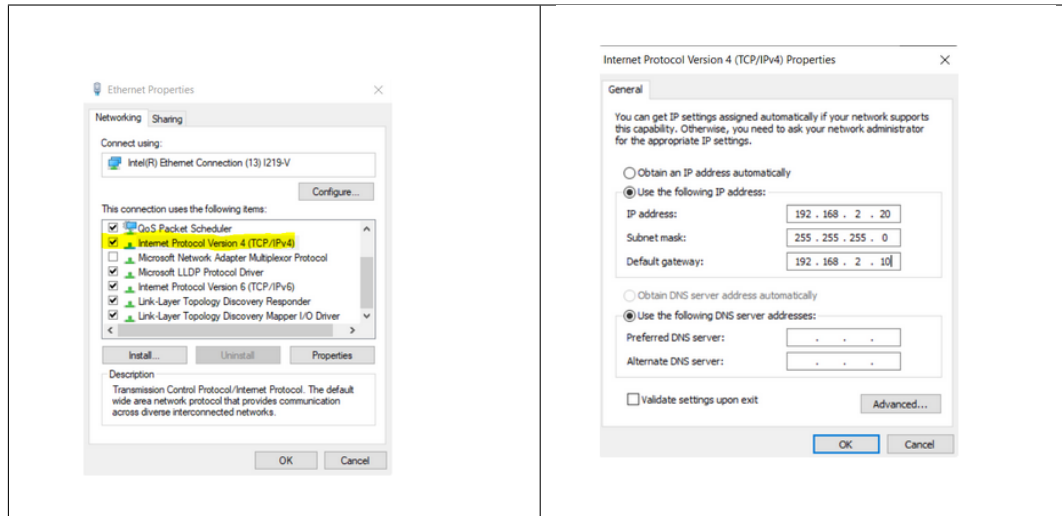
To configure the network for controlling the robot via an external machine connected through Ethernet, follow these steps:

1. Connect Ethernet to the Control Box:
  - Open the control box using the provided key.
  - Locate the Ethernet port on the lower left side of the control box. It will be labeled as Ethernet RJ45.
  - Connect one end of the Ethernet cable to the Ethernet port on the control box, and the other end to the Ethernet port on the external PC.



2. Control Panel → Network status and tasks → Ethernet → properties → ip4 setting





### 3.1.4 Configuring client Ip-address

This section is only import for those that run their communication on a non-default IP-address (anything but 192.168.2.13). If this is the case and the communication is set up on another IP-address e.g. 192.168.2.99 we need to configure it on the client side.

1. Set an environment variable called SOCKET\_ADDRESS to 192.168.2.99
  - Open a console by pressing the Windows-key and typing cmd
  - Choose run as administrator
  - type in the console: `setx SOCKET_ADDRESS="192.168.2.99"`

## 3.2 Installation Instructions for Ubuntu

**Note:** NeuraPy API is currently supported only for Ubuntu 18 and Ubuntu 20.

1. Check the Version of Ubuntu:

Open the terminal and type the following command:

```
lsb_release -a
```

2. Depending on the version of the ubuntu, download and navigate to DEP\_Deployment folder of the neurapy and execute `deploy_neurapy.sh` script located inside the downloaded folder with LARA5(respective robot name) argument like below

```
cd neurapy_ubuntu18 or neurapy_ubuntu20
. deploy_neurapy.sh LARA5 #respective robot name
```

### 3.2.1 Network Configuration

When the robot is controlled via an external machine connected through Ethernet, you need to set up the network on the external machine with the following configuration

netplan-enp2s0

Cancel Apply

Details Identity IPv4 IPv6 Security

**IPv4 Method**

☐ Automatic (DHCP) ☐ Link-Local Only

☒ Manual ☐ Disable

**Addresses**

| Address      | Netmask       | Gateway      |
|--------------|---------------|--------------|
| 192.168.2.20 | 255.255.255.0 | 192.168.2.10 |
|              |               |              |

**DNS** Automatic ☒

Separate IP addresses with commas

**Routes** Automatic ☒

| Address | Netmask | Gateway | Metric |
|---------|---------|---------|--------|
|         |         |         |        |

### 3.2.2 Installing an IDE Visual Studio Code (VSCode)

This step-by-step guide will walk you through the process of installing VSCode on your system, regardless of your technical expertise. 1. Pre-installation Preparation

Before we begin, ensure that your system meets the following requirements:

Operating System: Windows, macOS, or Linux. Internet Connection: A stable internet connection is required for downloading the installation files. Storage Space: Make sure you have enough free space on your hard drive for the installation.

#### 2. Downloading VSCode

Open your web browser (e.g., Chrome, Firefox). In the address bar, type <https://code.visualstudio.com/> and press Enter. You will be directed to the official Visual Studio Code website. Look for the download button and click on it. The website should automatically detect your operating system and provide the appropriate download link.

#### 3. Installing VSCode

For Windows:

Once the download is complete, locate the downloaded file (usually in your Downloads folder). Double-click on the downloaded file (ending with .exe) to start the installation process. Follow

the on-screen instructions in the installation wizard. You may be prompted to choose installation options. You can usually leave these at their default settings unless you have specific preferences. Click on “Install” or “Finish” to complete the installation process. VSCode is now installed on your Windows system!

For macOS:

Locate the downloaded file (usually in your Downloads folder). Double-click on the downloaded file (ending with .dmg) to mount the disk image. Drag the Visual Studio Code icon to the Applications folder to install it. Once the copying process is complete, eject the disk image. VSCode is now installed on your macOS system!

For Linux:

Depending on your Linux distribution, installation methods may vary. The following example demonstrates installation on Ubuntu using the Terminal. Open a Terminal window by pressing Ctrl + Alt + T. Navigate to the directory where the downloaded file is located. Use the appropriate package manager command to install the .deb package. For example:

```
` sudo dpkg -i <file-name>.deb `
` sudo apt-get install -f `
```

Replace <file-name> with the actual name of the downloaded file. Once the installation is complete, you can launch VSCode from the applications menu or by typing code in the Terminal. VSCode is now installed on your Linux system!

#### 4. Getting Started

Launch Visual Studio Code by searching for it in your applications menu or by double-clicking its icon. Congratulations! You have successfully installed VSCode. Familiarize yourself with the interface and explore the features to get started with your coding projects.

That’s it! You’re now ready to start coding with Visual Studio Code. Happy coding!

## API REFERENCE

### 4.1 Robot Class

**class** neurapy.robot.Robot(\*args, \*\*kwargs)

Bases: object

A python class to access and control the robot's software functionalities. It serves as an interface for users to interact with the robot's capabilities.

Initializes the Robot class with the specified parameters. By default these values are taken from *robot\_overview.yaml* file.

**Note:** Positional arguments can only be used when the *Robot* class is initialized on Ubuntu 18 and 20. On other Ubuntu versions and Windows, no arguments are needed.

#### Parameters

**args** – Positional arguments to initialize the robot.

- args[0] (str): Name of the robot.
- args[1] (int): Degrees of freedom of the robot (6 or 7).
- args[2] (str): IP address for the Orocos framework.
- args[3] (str): IP address of the robot.
- args[4] (str): Architecture type of the robot ('single' or 'dual').

#### Sample Usage:

##### Example 1:

```
from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.kURL)
```

##### Example 2:

```
from neurapy.robot import Robot

r = Robot('lara5',6,'192.168.2.23','192.168.2.23','single')
print(r.robot_name)
```

(continues on next page)

(continued from previous page)

```
print(r.dof)
print(r.kURL)
```

**activate\_ethercat\_interface()**

Activate the Ethercat/el6695 interface.

**Returns**

True if the EtherCAT interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_ethercat_interface()
```

**activate\_ethernetIP\_interface(\*args)**

Activate the Ethernet/IP interface.

**Parameters**

**path** (*str*) – The path of the YAML file to activate the Ethernet/IP interface.

**Returns**

True if the Ethernet/IP interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_ethernetIP_interface('/path/to/yaml')
```

**activate\_modbus\_interface(\*args)**

Activate the Modbus interface.

**Parameters**

**path** (*str*) – The path of the YAML file to activate the Modbus interface.

**Returns**

True if the Modbus interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_modbus_interface('/path/to/yaml')
```

**activate\_opcua\_interface(\*args)**

Activate the OPC UA interface.

**Parameters**

**path** (*str*) – The path of the YAML file to activate the opcua interface.

**Returns**

True if the OPC UA interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_opcua_interface('/path/to/yaml')
```

**activate\_profinet\_interface()**

Activate the Profinet interface.

**Returns**

True if the Profinet interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_profinet_interface()
```

**activate\_ros\_interface(\*args)**

Activate the ROS interface.

**Parameters**

**mode** (*str*) – The mode to activate the ROS interface. Options:

- 'position': Activate the ROS interface in position mode.
- 'torque': Activate the ROS interface in torque mode.

**Returns**

True if the ROS interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_ros_interface('position')
```

**activate\_servo\_interface(\*args)**

Activate the Servo interface.

**Parameters**

**mode** (*str*) – The mode to activate the Servo interface. Options:

- 'position': Activate the Servo interface in position mode.
- 'torque': Activate the Servo interface in torque mode.

**Returns**

True if the Servo interface is activated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.activate_servo_interface('position')
```

**associate\_pose\_to\_new\_parent\_frame**(*list\_pose\_id*, *new\_parent\_id*)

Method which shall change the association of the input pose(point/frame) to a new frame. This shall retain the world coordinates of the input pose but only change its association to a new parent frame such that when the new parent is updated, this input pose shall also get itself updated.

**Parameters**

- **list\_pose\_id** (*list[str]*) – The list of ids in DB for the input poses. Even if it is just one id, pass it in a list.
- **new\_parent\_id** (*str*) – The id in DB for the new parent pose.

**Raises**

**ValueError** – If the input{list\_pose\_id} is not a list or if any element in the list is not a string.

**Returns**

True if operation successfull.

**Return type**

bool

**compute\_forward\_kinematics**(*joint\_angles*, *target\_frame*='tool', *representation*='rpy', *tool\_params*=None)

Compute forward kinematics (end-effector's Cartesian pose) for the given joint configuration.

**Parameters**

- **joint\_angles** (*list*) – list of joint angles in radians.
- **target\_frame** (*str*, *optional*) – the node for which the pose is requested. Default: 'tool'. Options: ['flange', 'wrist', 'elbow', 'link6', 'link5', ...]
- **Default** (*representation:string defining the return type of the rotational part of the pose.*) – 'rpy'. Options: ['rpy', 'quaternion']

**Raises**

**ValueError** – If the size of the argument joint\_angles is not equal to the number of joints on the robot or if the representation is not valid

**Returns**

list representing the TCP pose in XYTRPY format.

**Return type**

list

**Sample Usage**

```
from neurapy.robot import Robot
r = Robot()
target_joint_angles = [0.2] * r.dof
tcp_pose = r.compute_forward_kinematics(joint_angles=target_joint_angles)
```

**compute\_inverse\_kinematics**(*target\_pose*, *reference\_joint*, *representation*='rpy', *tool\_params*=None, *return\_pose*: bool = False)

Compute inverse kinematics(joint configuration) for the given end-effector's cartesian pose.

#### Parameters

- **target\_pose** (*list*) – list representing the target end\_effector pose in XYZRPY format if representation == “rpy”. For the representation “quaternion” the format is XYZQWQ1Q2Q3
- **reference\_joint** (*list*) – list of reference joint angles (of the robot configuration) in radians.
- **representation** (*string*, *optional*) – string defining the input type of the rotational part of the target\_pose. Default: ‘rpy’. Options: [‘rpy’, ‘quaternion’]

#### Raises

- **IKNotFound** – If IK solution could not be found.
- **ValueError** – If the size of the argument joint\_angles is not equal to the number of joints on the robot or if the representation is not valid

#### Returns

list of joint angles in radians that achieve the target end-effector pose

#### Return type

list

#### Sample Usage

```
from neurapy.robot import Robot
r = Robot()
target_end_effector_pose = [0.140448, -0.134195, 1.197456, 3.1396, -0.589, -
↪ 1.025]
reference_joint_angles = [-1.55, -0.69, 0.06, 1.67, -0.02, -1.57, 0.11]
joint_angle_solution = r.compute_inverse_kinematics(target_pose=target_end_
↪ effector_pose, reference_joint=reference_joint_angles)
```

**convert\_euler\_to\_quaternion\_pose**(*cartesian\_pose*)

Convert a pose with Euler angles to quaternions.

If the input pose is already in quaternion format (length 7), it is returned unchanged.

#### Parameters

**cartesian\_pose** (*list*) – The pose to be converted to quaternion format ([X, Y, Z, R, P, Y], required: Yes).

#### Returns

The pose in quaternion format ([X, Y, Z, W, EX, EY, EZ]).

#### Return type

list

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
euler_pose = [1.0, 2.0, 3.0, 0.1, 0.2, 0.3] # [X, Y, Z, R, P, Y]
```

(continues on next page)

(continued from previous page)

```

quaternion_pose = r.convert_euler_to_quaternion_pose(euler_pose)
print(quaternion_pose) # Output: [X, Y, Z, W, EX, EY, EZ] with
↳ quaternion values.

```

**convert\_quaternion\_to\_euler\_pose(cartesian\_pose)**

Convert a pose with quaternion representation to a pose with Euler angles.

If the input pose is already in Euler format (length 6), it is returned unchanged.

**Parameters**

**cartesian\_pose** (*list*) – The pose to be converted to Euler angles ([X, Y, Z, W, EX, EY, EZ], required: Yes).

**Returns**

The pose in Euler angle format ([X, Y, Z, R, P, Y]).

**Return type**

list

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
quaternion_pose = [1.0, 2.0, 3.0, 0.707, 0.0, 0.707, 0.0] # [X, Y, Z, W,
↳ EX, EY, EZ]
euler_pose = r.convert_quaternion_to_euler_pose(quaternion_pose)
print(euler_pose) # Output: [X, Y, Z, R, P, Y] with Euler angle values.

```

**create\_external\_source\_app(file\_path: str, drymode: bool = False) → ExternalSourceApp**

Creates an instance of the external source, returning an object of the connected external source type. Supported industrial protocols are EthernetIP, ModbusRTU (Rs485), and ModbusTCP. Check the “External source” tab in settings page to get an overview of the supported sources and functions and parameters associated with them.

**Parameters**

- **file\_path** (*str*) – The path to the JSON configuration file of the external source (required: Yes).
- **dry\_mode** (*bool, optional*) – If set to True, enables dry mode, where it tries to connect to the hardware but does not stop the python program when connection is not established and actual hardware connection is not performed, “start” process (process\_on) will only be skipped. Note : If you set drymode as true call execute\_process function, the function will be executed. Defaults to False.

**Raises**

**DriverError** – If the creation of the external source app is unsuccessful.

**Returns**

An instance of ExternalSourceApp, allowing various operations to be executed with the external source.

**Return type**

ExternalSourceApp

**Sample Usage**

```
import time
from neurapy.robot import Robot
r = Robot()
external_source_app = r.create_external_source_app("~/data/process/
↪external_source.json", dry_mode=False)
```

**create\_point**(*name*, *reference\_frame\_name*='World', *target\_end\_effector\_pose*=None, *target\_joint\_angles*=None, *tool\_name*=None)

Store a point in the database.

#### Parameters

- **name** (*str*) – Name of the point to be saved (required: Yes).
- **reference\_frame\_name** – Name of the reference\_frame (default: “World”).
- **target\_end\_effector\_pose** (*list*) – A list containing Cartesian points in XYZRPY format (required only if *target\_joint\_angles* is not provided).
- **target\_joint\_angles** (*list*) – A list containing joint angles for the point (required only if *target\_end\_effector\_pose* is not provided).
- **tool\_name** – Name of the tool for the kinematics needed to save a valid input point. (default: Active tool in GUI).

#### Returns

True if the point was added successfully.

#### Return type

bool

#### Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If point creation fails with the given input data.

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.create_point(name='new_point', target_end_effector_pose=[0.36, 0.36, 0.
↪1, 3.14, 0, 0, 3.14])
```

**create\_tool**(*tool\_data*)

Create a new tool in the robot’s database using the provided tool data.

#### Parameters

**tool\_data** (*dict*) – A dictionary containing the data for the new tool (required: Yes).

#### Returns

True if the tool was added successfully.

#### Return type

bool

#### Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If tool creation fails with the given input data.

**Tool Description**

| Key/Value    | De-<br>fault<br>Value | Description                                                                                                       | Required or Not |
|--------------|-----------------------|-------------------------------------------------------------------------------------------------------------------|-----------------|
| _controlOA   | False                 | True, if the tool is controlled by controlbox analog outputs                                                      | Not Required    |
| _controlOD   | False                 | True, if the tool is controlled by controlbox digital outputs                                                     | Not Required    |
| _toolOA      | False                 | True, if the tool is controlled by analog outputs of port present on robot tool flange                            | Not Required    |
| _toolOD      | False                 | True, if the tool is controlled by digital outputs of port present on robot tool flange                           | Not Required    |
| autoM        | 0                     | Mass of the tool                                                                                                  | Required        |
| autoMeasureX | 0                     | tool center of gravity (COG) in X direction, measured from robot's flange frame                                   | Not Required    |
| autoMeasureY | 0                     | tool center of gravity (COG) in Y direction, measured from robot's flange frame                                   | Not Required    |
| autoMeasureZ | 0                     | tool center of gravity (COG) in Z direction, measured from robot's flange frame                                   | Not Required    |
| closeInput   | 0                     | percentage of gripper width for close action                                                                      | Not Required    |
| cmdID        | 16                    | Input specific for certain kind of grippers                                                                       | Not Required    |
| force        | 0                     | Gripper closing force                                                                                             | Not Required    |
| gripper      |                       | Gripper type                                                                                                      | Not Required    |
| gripper-type | Standard Gripper      | Type of gripper Modbus/Standard                                                                                   | Not Required    |
| inertiaXX    | 0                     | Ixx of tool                                                                                                       | Not Required    |
| inertiaXY    | 0                     | Ixy of tool                                                                                                       | Not Required    |
| inertiaXZ    | 0                     | Ixz of tool                                                                                                       | Not Required    |
| inertiaYY    | 0                     | Iyy of tool                                                                                                       | Not Required    |
| inertiaYZ    | 0                     | Iyz of tool                                                                                                       | Not Required    |
| inertiaZZ    | 0                     | Izz of tool                                                                                                       | Not Required    |
| name         | N/A                   | Name of the tool                                                                                                  | Required        |
| offCOA       | [0, 0, 0, 0, 0, 0]    |                                                                                                                   | Not Required    |
| offCOD1      | 0                     | if the tool is controlled via control box digital outputs, offCOD1 is the pin mapped to turn off (close) the tool | Not Required    |
| offCOD2      | 0                     |                                                                                                                   | Not Required    |
| offTOA       | [0, 0]                |                                                                                                                   | Not Required    |
| offTOD       | 0                     | if the tool is controlled via tool digital outputs, offTOD is the pin mapped to turn off (close) the tool         | Not Required    |
| offsetA      | 0                     | TCP roll offset                                                                                                   | Not Required    |
| offsetB      | 0                     | TCP pitch offset                                                                                                  | Not Required    |
| offsetC      | 0                     | TCP yaw offset                                                                                                    | Not Required    |
| offsetX      | 0                     | TCP offset in X                                                                                                   | Not Required    |
| offsetY      | 0                     | TCP offset in Y                                                                                                   | Not Required    |

continues on next page

Table 1 – continued from previous page

| Key/Value  | De-<br>fault<br>Value             | Description                                                                                                    | Required or Not |
|------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------|
| offsetZ    | 0                                 | TCP offset in Z                                                                                                | Not Required    |
| onCOA      | [0, 0,<br>0, 0,<br>0, 0,<br>0, 0] |                                                                                                                | Not Required    |
| onCOD1     | 0                                 | if the tool is controlled via control box digital outputs, onCOD1 is the pin mapped to turn on (open) the tool | Not Required    |
| onCOD2     | 0                                 |                                                                                                                | Not Required    |
| onTOA      | [0, 0]                            |                                                                                                                | Not Required    |
| onTOD      | 0                                 | if the tool is controlled via tool digital outputs, onTOD is the pin mapped to turn on (open) the tool         | Not Required    |
| openInput  | 0                                 |                                                                                                                | Not Required    |
| portID     | 0                                 |                                                                                                                | Not Required    |
| protocol   | 0                                 |                                                                                                                | Not Required    |
| robot_type | Tool                              |                                                                                                                | Required        |
| slaveID    | 0                                 |                                                                                                                | Not Required    |
| speed      | 0                                 |                                                                                                                | Not Required    |

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
tool_data = { '_control0A': False,
  '_control0D': False,
  '_tool0A': False,
  '_tool0D': False,
  'autoM': 0,
  'autoMeasureX': 0,
  'autoMeasureY': 0,
  'autoMeasureZ': 0,
  'closeInput': 0,
  'cmdID': 16,
  'description': 'Tool Description',
  'force': 0,
  'gripper': '',
  'grippertype': 'Standard Gripper',
  'inertiaXX': 0,
  'inertiaXY': 0,
  'inertiaXZ': 0,
  'inertiaYY': 0,
  'inertiaYZ': 0,
  'inertiaZZ': 0,
  'name': 'somerandomtool',
  'offCOA': [0, 0, 0, 0, 0, 0, 0, 0, 0],
  'offCOD1': 0,

```

(continues on next page)

(continued from previous page)

```

'offCOD2': 0,
'offTOA': [0, 0],
'offTOD': 0,
'offsetA': 0,
'offsetB': 0,
'offsetC': 0,
'offsetX': 0,
'offsetY': 0,
'offsetZ': 0,
'onCOA': [0, 0, 0, 0, 0, 0, 0, 0],
'onCOD1': 0,
'onCOD2': 0,
'onTOA': [0, 0],
'onTOD': 0,
'openInput': 0,
'portID': '',
'protocol': 0,
'robot_type': 'Tool',
'slaveID': 0,
'speed': 0}

tools_data = r.create_tool(tool_data)

```

**deactivate\_ethercat\_interface()**

Deactivate the Ethercat interface.

**Returns**

True if the Ethercat interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.deactivate_ethercat_interface()

```

**deactivate\_ethernetIP\_interface()**

Deactivate the Ethernet/IP interface.

**Returns**

True if the Ethernet/IP interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.deactivate_ethernetIP_interface()

```

**deactivate\_modbus\_interface()**

Deactivate the Modbus interface.

**Returns**

True if the Modbus interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_modbus_interface()
```

**deactivate\_opcua\_interface()**

Deactivate the OPCUA interface.

**Returns**

True if the OPCUA interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_opcua_interface()
```

**deactivate\_profinet\_interface()**

Deactivate the Profinet interface.

**Returns**

True if the Profinet interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_profinet_interface()
```

**deactivate\_ros\_interface()**

Deactivate the ROS interface.

**Returns**

True if the ROS interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_ros_interface()
```

**deactivate\_servo\_interface()**

Deactivate the Servo interface.

**Returns**

True if the Servo interface is deactivated successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_servo_interface()
```

**delete\_pose\_in\_DB(input\_str, new\_parent\_str='World')**

Deletes an existing pose in the DB. If the pose is a frame with children, then the children are properly handled.

**Parameters**

- **input\_str** (*str*) – The id/name of the pose to be deleted from DB.
- **new\_parent\_str** (*str*) – The id/name of the new parent frame for the children of the input if it is a frame. Default 'World'.

**Returns**

True if the pose was deleted successfully.

**Return type**

bool

**disable\_collision\_detection()**

Disable collision detection on the robot.

**Returns**

True if reflex collision avoidance is disabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.disable_collision_detection()
```

**disable\_friction\_mode()**

To turn off friction mode (smooth free drive mode).

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **WrongMode** – If the Robot is either not in 'teach' mode, in simulation, or if free drive mode is not enabled.
- **ConnectionError** – If the Robot is powered off or if an internal error occurs.
- **RuntimeError** – If friction mode cannot be controlled in the current version of the Control Software.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.turn_off_friction_mode()
```

**disable\_reflex()**

Disable reflex after collision.

**Returns**

True if reflex collision avoidance is disabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.disable_reflex()
```

**enable\_collision\_detection()**

Enable collision detection on the robot.

**Returns**

True if reflex collision avoidance is enabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.enable_collision_detection()
```

**enable\_friction\_mode()**

To turn on friction mode (smooth free drive mode). Free drive mode needs to be activated beforehand.

**Warning:**

This might lead to the robot falling!

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **WrongMode** – If the Robot is either not in ‘teach’ mode, in simulation, or if free drive mode is not enabled.
- **ConnectionError** – If the Robot is powered off or if an internal error occurs.
- **RuntimeError** – If friction mode cannot be controlled in the current version of the Control Software.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.turn_on_free_drive_mode()
r.turn_on_friction_mode()
```

**enable\_reflex()**

Enable reflex after collision.

**Returns**

True if reflex collision avoidance is enabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.enable_reflex()
```

**encoder2rad(*impulses*, *offset=False*)**

Convert encoder impulses to joint angles in radians.

**Parameters**

- **impulses** (*list*) – A list of encoder impulses for each joint (Required).
- **offset** (*bool*) – If True considers robot offset values for calculation (Optional).

**Returns**

A list of joint angles in radians corresponding to the provided encoder impulses.

**Return type**

list

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.robot_status('loadSideEncValue')
joint_angles = r.encoder2rad(encoder_ticks)
```

**execute\_closest\_orthogonal\_tcp\_align(\*args, \*\*kwargs)**

Execute the closest orthogonal axes-aligned TCP alignment motion.

**Parameters**

**args** (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

**Returns**

True if the TCP alignment is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = [x, y, z, a, b, c] # list of floating point values,
    ↪denoting the 6D pose.
robot.execute_closest_orthogonal_tcp_align(target_pose)
```

**Example 2:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_closest_orthogonal_tcp_align(target_pose)
```

**execute\_exact\_tcp\_align(\*args, \*\*kwargs)**

Execute the exact axes-aligned TCP alignment motion.

**Parameters**

**args** (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

**Returns**

True if the TCP alignment is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = [x, y, z, a, b, c] # list of floating point values,
    ↪denoting the 6D pose.
robot.execute_exact_tcp_align(target_pose)
```

**Example 2:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_exact_tcp_align(target_pose)
```

**execute\_program(name)**

To run a program created from teach pendant’s tree program creator

**Parameters**

**name** (*str*) – The name of the saved program to execute.

**Returns**

True if the program executed successfully, raises an exception if execution encountered an error.

**Return type**

bool

**Raises**

**Exception** – If an exception is raised during program execution, this function raises it with the exception message.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
try:
    result = r.execute_program("Program_001")
    if result:
        print("Program executed successfully.")
except Exception as e:
    print("Error: " + str(e))
```

**execute\_z\_axis\_tcp\_align(\*args, \*\*kwargs)**

Execute the z-axis-aligned TCP alignment motion.

**Parameters**

**args** (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

**Returns**

True if the z-axis alignment motion is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = [x, y, z, a, b, c] # list of floating point values,
    denoting the 6D pose.
robot.execute_z_axis_tcp_align(target_pose)
```

**Example 2:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_z_axis_tcp_align(target_pose)
```

**executor(\*args, \*\*kwargs)**

Execute already planned motion IDs.

This method allows you to execute a list of planned motion IDs, which represent pre-defined motion sequences or plans. This needs to be used in conjunction with `plan_move_circular/joint/linear/composite/record_path` functions.

#### Parameters

**Int (List of)** – List of planned motion IDs to be executed.

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
planned_motion_ids = r.plan_move_joint([0]*r.dof) # Replace with your
↳planned motion IDs
r.executor([planned_motion_ids])
```

#### `get_analog_input(io_name)`

Retrieve the value of an analog input.

Use the `get_io_configuration()` function to determine the number of available I/Os.

#### Parameters

**io\_name (int)** – The identifier of the analog input.

#### Returns

The value of the specified analog input.

#### Return type

float

#### Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_analog_input(1)
print(io_get)
```

#### `get_analog_output(io_name)`

Retrieve the value of an analog output.

Use the `get_io_configuration()` function to determine the number of available I/Os.

#### Parameters

**io\_name (int)** – The identifier of the analog output.

#### Returns

The value of the specified analog output.

#### Return type

float

#### Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_analog_output(1)
print(io_get)
```

**get\_current\_joint\_angles()**

Get current joint angles of the robot.

**Returns**

A list of joint angles in radians.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_angles = r.get_current_joint_angles()
```

**get\_current\_joint\_angles\_with\_timestamp()**

Get the current joint angles of the robot along with UTC timestamp.

**Returns**

Tuple of a list of robot joint angles in radians and UTC timestamp

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_angles, timestamp = r.get_current_joint_angles_with_timestamp()
```

**get\_current\_joint\_torques()**

Get the current joint torques of the robot.

**Returns**

List of joint torque values

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_torques = r.get_current_joint_torques()
```

**get\_current\_joint\_torques\_with\_timestamp()**

Get the current joint torques of the robot along with UTC timestamp.

**Returns**

Tuple of a list of joint torque values and UTC timestamp.

**Return type**

tuple

**Sample Usage**

```
from neurapy.robot import Robot
r = Robot()
joint_torques, timestamp = r.get_current_joint_torques_with_timestamp()
```

**get\_current\_joint\_velocities()**

Get current joint velocities of the robot.

**Returns**

List of robot joint velocities in radians/sec

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_velocities = r.get_current_joint_velocities()
```

**get\_current\_joint\_velocities\_with\_timestamp()**

Get the current joint velocities of the robot along with UTC timestamp.

**Returns**

Tuple of a list of robot joint velocities in radians/sec and UTC timestamp

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_velocities, timestamp = r.get_current_joint_velocities_with_
    ↪ timestamp()
```

**get\_current\_load\_side\_encoder\_values()**

Get the current load side encoder ticks of the robot.

**Returns**

List of encoder tick values

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_load_side_encoder_values()
```

**get\_current\_motor\_side\_encoder\_values()**

Get the current motor side encoder ticks of the robot .

**Returns**

List of encoder tick values

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_motor_side_encoder_values()
```

**get\_current\_tool\_cogs()**

Get the current tool center of gravity (COG) in XYZ directions(measured from robot's flange frame/tool mounting point).

**Returns**

List representing the tool COG in XYZ directions.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_cog = r.get_current_tool_cogs()
```

**get\_current\_tool\_inertias()**

Get the current tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.

**Returns**

List representing the tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_inertias = r.get_current_tool_inertias()
```

**get\_current\_tool\_mass()**

Get the current tool mass.

**Returns**

The mass of the tool.

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_mass = r.get_current_tool_mass()
```

**get\_current\_tool\_properties()**

Get the current tool properties.

**Returns**

List containing the updated tool properties, i.e., [tool\_mass, roll\_offset, pitch\_offset, yaw\_offset, x\_offset, y\_offset, z\_offset, COG\_x, COG\_y, COG\_z, Ixx, Iyy, Izz, Ixy, Ixz, Iyz].

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_properties = r.get_current_tool_properties()
```

**get\_current\_tool\_rpy\_offsets()**

Get the current tool roll, pitch, and yaw (RPY) offsets in radians.

**Returns**

List representing the tool RPY offsets in radians.

**Return type**

List

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_rpy_offsets = r.get_current_tool_rpy_offsets()
```

**get\_current\_tool\_translation\_offsets()**

Get the current tool translation offsets in XYZ directions.

**Returns**

List representing the tool translation offsets in XYZ directions.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool_offsets = r.get_current_tool_translation_offsets()
```

**get\_diagnostics()**

Method to query current robot diagnostics

**Returns**

If critical, a dictionary containing diagnostics values (i.e., critical, powered\_off, collision\_status); otherwise, an empty dictionary.

**Return type**

dict

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
print(r.get_diagnostics())
```

**get\_digital\_input(io\_name)**

Get the value of a digital input.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*int*) – The name (pin number) of the digital input to retrieve the value from.

**Returns**

The value of the digital input, either 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_input(1)
print(io_get)
```

**get\_digital\_output(io\_name)**

Get the value of a digital output.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (int) – The name (pin number) of the digital output to retrieve the value from.

**Returns**

The value of the digital output, either 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_output(1)
print(io_get)
```

**get\_doc(function\_name)**

Get the description and sample usage of the specified function.

**Parameters**

**function\_name** (str) – The name of the function for which description is needed.

**Returns**

Short description and usage of the function.

**Return type**

str

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
description = r.get_doc("move_joint")
print(description)
```

**get\_encoder\_offsets()**

To retrieve the encoder offsets of a robot.

**Returns**

A list containing the encoder offsets for the robot's encoders.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
encoder_offsets = r.get_encoder_offsets()
```

**get\_errors(\*args, \*\*kwargs)**

Query the list of errors present on the robot.

**Returns**

A list of errors present on the robot.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
print(r.get_errors())
```

**get\_ext\_digital\_input(io\_name)**

Get the value of an external digital input.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*int*) – The pin number of the external digital input to retrieve the value from.

**Returns**

The value of the digital input, either 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_input(1)
print(io_get)
```

**get\_ext\_digital\_output(io\_name)**

Get the value of an external digital output.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*int*) – The pin number of the external digital output to retrieve the value from.

**Returns**

The value of the digital output, either 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_ext_digital_output(1)
print(io_get)
```

**get\_flange\_pose**(*timestamp=False, representation='rpy'*)

Get the current flange pose in XYZRPY format.

**Returns**

List representing the flange pose with XYZRPY values.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
flange_pose = r.get_flange_pose()
```

**get\_flange\_pose\_quaternion**()

Get the current flange pose in XYZwxyz format.

**Returns**

List representing the flange pose with [X,Y,Z,w,x,y,z].

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
flange_pose_quaternion = r.get_flange_pose_quaternion()
```

**get\_flange\_pose\_quaternion\_with\_timestamp**()

Get the current flange pose in XYZwxyz format with UTC timestamp.

**Returns**

Tuple representing the flange pose with [X,Y,Z,w,x,y,z] and UTC timestamp.

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
flange_pose_quaternion, timestamp = r.get_flange_pose_quaternion_with_
↳ timestamp()
```

**get\_flange\_pose\_with\_timestamp**()

Get the current tool flange pose in XYZRPY format with UTC timestamp.

**Returns**

Tuple of the flange pose with XYZRPY values and UTC timestamp.

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
flange_pose, timestamp = get_flange_pose_with_timestamp
```

**get\_io\_configuration()**

Get the number of available IOs.

**Returns**

A dictionary containing the type and number of the IOs.

**Return type**

dict

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_configuration = r.get_io_configuration()
print(io_configuration)
```

**get\_io\_status()**

Get all the available IOs and it's values

**Returns**

A dictionary containing all the IOs and it's available values

**Return type**

dict

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_io_status()
print(io_get)
```

**get\_joint\_acceleration()**

Get the current acceleration value for joint motion.

**Returns**

The current acceleration value for joint motion (percentage of maximum angular acceleration).

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
acceleration = r.get_joint_acceleration()
print(f"Current joint acceleration: {acceleration}")
```

**get\_joint\_speed()**

Get the current speed value for joint motion.

**Returns**

The current speed value for joint motion (percentage of maximum angular speed).

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
speed = r.get_joint_speed()
print(f"Current joint speed: {speed}")
```

**get\_linear\_acceleration()**

Get the current acceleration value for linear motion.

**Returns**

The current acceleration value for linear motion (m/s<sup>2</sup>).

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
current_acceleration = r.get_linear_acceleration()
print(f"Current linear acceleration: {current_acceleration} m/s2")
```

**get\_linear\_speed()**

Get the current speed value for linear motion.

**Returns**

The current speed value for linear motion (m/s).

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
current_speed = r.get_linear_speed()
print(f"Current linear speed: {current_speed} m/s")
```

**get\_load\_side\_encoder\_values\_with\_timestamp()**

Get the current load side encoder ticks of the robot along with UTC timestamp.

**Returns**

Tuple of a list of encoder tick values and UTC timestamp

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks, timestamp = r.get_current_load_side_encoder_values_with_
    ↪ timestamp()
```

### get\_motor\_side\_encoder\_values\_with\_timestamp()

Get the current motor side encoder ticks of the robot along with UTC timestamp.

#### Returns

Tuple of a list of encoder tick values and UTC timestamp

#### Return type

tuple

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks, timestamp = r.get_current_motor_side_encoder_values_with_
    ↪ timestamp()
```

### get\_override()

Get the current override value.

#### Returns

The current override value.

#### Return type

float

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
override_value = r.get_override()
print(f"Current override value: {override_value}")
```

### get\_point(name, representation='Joint')

Retrieves a point from the database based on the given name.

#### Parameters

- **name** (*str*) – The name/id of the point to retrieve from the database.
- **representation** (*str*) – To specify the point representation type.
  - **Joint** - Default - returns Joint values of the stored point
  - **Cartesian** - returns Cartesian pose of the stored point

#### Returns

A list containing the details of the point. For Joint representation - List contains the values of joint angles in radian. For Cartesian representation - List contains X,Y,Z,R,P,Y values

#### Return type

list

#### Raises

- **ConnectionError** – If a connection to the remote database cannot be established.

- **ValueError** – If no point is saved in the database with the given name or the given representation type is invalid.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
point = r.get_point("P1",representation="Joint")
print(point)
point = r.get_point("P1",representation="Cartesian")
print(point)
```

**get\_point\_names()**

Retrieves a list of names of all saved points in the database.

**Returns**

A list of names of the saved points.

**Return type**

list

**Raises**

**ConnectionError** – If connection to the remote database cannot be established.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
point_names = r.get_point_names()
print(point_names)
```

**get\_pose\_id\_from\_name(input\_name)**

Get the id of the pose in DB from the corresponding name saved.

**Parameters**

**input\_name** (*str*) – The name of the pose under query.

**Returns**

The id of the pose associated with the input name.

**Return type**

str

**get\_pose\_in\_parent\_frame\_from\_DB(input\_str, is\_response\_in\_snake\_case=False)**

Retrieves the pose for the input\_str and converts it from world frame to its parent frame which is returned by this function.

**Parameters**

- **input\_str** (*str*) – The DB id/name for the input whose pose shall be returned in its parent frame.
- **is\_response\_in\_snake\_case** (*bool*) – Flag if the json should be in snake\_case and not camelCase. Default False.

**Returns**

The pose json-structure as saved in DB. Will be empty if the query is invalid. This pose is w.r.t. its parent frame.

**Return type**

dict

**get\_pose\_in\_reference\_frame\_from\_DB**(*pose\_id*, *reference\_frame\_id*=None, *is\_response\_in\_snake\_case*=False)

Gets the pose for the {pose\_id} from DB, which is saved in world frame, and transforms it w.r.t the {reference\_frame\_id}.

#### Parameters

- **pose\_id** (*str*) – The DB id/name of the pose.
- **reference\_frame\_id** (*str*) – The DB id/name of the frame w.r.t which the pose shall be calculated. Default None which shall be taken as parent frame.
- **is\_response\_in\_snake\_case** (*bool*) – Flag if the json should be in snake\_case and not camelCase. Default False.

#### Returns

The pose json-structure as saved in DB. Will be empty if the query is invalid. This pose is w.r.t. the reference frame.

#### Return type

dict

**get\_pose\_in\_world\_frame\_from\_DB**(*input\_str*, *is\_response\_in\_snake\_case*=False)

Gets the pose in world frame for the input\_str from DB.

#### Parameters

- **input\_str** (*str*) – The DB id/name for the input whose pose shall be returned in world frame.
- **is\_response\_in\_snake\_case** (*bool*) – Flag if the json should be in snake\_case and not camelCase. Default False.

#### Returns

The pose json-structure as saved in DB. Will be empty if the query is invalid. This pose is w.r.t. the world frame.

#### Return type

dict

**get\_program\_names**()

Retrieves list of program names from the robot's database

#### Returns

A list of names of the programs available in the robot's database.

#### Return type

list

#### Raises

**ConnectionError** – If connection to the remote database cannot be established.

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
program_names = r.get_program_names()
print(program_names)
```

**get\_reference\_frame**(*name*)

Retrieve the coordinates and orientations of a user-defined reference frame.

**Parameters**

**name** (*str*) – The name of the reference frame to retrieve.

**Returns**

A list containing the values of the reference frame ([X,Y,Z,R,P,Y]).

**Return type**

list

**Raises**

**ConnectionError** – If there is a failure to connect to the robot's database.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
frame = r.get_reference_frame("tool_frame")
print(frame)
```

**get\_reference\_frame\_with\_offset**(*name, offset*)

Retrieve a reference frame with an applied offset to its position.

**Parameters**

- **name** (*str or list*) – The name of the reference frame or the reference frame as a list.
- **offset** (*list*) – A list containing the offset values [X, Y, Z] to apply to the frame's position.

**Returns**

A list containing the [X, Y, Z, A, B, C] values (coordinates and angles) of the translated reference frame.

**Return type**

list

**Raises**

**ValueError** – If the name argument is neither a valid string nor a list. If the offset argument is not a list of length 3.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
x_offset = 0.02 # in meters
y_offset = 0.1 # in meters
z_offset = 0.0 # in meters
frame = r.get_reference_frame_with_offset("world", [x_offset, y_offset, z_
↪ offset])
print(frame)
```

**get\_register\_bytes**(*tag\_name, first\_element, last\_element, T2O=False*)

Returns a python "bytes" object from a T2O or O2T register specified by its tag name and index range from first to last element.

**Parameters**

- **tag\_name** (*str*) – The register's tag name (without any element index) as found in the official EDS files or in the detailed mapping in EtherNET/IP users documentation.
- **first\_element** (*int*) – Index of the first requested register element.

- **last\_element** (*int*) – Index of the last requested register element.
- **T20** (*bool*) – Use True if the register is in Target-to-Originator registry instead of Originator-to-Target.

**Returns**

Encoded values or None on failure (invalid range, communication failure).

**Return type**

bytes

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
octet_list = r.get_register_bytes("INTIN", 1, 4) # return second to
↪ fifth integer elements from
# input (O2T) integer register
```

**get\_register\_values**(tag\_name, first\_element, last\_element, T2O=False)

Returns values from a T2O or O2T register specified by its tag name and index range from first to last element

**Parameters**

- **tag\_name** (*str*) – The register's tag name (without any element index) as found in the official EDS files or in the detailed mapping in EtherNET/IP users documentation.
- **first\_element** (*int*) – Index of the first requested register element.
- **last\_element** (*int*) – Index of the last requested register element.
- **T20** (*bool*) – Use True if the register is in Target-to-Originator registry instead of Originator-to-Target.

**Returns**

List of Python values or an empty list on failure (invalid range, communication failure).

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
octet_list = r.get_register_values("INTIN", 1, 4) # return second to
↪ fifth integer elements from
# input (O2T) integer register
```

**get\_safety()****get\_selected\_fts\_data()**

Retrieves the selected external fts from the robot's database.

**Returns**

selected\_fts\_data

**Return type**

dict

**Raises**

**ConnectionError** – If connection to the remote database cannot be established.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
selected_fts_data = r.get_selected_fts_data()
```

**get\_selected\_tool\_data()**

Retrieves the selected tool data from the robot's database.

**Returns**

The selected tool data from the robot's database.

**Return type**

list

**Raises**

**ConnectionError** – If connection to the remote database cannot be established.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tool = r.get_selected_tool_data()
print(tool)
```

**get\_selected\_tool\_name()**

Retrieves the selected tool name from the robot's database.

**Returns**

The name of the selected tool.

**Return type**

str

**Raises**

- **KeyError** – If the retrieved tool data does not contain a valid tool name.
- **ConnectionError** – If connection to the remote database cannot be established.

**get\_servo\_trajectory\_scaling\_factor(\*args, \*\*kwargs)**

Method to get the trajectory scaling factor from ServoJ

**Returns**

The scaling factor.

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')
```

(continues on next page)

(continued from previous page)

```

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]
inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

**get\_tcp\_force()**

Get the current TCP (Tool Center Point) force values of the robot.

**Returns**

List of TCP force values (Fx, Fy, Fz)

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_forces = r.get_tcp_force()
print("TCP Force Values:", tcp_forces)
```

**get\_tcp\_pose(timestamp=False, representation='rpy')**

to get the current tcp pose in XYZRPY format

**Returns**

List of TCP pose as [X, Y, Z, R, P, Y]

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_pose = r.get_tcp_pose()
print(tcp_pose)
```

**get\_tcp\_pose\_quaternion()**

to get the current tcp pose in XYZwxyz format

**Returns**

List of TCP pose as [X, Y, Z, w, x, y, z].

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_pose_quaternion = r.get_tcp_pose_quaternion()
```

**get\_tcp\_pose\_quaternion\_with\_timestamp()**

Get the current tcp pose in XYZwxyz format along with UTC timestamp

**Returns**

Tuple of TCP pose as [X, Y, Z, w, x, y, z] and UTC timestamp.

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_pose_quaternion = r.get_tcp_pose_quaternion()
```

**get\_tcp\_pose\_with\_timestamp()**

Get the current robot's TCP pose in XYZRPY format along with UTC timestamp.

**Returns**

Tuple of the robot's TCP pose and UTC timestamp.

**Return type**

tuple

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_pose, timestamp = r.get_tcp_pose_with_timestamp()
```

**get\_tcp\_speed()**

Get the current speed of the robot's TCP (Tool Center Point).

**Returns**

The current TCP speed.

**Return type**

float

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_speed = r.get_tcp_speed()
```

**get\_tcp\_torque()**

Get the current TCP (Tool Center Point) torque values of the robot.

**Returns**

List of TCP torque values (Mx, My, Mz)

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_torques = r.get_tcp_torque()
print("TCP torque Values:", tcp_torques)
```

**get\_tcp\_wrench()**

Get the current TCP (Tool Center Point) wrench values, including both force and torque components.

**Returns**

List containing TCP force values (Fx, Fy, Fz) followed by torque values (Mx, My, Mz).

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tcp_wrench = r.get_tcp_wrench()
print("TCP Wrench Values:", tcp_wrench)
```

**get\_tool\_analog\_input(*io\_name*)**

Get the value of a tool analog input.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*str*) – The name of the tool analog input.

**Returns**

The value of the tool analog input.

**Return type**

float

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_analog_input(1)
print(io_get)
```

**get\_tool\_digital\_input(*io\_name*)**

Get the value of a tool digital input.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*str*) – The name of the tool digital input.

**Returns**

The value of the tool digital input, typically represented as 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_digital_input(1)
print(io_get)
```

**get\_tool\_digital\_output(*io\_name*)**

Get the value of a tool digital output.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**io\_name** (*str*) – The name of the tool digital output.

**Returns**

The value of the tool digital output, typically represented as 0 or 1.

**Return type**

int

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_digital_output(1)
print(io_get)
```

**get\_tools()**

Retrieves list of tools from the robot's database.

**Returns**

A list of tools available in the robot's database.

**Return type**

list

**Raises**

**ConnectionError** – If connection to the remote database cannot be established.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tools = r.get_tools()
print(tools)
```

**get\_warnings(\*args, \*\*kwargs)**

Query the list of warnings present on the robot.

**Returns**

A list of warnings present on the robot.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
print(r.get_warnings())
```

**grasp()**

To grasp the object with the attached gripper

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.grasp()
```

**gripper(\*args, \*\*kwargs)**

Method to control the gripper attached to the robot

**Sample Usage:**

```
from time import sleep
from neurapy.robot import Robot
r = Robot()
r.gripper('on')
sleep(2)
r.gripper('off')
```

**init\_program(\*args, \*\*kwargs)**

To allow running motion commands, without this, the planning of motion will be blocked and no motion will take place.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.init_program()
r.move_joint('Home')
r.stop()
```

**io(\*args, \*\*kwargs)**

Access and manipulate (output) I/O values on the robot.

This method allows for retrieving or setting the values of I/O operations.

**Parameters**

- **operation\_type** (*str*) – The type of operation to perform.
  - **get**: Retrieve the I/O value.
  - **set**: Set the output I/O value.
- **io\_name** (*str*) – The name of the I/O. (A list of available I/O names can be found in the documentation.)
- **target\_value** (*Any*) – The target value of the I/O. This parameter is required only for the 'set' operation.

**Returns**

- For 'get' operation: The value of the queried *io\_name*.
- For 'set' operation: True if the operation succeeded, False otherwise.

**Return type**

Union[Any, bool]

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.io("get", io_name = "DO_1")
io_set = r.io("set", io_name = "DO_2", target_value = True)
print(io_get, io_set)
```

**is\_collision\_enabled()**

Check if reflex after collision is enabled for the robot.

**Returns**

True if reflex collision avoidance is enabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.is_collision_enabled():
    print("Collision detection is enabled.")
else:
    print("Collision detection is not enabled.")

```

**is\_free\_drive\_mode\_enabled()**

Get the status of the robot free drive mode.

**Returns**

The status of the free drive mode. True if turned, else False.

**Return type**

bool

**Sample Usage:**

```

import time
from neurapy.robot import Robot
r = Robot()
status = r.is_free_drive_mode_enabled()

```

**is\_reflex\_enabled()**

Check if reflex after collision is enabled for the robot.

**Returns**

True if reflex collision avoidance is enabled, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.is_reflex_enabled():
    print("Reflex collision avoidance is enabled.")
else:
    print("Reflex collision avoidance is not enabled.")

```

**is\_robot\_in\_automatic\_mode()**

Method to check whether the robot is in automatic mode or not

**Returns**

True if the robot is in automatic mode, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_automatic_mode():
    print("Robot is in automatic mode")
else:
    print("Robot is not in automatic mode")

```

**is\_robot\_in\_collision()**

Check if the robot is currently in collision.

**Returns**

True if the robot is in collision, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_collision():
    print("Robot is in collision.")
else:
    print("Robot is not in collision.")
```

**is\_robot\_in\_semi\_automatic\_mode()**

Method to check whether the robot is in semi automatic mode or not

**Returns**

True if the robot is in semi-automatic mode, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_semi_automatic_mode():
    print("Robot is in semi automatic mode")
else:
    print("Robot is not in semi automatic mode")
```

**is\_robot\_in\_simulation()**

Method to check whether the robot is in simulation or not

**Returns**

True, if the robot is in simulation, False if it is in real mode.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_simulation():
    print("Robot is in simulation")
else:
    print("Robot is in real")
```

**is\_robot\_in\_teach\_mode()**

Method to check whether the robot is in teach mode or not

**Returns**

True if the robot is in teach mode, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_teach_mode():
    print("Robot is in teach mode")
else:
    print("Robot is not in teach mode")

```

**jog(\*args, \*\*kwargs)**

Method to jog programatically. This needs to be used in conjunction with turn\_on\_jog, turn\_off\_jog methods.

**Parameters**

**set\_jogging\_external\_flag (int)** – A flag to enable external jogging mode.

**Sample Usage:**

```

import time
from neurapy.robot import Robot
r = Robot()

"""
jog_velocity - velocity ranging from [-1,1] for all joints. Signifies
↳ percentage of total speed used (1 = 100%)
jog_type - can be either cartesian or joint jogging
turn_on_jog sets the parameters of external jogging, jogging velocity,
↳ jogging type and sets the mode for jogging to external mode.
"""

r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
↳ 'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
"""
Requires minimum number of cycles in the loop for performing jogging.
Depends upon jogging velocity, override.
"""
while i < 500:
    #command to set flag to start jogging in external mode. This command
    ↳ has to be used each time external jog command has to be sent
    r.jog(set_jogging_external_flag=1)
    i += 1

#Command to switch back to internal jogging mode (GUI)
r.turn_off_jog()

```

**list\_methods()**

To list the available functions in the API

**Returns**

A table listing the available functions and short descriptions of the functions.

**Return type**

PrettyTable

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.list_methods()
```

**list\_recorded\_path()**

To list the recorded path using ZeroG.

**Returns**

List of file names. Empty list, if no files are found.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
recorded_path = r.list_recorded_path()
print(recorded_path)
```

**lock\_cartesian\_axes(*cartesian\_axes*)**

Lock specific Cartesian axes while in free drive mode.

**Parameters**

**cartesian\_axes** (*List[str]*) – A list of Cartesian axes (“X”, “Y”, “Z”, “A”, “B”, “C”) to be locked.

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **ValueError** – If an unknown axis is provided. For example, if an invalid axis like “D” is passed when locking Cartesian axes.
- **ValueError** – If the number of provided Cartesian axes does not match the robot’s available axes.
- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.lock_cartesian_axes(["X", "Z"])
```

**lock\_joint\_axes(*joint\_axes*)**

Lock specific joint axes while in free drive mode.

**Parameters**

**joint\_axes** (*List[bool]*) – A list of booleans indicating which joint axes should be locked (True for locked, False for free).

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **ValueError** – If the number of provided joint axes does not match the robot's degrees of freedom. For example, if there are more or fewer values than the robot's DOF (degrees of freedom).
- **ValueError** – If an unknown axis is provided when locking Cartesian axes.
- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.lock_joint_axes([True, False, True, False, False, True])
```

**motion\_status()**

Query the motion status of the robot.

**Returns**

The motion status of the robot.

- 'NOT\_RUNNING': If the robot is not running.
- 'RUNNING' : If the robot is running.
- 'PAUSED' : If the robot is in a paused state.

**Return type**

str

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
print(r.motion_status())
```

**move\_circular(\*args, \*\*kwargs)**

To move the robot in a circular path across the given poses.

This method takes the following arguments/keyword arguments:

**Parameters**

- **target\_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default\_value: 0.25, required: No)

- **jerk** (*float*) – Translation Jerk (units: m/sec<sup>3</sup>, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec<sup>2</sup>, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec<sup>3</sup>, default\_value: 500.0, required: No)
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: DYNAMIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.
  - 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
  - 2 : STATIC\_BLENDED, blending based on the given blend\_radius.
- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion(units: N/A, default\_value: False, required: No)
  - True : Weaving is turned on, motion inside are executed with given weaving parameters
  - False : Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)

- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%.
  - False - No reduction in already set max speed.
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- `controller_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 1,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
```

(continues on next page)

(continued from previous page)

```

        -0.48725831508636475
    ],
    [
        0.5190337951593321,
        -0.1969996948428492,
        0.23267853691809767,
        3.1414194107055664,
        -0.00017726201622281224,
        -0.48750609159469604
    ]
],
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}

```

(continues on next page)

(continued from previous page)

```

}
r.move_circular(**circular_property)
r.stop() # if there are multiple motions than, this needs to be called.
        ↪ only once at the end of the script

```

**Example 2:**

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
r.move_circular(["P1", "P2", "P3"])
r.stop()

```

**move\_composite**(\*args, \*\*kwargs)

To move the robot in the specified linear and circular motion combinations.

**Parameters**

- **commands** (*list*) – List of linear and circular command combinations.

**–linear command:**

- \* **targets** (*list*): list of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes)
- \* **velocity** (*float*): Translation Velocity, required when use\_children\_velocities set to True. (units: m/sec, default\_value: speed set for entire MoveComposite, required: No)
- \* **rotation\_velocity** (*float*): Rotational Velocity, required when use\_children\_velocities set to True. (units: rad/sec, default\_value: rotation\_speed set for entire MoveComposite, required: No)

**–circular command:**

- \* **targets** (*list*): list of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes)
- \* **velocity** (*float*): Translation Velocity, required when use\_children\_velocities set to True. (units: m/sec, default\_value: speed set for entire MoveComposite, required: No)
- \* **rotation\_velocity** (*float*): Rotational Velocity, required when use\_children\_velocities set to True. (units: rad/sec, default\_value: rotation\_speed set for entire MoveComposite, required: No)

- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec<sup>2</sup>, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec<sup>3</sup>, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec<sup>2</sup>, default\_value: 1.57, required: No)

- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec<sup>3</sup>, default\_value: 500.0, required: No)
- **use\_children\_velocities** (*bool*) – Toggle whether speed and rotational speed of child motions should be used (units: N/A, default\_value: False, required: No)
  - True : Different speed and rotation speeds are used for each child motion, as set in commands parameter.
  - False : The same speed and rotation speed are used throughout the motion, set with speed and rotation\_speed parameters.
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: STATIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.
  - 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
  - 2 : STATIC\_BLENDED, blending based on the given blend\_radius.
- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion (units: N/A, default\_value: False, required: No)
  - True : Weaving is turned on, motion inside are executed with given weaving parameters
  - False : Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)

- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%
  - False - No reduction in already set max speed
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- `controller_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 2,
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "targets": [
                    -0.000259845199876027,
                    -0.5211437049195536,
                    0.4429382717719519,
                    3.14123272895813,
                    -0.0007908568368293345,
                    -1.570908784866333
                ]
            }
        ]
    ]
}
```

(continues on next page)

(continued from previous page)

```

        -0.16633498440272945,
        -0.5201452059140722,
        0.4427486025872017,
        3.140937089920044,
        -0.0005319403717294335,
        -1.571555495262146
    ]
}
},
{
    "circular": {
        "targets": [
            [
                -0.16633498440272945,
                -0.5201452059140722,
                0.4427486025872017,
                3.140937089920044,
                -0.0005319403717294335,
                -1.571555495262146
            ],
            [
                -0.16540090985202305,
                -0.3983552679378624,
                0.44267608017426174,
                3.1407113075256348,
                -0.00036628879024647176,
                -1.5714884996414185
            ],
            [
                -0.33446498807559716,
                -0.3989652352814891,
                0.4421152856242009,
                3.1402060985565186,
                0.00030071483342908323,
                -1.572899580001831
            ]
        ]
    }
},
],
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",

```

(continues on next page)

(continued from previous page)

```

        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}
r.move_composite(**composite_motion_property)
r.stop() # if there are multiple motions than, this needs to be called,
        ↪ only once at the end of the script

```

**Example 2:**

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 2,
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "use_children_velocities": True,
    "commands": [
        {
            "linear": {

```

(continues on next page)

(continued from previous page)

```

        "targets": [
            [
                0.7006,
                0.0001,
                0.6875,
                3.1416,
                0,
                3.1416
            ],
            [
                0.7006,
                0.0001,
                0.6875,
                3.1416,
                0,
                3.1416
            ],
            [
                0.7509102896938852,
                0.27460943971123764,
                0.6864972996055627,
                3.141592653589793,
                -7.71435533383267e-06,
                -3.1415926535897882
            ],
            [
                0.7747260445464177,
                0.15721988912449852,
                0.8931631506500376,
                -3.1415926535897745,
                -7.714355718231237e-06,
                -3.1415926535897354
            ],
            [
                0.7006,
                0.0001,
                0.6875,
                3.1416,
                0,
                3.1416
            ]
        ],
        "velocity": 0.1,
        "rotation_velocity": 0.174533
    },
    {
        "circular": {
            "targets": [
                [
                    0.7006,
                    0.0001,

```

(continues on next page)

(continued from previous page)

```

        0.6875,
        3.1416,
        0,
        3.1416
    ],
    [
        0.6999986609380567,
        -0.25149522267751856,
        0.6864972999318248,
        -3.141592653589793,
        -7.71435539238761e-06,
        3.1415926535897905
    ],
    [
        0.7747260445464177,
        0.15721988912449852,
        0.8931631506500376,
        -3.1415926535897745,
        -7.714355718231237e-06,
        -3.1415926535897354
    ]
],
"velocity": 0.4,
"rotation_velocity": 0.436332
}
}
],
"weaving": False
}
r.move_composite(**composite_motion_property)
r.stop()

```

**move\_joint**(\*args, \*\*kwargs)

To move the robot to specified joint configuration in joint space.

#### Parameters

- **target\_joint** (*list*) – List of joint configurations (type: List of Joint Values - float or int, units: radians, required: Yes) Alternative: (List of) String (type: String, name of existing point)
- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default\_value: 0.25, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%

- False, No reduction in already set max speed
- **enable\_blending** (*bool*) – Allows to switch between blending and non blending mode. (units: N/A, default\_value: False, required: No)
  - True : Joint Motion does not stop at every point
  - False : Joint Motion stops at every point.
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- `controller_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot
r = Robot()
r.init_program()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
            -1.8651270179353125,
            0.044771940725327274,
            -1.173860821592129,
            0.10315646291502645
        ],
        [
            1.9180047887810003,
            -0.24855170101601043,
```

(continues on next page)

(continued from previous page)

```

        -1.3680228668892351,
        0.12404421791100637,
        -1.1914147150222498,
        -0.13255713717112075
    ]
],
"current_joint_angles": r.get_current_joint_angles(),
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}
r.move_joint(**joint_property)
r.stop() # if there are multiple motions than, this needs to be called,
        ↪ only once at the end of the script

```

**Example 2:**

```

from neurapy.robot import Robot
r = Robot()
r.init_program()
r.move_joint("Home")
r.move_joint(["P1", "P2"])
r.stop()

```

`move_joint_relative(*args, **kwargs)`

To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.

### Parameters

- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default\_value: 25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default\_value: 25, required: No)
- **(units** (*jerk Angular Jerk.*) – % of maximum angular jerk, default value: 25, required No)
- **blocking** (*bool*) – block execution. (units: N/A, default value: True, required: No) - True, after calling move\_linear\_relative, the execution is blocked till the motion is completed - False, after calling move\_linear\_relative, the execution is not blocked
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **offset\_frame** (*str*) – (units: N/A, default\_value: “Base”, required: No) - “Base” : Applies relative offset to the base frame - “Tool” : Applies relative offset to the tool frame
- **cartesian\_offset** (*list*) – The cartesian offset that is applied. (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m - [R,P,Y] in radian, default\_value: [0.0,0.0,0.0,0.0,0.0,0.0], required: Yes)
- **target\_joint** (*list*) – The joint configuration at which the relative motion should start from (type: list - [a1, a2, a3, a4, a5, a6] for lara and [a1, a2, a3, a4, a5, a6, a7] for maira, units: axis values in radian, default: current joint angles, required: No) If the target\_joint is used, the target\_pose also needs to be configured, else the current joint angles and current pose are used
- **target\_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No) If the target\_pose is used, the target\_joint also needs to be configured, else the current joint angle and current pose are used
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.

- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go\_till\_force\_parameters["is\_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go\_till\_force\_parameters["enable\_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go\_till\_force\_parameters["extra\_search\_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go\_till\_force\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.

- `go_till_force_parameters["force_vector"]["Fx"]["is_active"]` (*bool*) – Whether force control is active for the X direction.
- `go_till_force_parameters["force_vector"]["Fx"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["force_vector"]["Fy"]` (*Dict[str, Any]*) – Force control setting in the Y direction.
- `go_till_force_parameters["force_vector"]["Fy"]["is_active"]` (*bool*) – Whether force control is active for the Y direction.
- `go_till_force_parameters["force_vector"]["Fy"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["force_vector"]["Fz"]` (*Dict[str, Any]*) – Force control setting in the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["is_active"]` (*bool*) – Whether force control is active for the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["torque_vector"]` (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- `go_till_force_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.

- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

#### Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
r.init_program()
joint_property = {
    "speed" : 50.0,
    "acceleration" : 50,
    "jerk": 50,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi / 4],
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    },
    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
```

(continues on next page)

(continued from previous page)

```

        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }

r.move_joint_relative(**joint_property)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # if there are multiple motions than, this needs to be called
↳ only once at the end of the script

```

**move\_linear(\*args, \*\*kwargs)**

To move the robot to specified poses in Cartesian/Task space.

**Parameters**

- **target\_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec<sup>2</sup>, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec<sup>3</sup>, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)

- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default\_value: 500.0, required: No)
- **blending** – Blending. (units: N/A, default\_value: False, required: No)
  - True : Blending is turned on, motions inside are executed with given blending mode.
  - False : Blending is turned off, motions stop at each point.
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: DYNAMIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.
  - 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
  - 2 : STATIC\_BLENDED, blending based on the given blend\_radius.
- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion (units: N/A, default\_value: False, required: No)
  - True : Weaving is turned on, motion inside are executed with given weaving parameters
  - False : Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)

- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%.
  - False - No reduction in already set max speed.
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go\_till\_force\_parameters["is\_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go\_till\_force\_parameters["enable\_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go\_till\_force\_parameters["extra\_search\_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go\_till\_force\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.

- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "rotation_speed": 0.5,
    "blending": True,
    "blending_mode": 2,
    "blend_radius": 0.005,
    "target_pose": [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ],
}
```

(continues on next page)

(continued from previous page)

```

[
    0.2093363791501374,
    -0.31711250784165884,
    0.422149168855134,
    -3.0565555095672607,
    -0.3447442352771759,
    -1.1323236227035522
],
[
    0.2090521916195534,
    -0.5246753336643587,
    0.4218773613553828,
    -3.0569007396698,
    -0.3448921740055084,
    -1.1323626041412354
],
[
    0.3287228886,
    -0.1903355329,
    0.4220780352,
    0.08535207028439847,
    -2.797181496822229,
    2.4713321627410485
]
],
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    }
},
"torque_vector": {

```

(continues on next page)

(continued from previous page)

```

        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    },
    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

r.move_linear(**linear_property)
r.stop() # if there are multiple motions than, this needs to be called
         only once at the end of the script

```

**Example 2:**

```

from neurapy.robot import Robot

r = Robot()
r.move_linear(["P1", "P2"])
r.stop()

```

### **move\_linear\_relative(\*args, \*\*kwargs)**

To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion.

#### **Parameters**

- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default\_value: 500.0, required: No)
- **blocking** (*bool*) – block execution. (units: N/A, default value: True, required: No) - True, after calling move\_linear\_relative, the execution is blocked till the motion is completed - False, after calling move\_linear\_relative, the execution is not blocked
- **cartesian\_offset** (*list*) – (type: offset list - [X,Y,Z,R,P,Y], units: [X,Y,Z] offset in m [R,P,Y] offset in radian, default: [.0,.0,.0,.0,.0,.0], required: Yes)
- **offset\_frame** (*str*) – (units: N/A, default\_value: “Base”, required: No) - “Base” : Applies relative offset to the base frame - “Tool” : Applies relative offset to the tool frame
- **target\_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.

- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go\_till\_force\_parameters["is\_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go\_till\_force\_parameters["enable\_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go\_till\_force\_parameters["extra\_search\_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.

- **go\_till\_force\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **go\_till\_force\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **go\_till\_force\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

#### Returns

True if motion is executed successfully, False if motion is not executed successfully

**Return type**

bool

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

**Sample Usage:**

```
import math
from neurapy.robot import Robot

r = Robot()
r.init_program()

linear_property = {
    "speed" : 4.5,
    "acceleration" : 10.0,
    "jerk": 1000.0,
    "rotation_speed" : 1.5,
    "rotation_acceleration" : 10.0,
    "rotation_jerk" : 1000.0,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi/4],
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}
```

(continues on next page)

(continued from previous page)

```

        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    },
    },
    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}

r.move_linear_relative(**linear_property)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # only needs to be executed at the end of the program

```

**move\_pose\_to\_new\_parent\_frame(list\_pose\_id, new\_parent\_id)**

Method to move the input pose(point/frame), which is defined with respect to a parent frame, to a new parent frame. This shall edit the existing pose which retains the local coordinates but with respect to the new parent frame.

**Parameters**

- **list\_pose\_id** (*list[str]*) – The list of ids in DB for the input poses. Even if it is just

one id, pass it in a list.

- **new\_parent\_id** (*str*) – The id in DB for the new parent pose.

#### Raises

**ValueError** – If the input{list\_pose\_id} is not a list or if any element in the list is not a string.

#### Returns

True if operation successful.

#### Return type

bool

### **move\_recorded\_path(\*args, \*\*kwargs)**

To move the robot in a pre-recorded path.

This method takes the following arguments/keyword arguments:

#### Parameters

- **file\_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)
- **is\_const\_velocity** (*bool*) – True if constant velocity is needed during the motion. (required: No)
- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (type: Bool - True/False, units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%
  - False, No reduction in already set max speed

#### Returns

True if motion is executed successfully, False if motion is not executed successfully

#### Return type

bool

#### Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

#### Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
recorded_path_property = {
    "file_name": "./test_paths/test_path1.txt",
    "velocity": 0.25,
```

(continues on next page)

(continued from previous page)

```

        "current_joint_angles": r.get_current_joint_angles(),
    }
    result = r.move_recorded_path(**recorded_path_property)
    r.stop() # if there are multiple motions than, this needs to be called.
    ↪ only once at the end of the script

```

**movelinear\_online(\*args, \*\*kwargs)**

Method to do Servo in Cartesian Space

#### Parameters

- **servo\_target\_position** (*list[float]*) – List of Cartesian target positions in meters. Length should be 7, containing [X, Y, Z, qw, qx, qy, qz].
- **servo\_target\_velocity** (*list[float]*) – List of maximum Cartesian velocities in meters per second. Length should be 6, containing [x\_dot, y\_dot, z\_dot, omega\_x, omega\_y, omega\_z].
- **servo\_target\_acceleration** (*list[float]*) – List of maximum Cartesian accelerations in meters per second squared. Length should be 6, containing [x\_ddot, y\_ddot, z\_ddot, alpha\_x, alpha\_y, alpha\_z].

#### Returns

Servo warning and error codes.

#### Return type

int

#### Sample Usage:

```

from neurapy.robot import Robot
import time
import copy

r = Robot()

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_current_cartesian_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6
acceleration = [2.]*6
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)

target = copy.deepcopy(r.get_current_cartesian_pose())

```

(continues on next page)

(continued from previous page)

```

#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion
time.sleep(5)

print("Robot stopped")
r.deactivate_servo_interface()
r.stop()

```

**notify\_error(msg)**

To notify an error message to teach pendant .

**Parameters**

**msg** (str) – The error message to be sent.

**Returns**

True if successful, else False

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.notify_error("Something went wrong!"):
    print("Error message sent successfully.")
else:
    print("Failed to send error message.")

```

**notify\_info(msg)**

To notify a info message to teach pendant.

**Parameters**

**msg** (str) – The error message to be sent.

**Returns**

True if successful, else False

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.notify_info("This is an info!"):
    print("Info message sent successfully.")
else:
    print("Failed to send info message.")

```

**notify\_warning(msg)**

To notify a warning message to teach pendant.

**Parameters**

**msg** (*str*) – The error message to be sent.

**Returns**

True if successful, else False

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.notify_warning("This is a warning!"):
    print("Warning message sent successfully.")
else:
    print("Failed to send warning message.")
```

**pause**(*\*args, \*\*kwargs*)

To pause the robot's motion.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.pause()
```

**plan\_move\_circular**(*\*args, \*\*kwargs*)

Plan circular motion.

**Parameters**

- **target\_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default\_value: 500.0, required: No)
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: DYNAMIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.
  - 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
  - 2 : STATIC\_BLENDED, blending based on the given blend\_radius.

- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion(units: N/A, default\_value: False, required: No)
  - True - Weaving is turned on, motion inside are exeuted with given weaving parameters
  - False - Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%.
  - False - No reduction in already set max speed.
- **store\_id** (*int*) – identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (default\_value: False )
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.

- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

#### Returns

plan\_id (equal to store\_id if provided in inputs, else an identifier function has generated to store the plan)

**Return type**

int

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

**Sample Usage:**

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 2,
    "blend_radius": 0.05,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    "store_id": 234,
    "current_joint_angles": r.get_current_joint_angles(),
    "weaving": False,
    "pattern": 1,

```

(continues on next page)

(continued from previous page)

```

    "amplitude_left": 0.003,
    "amplitude_right": 0.003,
    "frequency": 1.5,
    "dwell_time_left": 0.0,
    "dwell_time_right": 0.0,
    "elevation": 0.0,
    "azimuth": 0.0,
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

plan_id = r.plan_move_circular(**circular_property)
execute = r.executor([plan_id])

```

**plan\_move\_composite**(\*args, \*\*kwargs)

This method takes the following arguments/keyword arguments:

#### Parameters

- **commands** (*list*) – List of linear and circular command combinations.

#### –linear command:

- \* **targets** (*list*): list of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes)
- \* **velocity** (*float*): Translation Velocity, required when use\_children\_velocities set to

True. (units: m/sec, default\_value: speed set for entire MoveComposite, required: No)

- \* **rotation\_velocity** (float): Rotational Velocity, required when use\_children\_velocities set to True. (units: rad/sec, default\_value: rotation\_speed set for entire MoveComposite, required: No)

**–circular command:**

- \* **targets** (list): list of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes)

- \* **velocity** (float): Translation Velocity, required when use\_children\_velocities set to True. (units: m/sec, default\_value: speed set for entire MoveComposite, required: No)

- \* **rotation\_velocity** (float): Rotational Velocity, required when use\_children\_velocities set to True. (units: rad/sec, default\_value: rotation\_speed set for entire MoveComposite, required: No)

- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default\_value: 500.0, required: No)
- **use\_children\_velocities** (*bool*) – Toggle whether speed and rotational speed of child motions should be used (units: N/A, default\_value: False, required: No)
  - True : Different speed and rotation speeds are used for each child motion, as set in commands parameter.
  - False : The same speed and rotation speed are used throughout the motion, set with speed and rotation\_speed parameters.
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: STATIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.
  - 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
  - 2 : STATIC\_BLENDED, blending based on the given blend\_radius.
- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion(units: N/A, default\_value: False, required: No)
  - True : Weaving is turned on, motion inside are executed with given weaving parameters

- False : Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%
  - False - No reduction in already set max speed
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (default\_value: False )
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.

- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

plan\_id (equal to store\_id if provided in inputs, else an identifier function has generated to store the plan)

**Return type**

int

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

**Sample Usage:****Example 1:**

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 1,
    "blend_radius": 0.01,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "blend_radius": 0.005,
                "targets": [
                    [
                        -0.000259845199876027,
                        -0.5211437049195536,
                        0.4429382717719519,
                        3.14123272895813,
                        -0.0007908568368293345,
                        -1.570908784866333
                    ],
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ]
                ]
            },
            "circular": {
                "targets": [
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ],
                    [
                        -0.16540090985202305,
                        -0.3983552679378624,
                        0.44267608017426174,

```

(continues on next page)

(continued from previous page)

```

        3.1407113075256348,
        -0.00036628879024647176,
        -1.5714884996414185
    ],
    [
        -0.33446498807559716,
        -0.3989652352814891,
        0.4421152856242009,
        3.1402060985565186,
        0.00030071483342908323,
        -1.572899580001831
    ]
]
}
}
],
"store_id": 234,
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },

```

(continues on next page)

(continued from previous page)

```

        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
plan_id= r.plan_move_composite(**composite_motion_property)
execute = r.executor([plan_id])

```

**Example 2:**

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": 2,
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "use_children_velocities": True,
    "commands": [
        {
            "linear": {
                "targets": [
                    [
                        0.7006,
                        0.0001,
                        0.6875,
                        3.1416,
                        0,
                        3.1416
                    ],
                    [
                        0.7006,
                        0.0001,
                        0.6875,
                        3.1416,
                        0,
                        3.1416
                    ],
                    [
                        0.7509102896938852,
                        0.27460943971123764,
                        0.6864972996055627,
                        3.141592653589793,
                        -7.71435533383267e-06,
                        -3.1415926535897882
                    ]
                ]
            }
        }
    ]
}

```

(continues on next page)

(continued from previous page)

```

    ],
    [
        0.7747260445464177,
        0.15721988912449852,
        0.8931631506500376,
        -3.1415926535897745,
        -7.714355718231237e-06,
        -3.1415926535897354
    ],
    [
        0.7006,
        0.0001,
        0.6875,
        3.1416,
        0,
        3.1416
    ]
],
"velocity": 0.1,
"rotation_velocity": 0.174533
},
{
    "circular": {
        "targets": [
            [
                0.7006,
                0.0001,
                0.6875,
                3.1416,
                0,
                3.1416
            ],
            [
                0.6999986609380567,
                -0.25149522267751856,
                0.6864972999318248,
                -3.141592653589793,
                -7.71435539238761e-06,
                3.1415926535897905
            ],
            [
                0.7747260445464177,
                0.15721988912449852,
                0.8931631506500376,
                -3.1415926535897745,
                -7.714355718231237e-06,
                -3.1415926535897354
            ]
        ],
        "velocity": 0.4,
        "rotation_velocity": 0.436332
    }
}

```

(continues on next page)

(continued from previous page)

```

        }
    }
    ],
    "weaving": False
}
plan_id = r.plan_move_composite(**composite_motion_property)
execute = r.executor([plan_id])

```

**plan\_move\_joint**(\*args, \*\*kwargs)

This method takes the following arguments/keyword arguments:

#### Parameters

- **target\_joint** (*list*) – List of joint configurations. (type: List of Joint Values - float, units: radians, required: Yes)
- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default\_value: 0.25, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%
  - False, No reduction in already set max speed
- **enable\_blending** (*bool*) – Allows to switch between blending and non blending mode. (units: N/A, default\_value: False, required: No)
  - True : Joint Motion does not stop at every point
  - False : Joint Motion stops at every point.
- **store\_id** (*int*) – identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (default\_value: False )
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.

- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

plan\_id (equal to store\_id if provided in inputs, else an identifier function has generated to store the plan)

**Return type**

int

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

**Sample Usage:****Example 1:**

```

from neurapy.robot import Robot
r = Robot()
r.init_program()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "enable_blending": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
            -1.8651270179353125,
            0.044771940725327274,
            -1.173860821592129,
            0.10315646291502645
        ],
        [
            1.9180047887810003,
            -0.24855170101601043,
            -1.3680228668892351,
            0.12404421791100637,
            -1.1914147150222498,
            -0.13255713717112075
        ]
    ],
    "store_id": 234,
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
plan_id = r.plan_move_joint(**joint_property)
execute = r.executor([plan_id])

```

**plan\_move\_linear(\*args, \*\*kwargs)**

To plan a move linear path across the given poses.

This method takes the following arguments/keyword arguments:

**Parameters**

- **target\_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default\_value: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default\_value: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default\_value: 500.0, required: No)
- **rotation\_speed** (*float*) – Rotational Speed. (units: rad/sec, default\_value: 0.5, required: No)
- **rotation\_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default\_value: 1.57, required: No)
- **rotation\_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default\_value: 500.0, required: No)
- **blending** – Blending. (units: N/A, default\_value: False, required: No)
  - True : Blending is turned on, motions inside are executed with given blending mode.
  - False : Blending is turned off, motions stop at each point.
- **blending\_mode** (*enum*) – The blending type that is selected to blend between points. (units: N/A, default\_value: DYNAMIC\_BLENDED, required: No)
  - 0 : NO\_BLENDED, if selected goes to the default blending mode.

- 1 : DYNAMIC\_BLENDED, blending based on velocity and acceleration.
- 2 : STATIC\_BLENDED, blending based on the given blend\_radius.
- **blend\_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default\_value: 0.01, required: No)
- **current\_joint\_angles** (*List*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Toggle if Weaving should be used for the Motion(units: N/A, default\_value: False, required: No)
  - True : Weaving is turned on, motion inside are executed with given weaving parameters
  - False : Weaving is turned off, motion does not include Weaving
- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default\_value: 2, required: No)
  - 1 : SINE, if selected uses a Sine wave, ignores dwell times
  - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
  - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude\_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **amplitude\_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default\_value: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default\_value: 10.0, required: No)
- **dwell\_time\_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **dwell\_time\_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default\_value: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rads, default\_value: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rads, default\_value: 0.0, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%.
  - False, No reduction in already set max speed.
- **store\_id** (*int*) – identifier to store the plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (type: Bool - True/False, default\_value: False )
- **controller\_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.

- **controller\_parameters["control\_mode"]** (*str*) – Control mode used to execute the motion. Options include: - “position”: Position control mode. - “joint\_impedance”: Joint impedance control mode. - “hybrid”: Hybrid Motion/Force control mode. - “admittance”: Admittance control mode.
- **controller\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller\_parameters["torque\_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["is\_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller\_parameters["torque\_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

- **go\_till\_force\_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go\_till\_force\_parameters["is\_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go\_till\_force\_parameters["enable\_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go\_till\_force\_parameters["extra\_search\_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go\_till\_force\_parameters["force\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["is\_active"]** (*bool*) – Whether force control is active for the X direction.
- **go\_till\_force\_parameters["force\_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["is\_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go\_till\_force\_parameters["force\_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["force\_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["is\_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go\_till\_force\_parameters["force\_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go\_till\_force\_parameters["torque\_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]["is\_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **go\_till\_force\_parameters["torque\_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go\_till\_force\_parameters["torque\_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **go\_till\_force\_parameters["torque\_vector"]["My"]["is\_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **go\_till\_force\_parameters["torque\_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

**Returns**

`plan_id` (equal to `store_id` if provided in inputs, else an identifier function has generated to store the plan)

**Return type**

`int`

**Raises**

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending": True,
    "blending_mode": 2,
    "blend_radius": 0.01,
    "target_pose": [
        [
            0.3287228886,
            -0.1903355329,
            0.4220780352,
            0.08535207028439847,
            -2.797181496822229,
            2.4713321627410485
        ],
        [
            0.2093363791501374,
            -0.31711250784165884,
            0.422149168855134,
            -3.0565555095672607,
            -0.3447442352771759,
            -1.1323236227035522
        ],
        [

```

(continues on next page)

(continued from previous page)

```

        0.2090521916195534,
        -0.5246753336643587,
        0.4218773613553828,
        -3.0569007396698,
        -0.3448921740055084,
        -1.1323626041412354
    ],
    [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ]
],
"store_id": 234,
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        }
    }
},

```

(continues on next page)

(continued from previous page)

```

        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    },
    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

plan_id = r.plan_move_linear(**linear_property)
execute_motion = r.executor([plan_id]) #To execute the planned id

```

**plan\_move\_recorded\_path**(\*args, \*\*kwargs)

To plan the given pre-recorded path.

This method takes the following arguments/keyword arguments:

#### Parameters

- **file\_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)
- **is\_const\_velocity** (*bool*) – True if constant velocity is needed during the motion. (re-

quired: No)

- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current\_joint\_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default\_value: Joint Configuration obtained from Robot Status method, required: No)
- **enable\_safety** (*bool*) – Safety toggle. (type: Bool - True/False, units: N/A, default\_value: value of the safety toggle in Program screen if not set, required: No)
  - True, Max speed is slashed to 25%
  - False, No reduction in already set max speed
- **store\_id** (*int*) – identifier to store the plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (type: Bool - True/False, default\_value: False)

#### Returns

plan\_id (equal to store\_id if provided in inputs, else an identifier function has generated to store the plan)

#### Return type

int

#### Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

#### Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
recorded_path_property = {
    "file_name": "./test_paths/test_path1.txt",
    "velocity": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
plan_id = r.plan_move_recorded_path(**recorded_path_property)
execute_motion = r.executor([plan_id]) #To execute the planned id
r.stop()
```

#### plan\_realtime\_wait(\*args, \*\*kwargs)

This function can be used to add a wait time to the motion buffer. This is at this time planned but not added to the execution.

#### Parameters

- **motion\_id** (*int*) – The id which the motion should get, this is needed for gui feedback. (units: N/A, default\_value: 0, required: No)
- **blocking** – If the execution of the wait command should be blocked till wait is done. (units: N/A, default\_value: True, required: No)

- **time\_in\_s** (*float*) – The time which the wait command should take. (units: sec, default\_value: 0.0, required: No)

**Sample Usage:****Example 1:**

```
from neurapy.robot import Robot

r = Robot()
wait_property = {
    'motion_id': 1,
    'time_in_s': 0.010,
}
r.plan_realtime_wait(**wait_property)
r.executor() # this is blocking as long as not given a non blocking
↳ command.
r.stop() # if there are multiple motions than, this needs to be called.
↳ only once at the end of the script
```

**power\_off()**

To power off the robot.

**Returns**

True if the robot's power is successfully turned off, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.power_off():
    print("Robot powered off successfully.")
else:
    print("Failed to power off the robot.")
```

**power\_on()**

To power on the robot.

**Returns**

True if the robot's power is successfully turned on, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.power_on():
    print("Robot powered on successfully.")
else:
    print("Failed to power on the robot.")
```

**program\_status()**

Query the program status of the robot.

**Returns**

The program status of the robot.

- 'NOT\_RUNNING': If the program is not running from the Teach pendant.
- 'RUNNING' : If the program is running from the Teach pendant.
- 'PAUSED' : If the program running from the Teach pendant is in a pause state.

**Return type**

str

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
print(r.program_status())
```

**quaternion\_to\_rpy**(w, x, y, z)

Convert a quaternion representation to roll-pitch-yaw (RPY) angles.

**Parameters**

- **w** (*float*) – Scalar component (real) of the quaternion.
- **x** (*float*) – First vector component of the quaternion.
- **y** (*float*) – Second vector component of the quaternion.
- **z** (*float*) – Third vector component of the quaternion.

**Returns**

A list containing [roll, pitch, yaw] angles in radians.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot

r = Robot()
quaternion = r.quaternion_to_rpy(0.85,0,0.52,0)
```

**rad2encoder**(joint\_angles, offset=False)

Convert joint angles in radians to encoder impulses.

**Parameters**

- **joint\_angles** (*list*) – A list of joint angles in radians (required : Yes).
- **offset** (*bool*) – If True considers robot offset values for calculation (Optional).

**Returns**

A list of encoder impulses corresponding to the provided joint angles.

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
joint_angles = r.get_current_joint_angles()
encoder_impulses = r.rad2encoder(joint_angles)
```

**read\_safeio**(\*args, \*\*kwargs)

Read the values of configurable/safe Input/Outputs.

**Parameters**

**io\_name** (*int*) – The identifier of the safe I/O channel to read.

**Returns**

Value of safe I/O.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
safe_io = r.read_safeio(1)
print(safe_io) #True/False
```

**realtime\_wait**(\*args, \*\*kwargs)

This function can be used, to add a wait time to the motion buffer.

**Parameters**

- **motion\_id** (*int*) – The id which the motion should get, this is needed for gui feedback. (units: N/A, default\_value: 0, required: No)
- **blocking** – If the execution of the wait command should be blocked till wait is done. (units: N/A, default\_value: True, required: No)
- **time\_in\_s** (*float*) – The time which the wait command should take. (units: sec, default\_value: 0.0, required: No)

**Sample Usage:**

**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
wait_property = {
    'motion_id': 1,
    'blocking': True,
    'time_in_s': 0.010,
}
r.realtime_wait(**wait_property)
r.stop() # if there are multiple motions than, this needs to be called
        ↪ only once at the end of the script
```

**release()**

To release the grasped object

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.release()
```

**remove\_load()**

Remove the load which was set in set\_load by setting the load to NoLoad.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.remove_load()
```

**replicate\_list\_poses\_in\_new\_parent\_frame**(*list\_pose\_info: list, new\_parent\_id*)

Method to replicate the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame. This shall create a new pose which retains the local coordinates but with respect to the new parent frame.

**Parameters**

- **list\_pose\_info** (*list*) – The list of dictionaries containing pose\_id and new\_pose\_name. pose\_id (str): The input pose id in DB. new\_pose\_name (str): The name to the newly created pose which is a replica of the input pose in the new parent.
- **new\_parent\_id** (*str*) – The new parent frame id in DB with respect to which the new pose shall be created.

**Raises**

**ValueError** – If the input{list\_pose\_info} is not a list or if any element in the list is not a dict.

**Returns**

True if operation successfull.

**Return type**

bool

**replicate\_pose\_in\_new\_parent\_frame**(*pose\_id, new\_pose\_name, new\_parent\_id*)

Method to replicate the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame. This shall create a new pose which retains the local coordinates but with respect to the new parent frame.

**Parameters**

- **pose\_id** (*str*) – The input pose id in DB.
- **new\_pose\_name** (*str*) – The name to the newly created pose which is a replica of the input pose in the new parent.
- **new\_parent\_id** (*str*) – The new parent frame id in DB with respect to which the new pose shall be created.

**Returns**

True if operation successfull.

**Return type**

bool

**replicate\_pose\_in\_new\_parent\_frame\_and\_delete\_original**(*pose\_id, new\_pose\_name, new\_parent\_id*)

Replicates the input pose(point/frame), which is defined with respect to a parent frame, in a new parent frame. This shall create a new pose which retains the local coordinates but with respect to the new parent frame. En suite, it deletes the input pose.

#### Parameters

- **pose\_id** (*str*) – The input pose id in DB.
- **new\_pose\_name** (*str*) – The name to the newly created pose which is a replica of the input pose in the new parent.
- **new\_parent\_id** (*str*) – The new parent frame id in DB with respect to which the new pose shall be created.

#### Returns

True if operation successful.

#### Return type

bool

#### **reset\_axes\_lock()**

Reset all joint and Cartesian axis locks in free drive mode.

#### Returns

True if the operation is executed successfully, False otherwise.

#### Return type

bool

#### Raises

- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.
- **ConnectionError** – If an internal error occurs while resetting axis locks.

#### Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.lock_cartesian_axes(["X", "Z"])
r.reset_axes_lock()
```

#### **reset\_collision()**

Reset the collision state of the robot.

#### Returns

True if the robot is not in collision, False otherwise.

#### Return type

bool

#### Sample Usage

```
from neurapy.robot import Robot
r = Robot()
r.reset_collision()
```

**reset\_control()**

To restart the control software running on the robot. Equivalent to reset control option from Teach pendant

**Returns**

True if the control reset was successful, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.reset_control():
    print("Control reset successful.")
else:
    print("Control reset failed.")
```

**reset\_errors(\*args, \*\*kwargs)**

Method to reset the errors on the robot

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
r.reset_error()
```

**reset\_led\_mode()**

Reset the LED mode to its default state.

**Returns**

Result of the LED reset execution.

**Return type**

bool

**Raises**

**RuntimeError** – If LED mode cannot be controlled in the current version of Control Software.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.reset_led_mode():
    print("LED reset to default state.")
else:
    print("Failed to reset LED.")
```

**reset\_warnings()**

Method to reset/clear the warnings on the robot

**Sample Usage:**

```
import time
from neurapy.robot import Robot
```

(continues on next page)

(continued from previous page)

```
r = Robot()
r.reset_warnings()
```

**rpy\_to\_quaternion**(*r, p, y*)

Convert roll-pitch-yaw (RPY) angles to a quaternion representation.

**Parameters**

- **r** (*float*) – Roll angle in radians.
- **p** (*float*) – Pitch angle in radians.
- **y** (*float*) – Yaw angle in radians.

**Returns**

A list representing the quaternion [w, x, y, z].

**Return type**

list

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
rpy = r.rpy_to_quaternion(0,1.1,0)
```

**save\_new\_pose\_in\_DB**(*input\_pose\_in\_parent, is\_kinematic\_check\_active=True*)

Saves a new pose in world frame in DB. Converts the input pose that is in parent to world frame prior to saving.

**Parameters**

- **input\_pose\_in\_parent** (-) – The json structure received from GUI for a pose which is defined in its parent frame.
- **is\_kinematic\_check\_active** (-) – Flag which shall check before saving if the joint\_angles are coherent with the pose values. Shall be applicable only for points. Shall raise a runtime error in case of check failure. Default True

**Returns**

- bool: True if the method is successful.
- int: Status code of the POST request.
- dict: Response of the POST request.

**Return type**

tuple

**save\_or\_update\_pose\_in\_DB**(*input\_pose\_in\_parent, is\_debug=False, is\_kinematic\_check\_active=True*)

Checks if a pose with the same name as the input already exists in DB. If exists, then update the existing pose else create new pose.

**Parameters**

- **input\_pose\_in\_parent** (*dict*) – The pose that shall be saved or updated in DB.
- **is\_debug** (*bool*) – Flag to print the debug statements. Default False.

- **is\_kinematic\_check\_active** (*bool*) – Flag which shall check before saving if the joint\_angles are coherent with the pose values. Shall be applicable only for points. Shall raise a runtime error in case of check failure. Default True

**Returns**

True if successfull

**Return type**

bool

**servo\_j**(\*args, \*\*kwargs)

Method to do Servo in Joint Space

**Parameters**

- **servo\_target\_position** (*list[float]*) – List of joint target positions in radians. Length of list should be equal to the degree of freedom.
- **servo\_target\_velocity** (*list[float]*) – List of joint target velocities in radians per second. Length of list should be equal to the degree of freedom.
- **servo\_target\_acceleration** (*list[float]*) – List of joint target accelerations in radians per second squared. Length of list should be equal to the degree of freedom.

**Returns**

Servo warning and error codes.

**Return type**

int

**Sample Usage:**

```
from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]
inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
```

(continues on next page)

(continued from previous page)

```

res = Result.Working

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

**servo\_x**(\*args, \*\*kwargs)

Method to do Servo in Cartesian Space

#### Parameters

- **servo\_target\_position** (*list[float]*) – List of Cartesian target positions in meters. Length of list should be 7, containing [X, Y, Z, qw, qx, qy, qz].
- **servo\_target\_velocity** (*list[float]*) – List of Cartesian target velocities in meters per second and radians per second. The first three elements represent translational velocity (m/s), and the next three elements represent rotational velocity (rad/s).
- **servo\_target\_acceleration** (*list[float]*) – List of Cartesian target accelerations in meters per second squared and radians per second squared. The first three elements represent translational acceleration ( $\text{m/s}^2$ ), and the next three elements represent rotational acceleration ( $\text{rad/s}^2$ ).
- **gain\_parameter** (*float*) – ServoX gain parameter used for the proportional controller; should be within the range [0.2, 100].

#### Returns

Servo warning and error codes.

#### Return type

int

#### Sample Usage:

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig
import copy

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

cart_pose_length = 7 #X,Y,Z,qw,qx,qy,qz

otg = Ruckig(cart_pose_length, 0.001) # control cycle
inp = InputParameter(cart_pose_length)
out = OutputParameter(cart_pose_length)

inp.current_position = r.get_current_cartesian_pose()
inp.current_velocity = [0.]*cart_pose_length
inp.current_acceleration = [0.]*cart_pose_length

target = copy.deepcopy(inp.current_position)
target[0] += 0.2 # Move 200mm in X direction
inp.target_position = target
inp.target_velocity = [0.]*cart_pose_length
inp.target_acceleration = [0.]*cart_pose_length

inp.max_velocity = [0.5]*cart_pose_length
inp.max_acceleration = [3]*cart_pose_length
inp.max_jerk = [10.]*cart_pose_length
res = Result.Working

servox_proportional_gain = 25

velocity = [0.] * 6 #Since ruckig does not provide rotational velocity,
↪if quaternion is input, we can send 0 rotational feedforward velocity
acceleration = [0.] * 6 #Since ruckig does not provide rotational,
↪acceleration if quaternion is input, we can send 0 rotational,
↪feedforward acceleration

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position

        for i in range(0,3): # Updating target translation velocity and,
↪accelerations
            velocity[i] = out.new_velocity[i]

```

(continues on next page)

(continued from previous page)

```

        acceleration[i] = out.new_acceleration[i]

        error_code = r.servo_x(position, velocity, acceleration, servox_
→proportional_gain)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

**set\_analog\_output**(*io\_name*, *target\_value*)

Set the analog outputs of the control box.

Use the `get_io_configuration()` function to determine the number of available I/Os.

#### Parameters

- **io\_name** (*int*) – The pin number of the analog output to set.
- **target\_value** (*float*) – The target value to set for the analog output.

#### Returns

True if the operation succeeded, False otherwise.

#### Return type

bool

#### Sample Usage:

```

import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_analog_output(1, 3.2)
print(io_set)

```

**set\_current\_position\_as\_zero**(*list\_of\_axis*, *pass\_phrase*=None)

Sets the current joint position of the given axis to zero.

#### Warning:

This function sets the current joint position to zero. Incorrect usage of this function, such as setting incorrect encoder values, may result in serious unexpected behavior of the robot.

#### Parameters

- **list\_of\_axis** (*list*) – A list of axes that need to be set to zero with values (bool) for each joint.
- **pass\_phrase** (*str*) – The password which is required to call this function.

#### Returns

True if the joint position was successfully updated.

**Return type**

bool

**Raises**

- **ValueError** – If the length of the provided encoder offsets list is not equal to the number of joints on the robot.
- **ConnectionError** – If there is a failure to connect to the robot or the database, or if there is an issue updating the encoder offsets, or if the pass\_phrase is not correct.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.set_current_position_as_zero([False, True, False, True, False, False],
    ↪pass_phrase = userpassword)
```

**set\_dblout\_register**(*first\_element*, *last\_element*, *\*values*)

Writes values into an element range of the DBLOUT register

**Parameters**

- **first\_element** (*int*) – Index of the first integer DBLOUT element to write into.
- **last\_element** (*int*) – Index of the last integer DBLOUT element to write into.
- **values** (*float*) – Target values as arguments.

**Returns**

Number of elements written, or 0 in case of failure (invalid range, communication failure).

**Return type**

int

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
num_written = r.set_dblout_register(0, 2, 3.3, 6.6, 9.9) # writes three
    ↪floating point values
                                     # into the first three elements of DBLOUT
```

**set\_digital\_output**(*io\_name*, *target\_value*)

Set the digital outputs of the control box.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

- **io\_name** (*int*) – The number of the digital output to set.
- **target\_value** (*bool*) – The target value to set for the digital output (True or False).

**Raises**

**TypeError** – If *io\_name* is not an integer or *target\_value* is not a boolean.

**Returns**

True if the digital output is set successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_digital_output(1, True)
print(io_set)
```

**set\_encoder\_offsets**(*list\_of\_encoder\_offsets*, *relative=False*, *pass\_phrase=None*)

**Warning:**

This function sets the encoder values of joints. Incorrect usage of this function, such as setting incorrect encoder values, may result in serious unexpected behavior of the robot.

Sets the encoder offsets for the robot's joints relative or absolute.

**Parameters**

- **list\_of\_encoder\_offsets** (*list*) – A list of encoder offset values (int) for each joint. The length of the list must be equal to the number of joints on the robot.
- **relative** (*bool*) – Defines if the given encoder offset should be added relative to the current encoder offsets or set as absolute encoder values.
- **pass\_phrase** (*str*) – The password which is required to call this function.

**Returns**

True if the encoder offsets were successfully updated, False otherwise.

**Return type**

bool

**Raises**

- **ValueError** – If the length of the provided encoder offsets list is not equal to the number of joints on the robot.
- **ConnectionError** – If there is a failure to connect to the robot or the database, or if there is an issue updating the encoder offsets, or if the *pass\_phrase* is not correct, or if the encoder offset should be applied relative.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.set_encoder_offsets([#encoder offsets from the provided robot manual/
↳ data sheet], relative = False, pass_phrase = userpassword)
```

**set\_ext\_digital\_output**(*io\_name*, *target\_value*)

Set the digital outputs of the external I/O module.

Use the `get_io_configuration()` function to determine the number of available I/Os.

**Parameters**

- **io\_name** (*int*) – The pin number of the digital output to set.
- **target\_value** (*bool*) – The target value to set for the digital output (True or False).

**Raises**

**TypeError** – If *io\_name* is not an integer or *target\_value* is not a boolean.

**Returns**

True if the digital output is set successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_ext_digital_output(1, True)
print(io_set)
```

**set\_intout\_register**(*first\_element*, *last\_element*, *\*values\_or\_bytes*)

Writes values or octets into an element range of the INTOUT register

**Parameters**

- **first\_element** (*int*) – Index of the first integer INTOUT element to write into.
- **last\_element** (*int*) – Index of the last integer INTOUT element to write into.
- **values\_or\_bytes** (*bytes/int*) – Either one Python “bytes” argument or target values as arguments.

**Returns**

Number of elements written, or 0 in case of failure (invalid range, communication failure).

**Return type**

int

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
num_written = r.set_intout_register(2, 3, b'{{{yyyy}}}') # writes 8 octets
# given as a binary bytes() into the register.
↳ storage of INTOUT elements 2 and 3
num_written = r.set_intout_register(0, 2, 14, 42, 84) # writes three
↳ integers into the first three
# elements of INTOUT
```

**set\_joint\_acceleration**(*acceleration*)

Set the acceleration value for joint motion.

**Parameters**

**acceleration** (*float*) – The acceleration value for joint motion, ranging from 0 to 100 (percentage of maximum angular acceleration).

**Raises**

- **TypeError** – If acceleration is not a numeric value (int or float).
- **ValueError** – If acceleration is outside the range of 0 to 100.

**Returns**

None

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.set_joint_acceleration(50) # Set joint acceleration to 50% of maximum
```

**set\_joint\_speed(speed)**

Set the speed value for joint motion.

**Parameters**

**speed** (*float*) – The speed value for joint motion, ranging from 0 to 100 (percentage of maximum angular speed).

**Raises**

- **TypeError** – If speed is not a numeric value (int or float).
- **ValueError** – If speed is outside the range of 0 to 100.

**Returns**

None

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.set_joint_speed(75) # Set joint speed to 75% of maximum
```

**set\_led\_mode(mode, color, frequency)**

Turn on the LED of the robot using the specified mode, a list of colors, and frequency.

**Parameters**

- **mode** (*int*) – LED operating mode. Allowed values:  
0 - LED\_MODE\_FLASH 1 - LED\_MODE\_HEARTBEAT 2 - LED\_MODE\_CYCLE  
(only mode that supports multiple colors) 3 - LED\_MODE\_SOLID
- **color** (*list[int]*) – List of LED colors. - If mode is LED\_MODE\_CYCLE (2), multiple colors may be provided and will be cycled. - For all other modes, the list must contain exactly one color. Allowed values for each item:  
0 - LED\_COLORLESS 1 - LED\_GREEN 2 - LED\_RED 3 - LED\_YELLOW 4 - LED\_BLUE 5 - LED\_LIGHTBLUE 6 - LED\_MAGENTA 7 - LED\_WHITE
- **frequency** (*float*) – Flashing or cycling frequency (in Hz). Relevant when mode is FLASH or CYCLE.

**Returns**

Result of the LED mode execution.

**Return type**

bool

**Raises**

- **TypeError** – If input types are invalid.
- **ValueError** – If values are outside accepted ranges or structure.
- **RuntimeError** – If LED mode cannot be controlled in the current version of Control Software.

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
# Cycle through GREEN, RED, and BLUE at 2.5 Hz
if r.set_led_mode(2, [1, 2, 4], 2.5):
    print("LED pattern applied.")
else:
    print("Failed to apply LED pattern.")

```

**set\_linear\_acceleration(*acceleration*)**

Set the acceleration value for linear motion.

**Parameters**

**acceleration** (*float*) – The acceleration value for linear motion, ranging from 0.0 to 1.0 (m/s<sup>2</sup>).

**Raises**

- **TypeError** – If acceleration is not a numeric value (int or float).
- **ValueError** – If acceleration is outside the range of 0.0 to 1.0.

**Returns**

None

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.set_linear_acceleration(0.5) # Set linear acceleration to 50%

```

**set\_linear\_speed(*speed*)**

Set the speed value for linear motion.

**Parameters**

**speed** (*float*) – The speed value for linear motion, ranging from 0.0 to 1.0 (m/s).

**Raises**

- **TypeError** – If speed is not a numeric value (int or float).
- **ValueError** – If speed is outside the range of 0.0 to 1.0.

**Returns**

None

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.set_linear_speed(0.75) # Set linear speed to 0.75 m/s

```

**set\_load(*load\_name*, *load\_mass*=None, *load\_cog*=None, *load\_inertia*=None)**

Sets a load, e.g., an object that is grasped. If the load is present in the database, it is updated. If it is not present, it is first added to the database and then updated.

**Parameters**

- **load\_name** (*string*, *required*) – String representing the payload name

- **load\_mass** (*float, optional*) – Mass of load in kg
- **load\_cog** (*list of float, optional*) – Center of gravity of the payload, specified as [x, y, z] coordinates in meters.
- **load\_inertia** (*list of float, optional*) – Upper triangular part of the inertia matrix, specified in the order [xx, yy, zz, xy, xz, yz].

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_load(load_name = 'NewLoad', load_mass=1.1, load_cog=[0.02, 0.03, 0.
↪04], load_inertia=[0.01, 0.02, 0.03, 0.04, 0.05, 0.06])
```

**set\_opcua\_msg** (\*args, \*\*kwargs)

Send a message to the OPC UA server running on the control box.

#### Parameters

- **message** (*str*) – The message to be sent to the OPC UA server.
- **opcua\_register\_index** (*int*) – The index of the OPC UA register where the message will be stored.

#### Returns

True if the message is successfully sent and stored in the specified register, False otherwise.

#### Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
message = "Hello, OPC UA Server!"
register_index = 1 # Replace with the index of the target OPC UA_
↪register
if r.set_opcua_msg(message, register_index):
    print("Message sent and stored in OPC UA register successfully.")
else:
    print("Failed to send the message or store it in the OPC UA register.
↪")
```

**set\_override**(*value*)

Set the override value.

#### Parameters

- **value** (*int*) – The new override value to set.

#### Returns

True if the override value is successfully set, False otherwise.

#### Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
new_override_value = 0.5 # it should be from 0 to 1
if r.set_override(new_override_value):
    print(f"Override value set to {new_override_value} successfully.")
else:
    print("Failed to set the override value.")

```

**set\_safety**(*safety: bool*)

**set\_tool**(\*args, \*\*kwargs)

Notify the gripper/tool change to software components after the physical change.

**Parameters**

**tool\_name** (*str*) – The name of the tool defined from the GUI.

**Returns**

True if the operation succeeds, False if the operation fails.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.set_tool(tool_name="name given during tool creation in GUI")

```

**set\_tool\_digital\_output**(*io\_number, target\_value*)

Set the target value on the specified tool digital output.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

- **io\_number** (*int*) – The number of the tool digital output to be triggered.
- **target\_value** (*bool*) – The target value to set for the tool digital output.

**Returns**

True if the operation succeeded, False otherwise.

**Return type**

bool

**Sample Usage:**

```

import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_tool_digital_output(1, True) # To set tool digital output.
↪ 1 to high
print(io_set)

```

**set\_tool\_digital\_outputs**(*target\_value*)

Set digital outputs of the tool.

Use the [get\\_io\\_configuration\(\)](#) function to determine the number of available I/Os.

**Parameters**

**target\_value** (*list of float*) – A list of target values to set for the tool digital outputs.

**Returns**

True if the operation succeeded, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_tool_digital_outputs([0.0,1.0,0.0]) # To set tool digital_
↪output to high
print(io_set)
```

**stop(\*args, \*\*kwargs)**

To stop the robot's motion and to terminate the script execution. This needs to be added at the end of the script(only once) for a proper script termination

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.stop()
```

**stop\_movelinear\_online(\*args, \*\*kwargs)**

Method to stop MoveLinearOnline motion if running

**Sample Usage**

```
from neurapy.robot import Robot
import time
import copy

r = Robot()

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_current_cartesian_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6
acceleration = [2.]*6
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)
```

(continues on next page)

(continued from previous page)

```

target = copy.deepcopy(r.get_current_cartesian_pose())
#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 2 sec to complete the motion
time.sleep(2)

#Stop the movelinear online motion
r.stop_movelinear_online()
time.sleep(3)
print("Robot stopped")
r.deactivate_servo_interface()
r.stop()

```

**switch\_to\_automatic\_mode()**

Switch the robot to Automatic mode.

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
if r.switch_to_automatic_mode():
    print("Robot switched to Automatic mode.")
else:
    print("Failed to switch to Automatic mode.")

```

**switch\_to\_real()**

Switches the robot's running context to real mode.

**Returns**

True if the switch was successful, False otherwise.

**Return type**

bool

**Sample Usage:**

```

from neurapy.robot import Robot
r = Robot()
r.switch_to_real()

```

**switch\_to\_semi\_automatic\_mode()**

Switch the robot to Semi Automatic mode.

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.switch_to_semi_automatic_mode():
    print("Robot switched to Semi Automatic mode.")
else:
    print("Failed to switch to Semi Automatic mode.")
```

**switch\_to\_simulation()**

Switches the robot's running context to simulation mode.

**Returns**

True if the switch was successful, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.switch_to_simulation()
```

**switch\_to\_teach\_mode()**

Switch the robot to Teach mode.

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
if r.switch_to_teach_mode():
    print("Robot switched to Teach mode.")
else:
    print("Failed to switch to Teach mode.")
```

**tare\_fts()**

Set the property for taring force torque sensor property to true when called and clears the force torque sensor offsets.

For lara the *tare\_fts\_* property in *rtt\_elfin* component is set to true and for maira *tare\_fts\_* property in the *rtt\_xenoff* component is set to true

**Returns**

None

**transform\_pose\_to\_reference(*pose\_json\_in\_parent*, *old\_reference\_id*)**

Transforms the pose data in {*pose\_json\_in\_parent*}, which is in its parent frame {*old\_reference\_id*}, to a target reference frame contained in {*pose\_json\_in\_parent*}. No other DB handling is performed apart from reading the reference frame pose from DB. Caution: joint angles in the returned value do not correspond to the pose.

**Parameters**

- **pose\_json\_in\_parent** (*dict*) – The pose data in its parent frame received from GUI. The referenceFrame contained in this dict is not the parent frame of the pose contained but rather the new reference frame into which the input should be transformed to.
- **old\_reference\_id** (*str*) – The parent frame for the input pose {pose\_json\_in\_parent}.

**Returns**

The input {pose\_json\_in\_parent} updated with the transformed pose data in the specified reference frame.

**Return type**

dict

**turn\_off\_free\_drive\_mode()**

To turn off free drive mode

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **WrongMode** – If the Robot is not in ‘teach’ mode or in simulation.
- **ConnectionError** – If the Robot is powered off.

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_off_free_drive_mode()
```

**turn\_off\_jog(\*args, \*\*kwargs)**

Method to disable jogging programatically. This needs to be used in conjunction with jog, turn\_on\_jog methods.

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
    'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
while i < 500:
    r.jog(set_jogging_external_flag=1)
    i += 1
r.turn_off_jog()
```

**turn\_on\_free\_drive\_mode()**

To turn on free drive mode

**Returns**

True if the operation is executed successfully, False otherwise.

**Return type**

bool

**Raises**

- **WrongMode** – If the Robot is not in ‘teach’ mode or in simulation.
- **ConnectionError** – If the Robot is powered off.

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_free_drive_mode()
```

**turn\_on\_jog(\*args, \*\*kwargs)**

Method to enable jogging programatically. This needs to be used in conjunction with jog, turn\_off\_jog methods.

Args: - jog\_velocity (list of float): A list of joint velocities ranging from -1 to 1 for all joints. - jog\_type (str): Specifies the type of jogging, which can be ‘Cartesian’ or ‘Joint’.

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
↳ 'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
while i < 500:
    r.jog(set_jogging_external_flag=1)
    i += 1
r.turn_off_jog()
```

**unpause(\*args, \*\*kwargs)**

To unpause the robot’s motion.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.unpause()
```

**update\_current\_tool\_parameters(\*args, \*\*kwargs)**

Update the current gripper/tool parameters in the tool list and the selected tool properties.

**Parameters**

- **\_control0A** (bool) – True if the tool is controlled by controlbox analog outputs; False otherwise. (Not Required)
- **\_control0D** (bool) – True if the tool is controlled by controlbox digital outputs; False otherwise. (Not Required)
- **\_tool0A** (bool) – True if the tool is controlled by analog outputs of port present on robot tool flange; False otherwise. (Not Required)

- **\_toolOD** (*bool*) – True if the tool is controlled by digital outputs of port present on robot tool flange; False otherwise. (Not Required)
- **autoM** (*float*) – Mass of the tool. (Required)
- **autoMeasureX** (*float*) – Tool center of gravity (COG) in X direction, measured from robot's flange frame. (Not Required)
- **autoMeasureY** (*float*) – Tool center of gravity (COG) in Y direction, measured from robot's flange frame. (Not Required)
- **autoMeasureZ** (*float*) – Tool center of gravity (COG) in Z direction, measured from robot's flange frame. (Not Required)
- **closeInput** (*float*) – Percentage of gripper width for close action. (Not Required)
- **cmdID** (*int*) – Input specific for certain kinds of grippers. (Not Required)
- **force** (*float*) – Gripper closing force. (Not Required)
- **gripper** (*str*) – Gripper type. (Not Required)
- **grippertype** (*str*) – Type of gripper Modbus/Standard. (Not Required)
- **inertiaXX** (*float*) – Ixx of tool. (Not Required)
- **inertiaXY** (*float*) – Ixy of tool. (Not Required)
- **inertiaXZ** (*float*) – Ixz of tool. (Not Required)
- **inertiaYY** (*float*) – Iyy of tool. (Not Required)
- **inertiaYZ** (*float*) – Iyz of tool. (Not Required)
- **inertiaZZ** (*float*) – Izz of tool. (Not Required)
- **offCOA** (*list*) – Offset for control analog outputs. (Not Required)
- **offCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offCOD2** (*int*) – (Not Required)
- **offTOA** (*list*) – Offset for tool analog outputs. (Not Required)
- **offTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offsetA** (*float*) – TCP roll offset. (Not Required)
- **offsetB** (*float*) – TCP pitch offset. (Not Required)
- **offsetC** (*float*) – TCP yaw offset. (Not Required)
- **offsetX** (*float*) – TCP offset in X. (Not Required)
- **offsetY** (*float*) – TCP offset in Y. (Not Required)
- **offsetZ** (*float*) – TCP offset in Z. (Not Required)
- **onCOA** (*list*) – Offset for control analog outputs when turning on. (Not Required)
- **onCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **onCOD2** (*int*) – (Not Required)
- **onTOA** (*list*) – Offset for tool analog outputs when turning on. (Not Required)

- **onTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **openInput** (*float*) – Percentage of gripper width for open action. (Not Required)
- **portID** (*int*) – (Not Required)
- **protocol** (*int*) – (Not Required)
- **robot\_type** (*str*) – Type of robot tool. (Not Required)
- **slaveID** (*int*) – (Not Required)
- **speed** (*float*) – (Not Required)

**Returns**

True if the operation succeeds, False if the operation fails.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.update_current_tool_parameters(offsetZ=0.1, autoM=0.5)
```

**update\_pose\_in\_DB**(*input\_pose\_in\_parent*, *is\_debug=False*)

Updates an existing pose in DB by converting the input which is defined in its parent frame to world frame. This method shall take care of the recursive updation of the child poses associated with the input.

**Parameters**

- **input\_pose\_in\_parent** (*json*) – The json structure received from GUI for a pose which is defined in its parent frame.
- **is\_debug** (*bool*) – Flag to print the debug statements. Default False.

**Returns**

True if the method is successful.

**Return type**

bool

**update\_tool\_parameters**(*\*args*, *\*\*kwargs*)

Update the gripper/tool parameters in the tool list with a given tool\_name.

**Parameters**

- **tool\_name** (*str*) – The name of the tool defined from the GUI.
- **\_control0A** (*bool*) – True if the tool is controlled by controlbox analog outputs; False otherwise. (Not Required)
- **\_control0D** (*bool*) – True if the tool is controlled by controlbox digital outputs; False otherwise. (Not Required)
- **\_tool0A** (*bool*) – True if the tool is controlled by analog outputs of port present on robot tool flange; False otherwise. (Not Required)
- **\_tool0D** (*bool*) – True if the tool is controlled by digital outputs of port present on robot tool flange; False otherwise. (Not Required)
- **autoM** (*float*) – Mass of the tool. (Required)

- **autoMeasureX** (*float*) – Tool center of gravity (COG) in X direction, measured from robot's flange frame. (Not Required)
- **autoMeasureY** (*float*) – Tool center of gravity (COG) in Y direction, measured from robot's flange frame. (Not Required)
- **autoMeasureZ** (*float*) – Tool center of gravity (COG) in Z direction, measured from robot's flange frame. (Not Required)
- **closeInput** (*float*) – Percentage of gripper width for close action. (Not Required)
- **cmdID** (*int*) – Input specific for certain kinds of grippers. (Not Required)
- **force** (*float*) – Gripper closing force. (Not Required)
- **gripper** (*str*) – Gripper type. (Not Required)
- **grippertype** (*str*) – Type of gripper Modbus/Standard. (Not Required)
- **inertiaXX** (*float*) – Ixx of tool. (Not Required)
- **inertiaXY** (*float*) – Ixy of tool. (Not Required)
- **inertiaXZ** (*float*) – Ixz of tool. (Not Required)
- **inertiaYY** (*float*) – Iyy of tool. (Not Required)
- **inertiaYZ** (*float*) – Iyz of tool. (Not Required)
- **inertiaZZ** (*float*) – Izz of tool. (Not Required)
- **offCOA** (*list*) – Offset for control analog outputs. (Not Required)
- **offCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offCOD2** (*int*) – (Not Required)
- **offTOA** (*list*) – Offset for tool analog outputs. (Not Required)
- **offTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offsetA** (*float*) – TCP roll offset. (Not Required)
- **offsetB** (*float*) – TCP pitch offset. (Not Required)
- **offsetC** (*float*) – TCP yaw offset. (Not Required)
- **offsetX** (*float*) – TCP offset in X. (Not Required)
- **offsetY** (*float*) – TCP offset in Y. (Not Required)
- **offsetZ** (*float*) – TCP offset in Z. (Not Required)
- **onCOA** (*list*) – Offset for control analog outputs when turning on. (Not Required)
- **onCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **onCOD2** (*int*) – (Not Required)
- **onTOA** (*list*) – Offset for tool analog outputs when turning on. (Not Required)
- **onTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **openInput** (*float*) – Percentage of gripper width for open action. (Not Required)

- **portID** (*int*) – (Not Required)
- **protocol** (*int*) – (Not Required)
- **robot\_type** (*str*) – Type of robot tool. (Not Required)
- **slaveID** (*int*) – (Not Required)
- **speed** (*float*) – (Not Required)

**Returns**

True if the operation succeeds, False if the operation fails.

**Return type**

bool

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
r.update_tool_parameters(tool_name="name given during tool creation in GUI", offsetZ=0.1, autoM=0.5)
```

**wait**(\*args, \*\*kwargs)

Wait for a specified signal.

**Parameters**

- **port\_name** (*str*) – The name or identifier of the tool-specific analog input.
- **expected\_value** (*float*) – The expected value of the analog input. If None, the function waits for any change in the input value.

**Returns**

True if the specified condition or event is met, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait(port_name="DI_2", expected_value=1.0, wait_for_
↳ signal=True, wait_time=0.0) #waits till digital input 2 is True
print(io_wait)
```

**wait\_for\_analog\_input**(*io\_name*, *expected\_value*, *wait\_time*=0.0)

Wait for an analog input signal to match the expected value.

**Parameters**

- **io\_name** (*str*) – The name or identifier of the analog input.
- **expected\_value** (*float*) – The expected value of the analog input.
- **wait\_time** (*float*) – The amount of time (in seconds) to wait before checking the analog input signal. Default is 0.0 seconds.

**Returns**

True if the analog input matches the expected value, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_analog_input(1,2.3) #waits till analog input 1 is 2.
↪3
print(io_wait)
```

**wait\_for\_digital\_input**(*io\_name*, *expected\_value*, *wait\_time*=0.0)

Wait for a digital input signal to match the expected value.

**Parameters**

- **io\_name** (*str*) – The name or identifier of the digital input.
- **expected\_value** (*bool*) – The expected value of the digital input.
- **wait\_time** (*float*) – The amount of time (in seconds) to wait before returning. Default is 0.0 seconds.

**Returns**

True if the digital input matches the expected value, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_digital_input(1,True) #waits till digital input 1
↪is True
print(io_wait)
```

**wait\_for\_digital\_input\_timer\_off\_delay**(*io\_name*, *delay*=0.0)

Wait for a given delay and returns after the specified digital input signal reaches low.

**Parameters**

- **io\_name** (*str*) – The name or identifier of the digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

**Returns**

True if the digital input signal reaches low, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
```

(continues on next page)

(continued from previous page)

```
io_wait = r.wait_for_digital_input_timer_off_delay(1,10) #waits for
↳ 10sec and returns after digital input 1 reaches low
print(io_wait)
```

**wait\_for\_digital\_input\_timer\_on\_delay**(*io\_name*, *delay*=0.0)

Wait for a given delay and returns after the specified digital input signal reaches high.

#### Parameters

- **io\_name** (*str*) – The name or identifier of the digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

#### Returns

True if the digital input signal reaches high, False otherwise.

#### Return type

bool

#### Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_digital_input_timer_on_delay(1,10) #waits for 10sec
↳ and returns after digital input 1 reaches high
print(io_wait)
```

**wait\_for\_tool\_analog\_input**(*io\_name*, *expected\_value*)

Wait for a tool-specific analog input signal to match the expected value.

#### Parameters

- **io\_name** (*str*) – The name or identifier of the tool-specific analog input.
- **expected\_value** (*float/None*) – The expected value of the analog input. If None, the function waits for any change in the input value.

#### Returns

True if the tool-specific analog input matches the expected value, False otherwise.

#### Return type

bool

#### Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_analog_input(1,2.3v) #waits till tool analog
↳ input 1 is 2.3v
print(io_wait)
```

**wait\_for\_tool\_digital\_input**(*io\_name*, *expected\_value*, *wait\_time*=0.0)

Wait for a tool-specific digital input signal to match the expected value.

#### Parameters

- **io\_name** (*str*) – The name or identifier of the tool-specific digital input.
- **expected\_value** (*bool*) – The expected value of the digital input (True or False).
- **wait\_time** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

**Returns**

True if the tool-specific digital input matches the expected value, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input(1,True) #waits till tool digital_
↪input 1 is True
print(io_wait)
```

**wait\_for\_tool\_digital\_input\_timer\_off\_delay**(*io\_name*, *delay=0.0*)

Wait for a given delay and return after the specified tool digital input signal reaches low.

**Parameters**

- **io\_name** (*str*) – The name or identifier of the tool digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

**Returns**

True if the tool digital input signal reaches low, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input_timer_off_delay(1,10) #waits for_
↪10sec and returns after tool digital input 1 reaches low
print(io_wait)
```

**wait\_for\_tool\_digital\_input\_timer\_on\_delay**(*io\_name*, *delay=0.0*)

Wait for a given delay and return after the specified tool digital input signal reaches high.

**Parameters**

- **io\_name** (*str*) – The name or identifier of the tool digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

**Returns**

True if the tool digital input signal reaches high, False otherwise.

**Return type**

bool

**Sample Usage:**

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input_timer_on_delay(1,10) #waits for
↪ 10sec and returns after tool digital input 1 reaches high
print(io_wait)
```

**wait\_motion\_finished()**

This method allows to wait for a motion to complete. If there is no motion queued or all motions are already done this motion, checks once and skips waiting. Note that this only is effective if the motion is called with the non blocking option.

**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()

joint_command = {
    "speed": 10,
    "acceleration": 10,
    "jerk": 100,
    "blocking": False,
    "target_joint": [
        r.get_current_joint_angles(),
        [0.0] * 6
    ]
    "current_joint_angles": r.get_current_joint_angles()
}

r.move_joint(**joint_command) # Does not block until motion is completed
r.wait_motion_finished() # waits till motion is done
```

## EXAMPLES

## 5.1 Examples for Lara 5

### 1. Robot Object Initialization

```
from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)
print(r.current_tool)
print(r.connection)
print(r.version)
```

### 2. Move joint

```
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
            -1.8651270179353125,
            0.044771940725327274,
```

(continues on next page)

(continued from previous page)

```

        -1.173860821592129,
        0.10315646291502645
    ],
    [
        1.9180047887810003,
        -0.24855170101601043,
        -1.3680228668892351,
        0.12404421791100637,
        -1.1914147150222498,
        -0.13255713717112075
    ]
],
"current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)

```

### 3. Move linear

```

from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "blend_radius": 0.005,
    "target_pose": [
        [
            0.3287228886,
            -0.1903355329,
            0.4220780352,
            0.08535207028439847,
            -2.797181496822229,
            2.4713321627410485
        ],
        [
            0.2093363791501374,
            -0.31711250784165884,
            0.422149168855134,
            -3.0565555095672607,
            -0.3447442352771759,
            -1.1323236227035522
        ],
        [
            0.2090521916195534,
            -0.5246753336643587,
            0.4218773613553828,
            -3.0569007396698,
            -0.3448921740055084,
            -1.1323626041412354
        ],
        [
            0.3287228886,

```

(continues on next page)

(continued from previous page)

```

        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ],
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_linear(**linear_property)

```

## 4. Move circular

```

from neurapy.robot import Robot

r = Robot()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_circular(**circular_property)

```

## 5. Move composite

```

from neurapy.robot import Robot

```

(continues on next page)

(continued from previous page)

```

r = Robot()
composite_motion_property = {
    "speed": 0.432,
    "acceleration": 0.2,
    "current_joint_angles": r.robot_status('jointAngles'),
    "commands": [
        {
            "linear": {
                "blend_radius": 0.005,
                "targets": [
                    [
                        -0.000259845199876027,
                        -0.5211437049195536,
                        0.4429382717719519,
                        3.14123272895813,
                        -0.0007908568368293345,
                        -1.570908784866333
                    ],
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ]
                ]
            },
            {
                "circular": {
                    "targets": [
                        [
                            -0.16633498440272945,
                            -0.5201452059140722,
                            0.4427486025872017,
                            3.140937089920044,
                            -0.0005319403717294335,
                            -1.571555495262146
                        ],
                        [
                            -0.16540090985202305,
                            -0.3983552679378624,
                            0.44267608017426174,
                            3.1407113075256348,
                            -0.00036628879024647176,
                            -1.5714884996414185
                        ],
                        [
                            -0.33446498807559716,
                            -0.3989652352814891,

```

(continues on next page)

(continued from previous page)

```

0.4421152856242009,
3.1402060985565186,
0.00030071483342908323,
-1.572899580001831
]
}
}
}
r.move_composite(**composite_motion_property)

```

## 6. Move trajectory

```

from neurapy.robot import Robot

r = Robot()
trajectory_motion_property = {
    "current_joint_angles": r.robot_status('jointAngles'),
    "timestamps": [
        0.01, 0.02, 0.03
    ],
    "target_joint": [
        [0.0531381, 0.157485, -0.100272, 1.29569, -5.99211e-05, 0.700957, -
↪0.000371511],
        [-0.208897, 0.461728, -0.433937, 1.66485, -5.99211e-05, 0.700382, -
↪0.000383495],
        [0.0120801, 0.621298, 0.0149563, 1.67381, 5.99211e-05, 0.687403, -0.
↪000359527]
    ]
}
r.move_trajectory(**trajectory_motion_property)

```

## 7. Move recorded path

```

from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "is_motion": False,
    "file_location": "<path to file location>",
    "speed": 0.25,
    "use_recordings_filter": False,
    "current_joint_angles": r.robot_status('jointAngles'),
}
r.record_path(**recorded_path_property)

```

## 8. Power

```

from time import sleep
from neurapy.robot import Robot

r = Robot()

```

(continues on next page)

(continued from previous page)

```
r.power_on()
sleep(2)
r.power_off()
```

## 9. Motion status

```
from neurapy.robot import Robot

r = Robot()
print(r.motion_status())
```

## 10. Program status

```
from neurapy.robot import Robot

r = Robot()
print(r.program_status())
```

## 11. Warnings

```
from neurapy.robot import Robot

r = Robot()
print(r.get_warnings())
```

## 12. Errors

```
from neurapy.robot import Robot

r = Robot()
print(r.get_errors())
```

## 13. Get point

```
from neurapy.robot import Robot

r = Robot()
point = r.get_point("P1")
print(point)
```

## 14. IOs

```
import time
from neurapy.robot import Robot

r = Robot()
io_get = r.io("get", io_name = "DO_1")
io_set = r.io("set", io_name = "DO_2", target_value = True)
print(io_get, io_set)
```

## 15. Gripper

```
from time import sleep
from neurapy.robot import Robot

r = Robot()
```

(continues on next page)

(continued from previous page)

```
r.grasp()
sleep(2)
r.release()
```

## 16. Set tool

```
from neurapy.robot import Robot
r = Robot()
r.set_tool(tool_name="name given during tool creation in GUI")
```

## 17. Zero G

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_freedrive_mode()
time.sleep(1)
r.turn_off_freedrive_mode()
```

## 18. Forward/Inverse kinematics

```
from neurapy.robot import Robot
r = Robot()
target_pose = r.compute_forward_kinematics(target_angle = [0.2,0.2,0.2,0.2,
↪ 0.2,0.2])
target_angle = r.compute_inverse_kinematics(target_pose = [0.140448, -0.
↪ 134195, 1.197456, 3.1396, -0.589, -1.025],current_joint = [-1.55, -0.69,↪
↪ 0.06, 1.67, -0.02, -1.57, 0.11])
```

## 19. Override

```
import time
from neurapy.robot import Robot
r = Robot()
override_value = r.get_override()
print("override_value : " + str(override_value))
time.sleep(1)

override_value = r.set_override(0.4)
print("Setting Override to 0.4")
time.sleep(1)

override_value = r.get_override()
print("override_value : " + str(override_value))
```

## 20. Reset Error

```
import time
from neurapy.robot import Robot
r = Robot()
r.reset_error()
```

## 21. Get tools

```
import time
from neurapy.robot import Robot
r = Robot()
tools_data = r.get_tools()
```

## 22. Create tool

```
from neurapy.robot import Robot
r = Robot()
tool_data = { '_controlOA': False,
               '_controlOD': False,
               '_toolOA': False,
               '_toolOD': False,
               'autoM': 0,
               'autoMeasureX': 0,
               'autoMeasureY': 0,
               'autoMeasureZ': 0,
               'closeInput': 0,
               'cmdID': 16,
               'description': 'Tool Description',
               'force': 0,
               'gripper': '',
               'grippertype': 'Standard Gripper',
               'inertiaXX': 0,
               'inertiaXY': 0,
               'inertiaXZ': 0,
               'inertiaYY': 0,
               'inertiaYZ': 0,
               'inertiaZZ': 0,
               'name': 'NoTool',
               'offCOA': [0, 0, 0, 0, 0, 0, 0, 0, 0],
               'offCOD1': 0,
               'offCOD2': 0,
               'offTOA': [0, 0],
               'offTOD': 0,
               'offsetA': 0,
               'offsetB': 0,
               'offsetC': 0,
               'offsetX': 0,
               'offsetY': 0,
               'offsetZ': 0,
               'onCOA': [0, 0, 0, 0, 0, 0, 0, 0, 0],
               'onCOD1': 0,
               'onCOD2': 0,
               'onTOA': [0, 0],
               'onTOD': 0,
               'openInput': 0,
               'portID': '',
               'protocol': 0,
               'robot_type': 'Tool',
               'slaveID': 0,
               'speed': 0}
```

(continues on next page)

(continued from previous page)

```
tools_data = r.create_tool(tool_data)
```

## 23. Get encoder offsets

```
from neurapy.robot import Robot
r = Robot()
encoder_offsets = r.get_encoder_offsets()
```

## 24. Encoder to radian values

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.robot_status('loadSideEncValue')
joint_angles = r.encoder2rad(encoder_ticks)
```

## 25. Quaternion to roll pitch yaw

```
from neurapy.robot import Robot
r = Robot()
rpy = r.quaternion_to_rpy(0.85,0,0.52,0)
```

## 26. Roll pitch yaw to quaternion

```
from neurapy.robot import Robot
r = Robot()
quaternion = r.rpy_to_quaternion(0,1.1,0)
```

## 27. Get reference frame

```
from neurapy.robot import Robot
r = Robot()
frame = r.get_reference_frame("tool_frame")
print(frame)
```

## 28. Jogging

```
from neurapy.robot import Robot

robot = Robot()
"""
jog_velocity - velocity ranging from [-1,1] for all joints
jog_type - can be either cartesian or joint jogging
turn_on_jog changes the state from interal jogging (from GUI) to external_
↳ jogging
"""
robot.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
↳ 'Joint')

# command to set flag for jogging in external mode.
robot.jog(set_jogging_external_flag = 1)
i = 0

"""
```

(continues on next page)

(continued from previous page)

```

Requires minimum number of cycles in the loop for performing jogging.
Depends upon jogging velocity, override.
"""
while(i < 500):
    """
    command to set flag for jogging in external mode. This command has to
    be used each time
    external jog command has to be sent
    """
    robot.jog(set_jogging_external_flag = 1)
    i+=1

    """
    Change the state from external jog to internal jog (GUI) and sets all other
    external parameters to false
    """
    robot.turn_off_jog()

```

## 29. Get reference frame with offset

```

from neurapy.robot import Robot
r = Robot()
x_offset = 0.02 # in meters
y_offset = 0.1 # in meters
z_offset = 0.0 # in meters
frame = r.get_reference_frame_with_offset("world",[x_offset,y_offset,z_
    offset])
print(frame)

```

## 30. Get TCP pose

```

from neurapy.robot import Robot
r = Robot()
tcp_pose = r.get_tcp_pose()
print(tcp_pose)

```

## 31. Read safeio

```

from neurapy.robot import Robot
r = Robot()
safe_io = r.read_safeio(1)
print(safe_io) #True/False

```

## 32. Servo J

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

```

(continues on next page)

(continued from previous page)

```

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]
inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

## 33. Servo X

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig
import copy

r = Robot()

```

(continues on next page)

(continued from previous page)

```

#Switch to external servo mode
r.activate_servo_interface('position')

cart_pose_length = 7 #X,Y,Z,qw,qx,qy,qz

otg = Ruckig(cart_pose_length, 0.001) # control cycle
inp = InputParameter(cart_pose_length)
out = OutputParameter(cart_pose_length)

inp.current_position = r.get_current_cartesian_pose()
inp.current_velocity = [0.]*cart_pose_length
inp.current_acceleration = [0.]*cart_pose_length

target = copy.deepcopy(inp.current_position)
target[0] += 0.2 # Move 200mm in positive X direction
inp.target_position = target
inp.target_velocity = [0.]*cart_pose_length
inp.target_acceleration = [0.]*cart_pose_length

inp.max_velocity = [0.5]*cart_pose_length
inp.max_acceleration = [3]*cart_pose_length
inp.max_jerk = [10.]*cart_pose_length
res = Result.Working

servox_proportional_gain = 25

velocity = [0.] * 6 #Since ruckig does not provide rotational velocity if
↪quaternion is input, we can send 0 rotational feedforward velocity
acceleration = [0.] * 6 #Since ruckig does not provide rotational
↪acceleration if quaternion is input, we can send 0 rotational feedforward
↪acceleration

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position

        for i in range(0,3): # Updating target translation velocity and
↪accelerations
            velocity[i] = out.new_velocity[i]
            acceleration[i] = out.new_acceleration[i]

        error_code = r.servo_x(position, velocity, acceleration, servox_
↪proportional_gain)
        scaling_factor = r.get_servo_trajectory_scaling_factor()

```

(continues on next page)

(continued from previous page)

```

        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

## 34. MoveLinearOnline

```

from neurapy.robot import Robot
import time
import copy

r = Robot()

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_current_cartesian_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6 #Max velocities
acceleration = [2.]*6 #Max accelerations
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)

target = copy.deepcopy(r.get_current_cartesian_pose())
#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion
time.sleep(5)

#Stop MoveLinearOnline if robot is still in motion
r.stop_movelinear_online()

#Sleep for 1 sec to stop the robot motion
time.sleep(1)

print("Robot stopped")
r.deactivate_servo_interface()
r.stop()

```

## 5.2 Examples for Lara 8

### 1. Robot Object Initialization

```
from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)
print(r.current_tool)
print(r.connection)
print(r.version)
```

### 2. Move joint

```
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        previous_joint_angles,
        [
            0.7740276502469322,
            0.13599234975308788,
            1.2223307505101257,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.30333999999999633,
            -0.5334353004938217,
            1.4419907505101492,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.059259210493824405,
            0.08133676975309524,
            1.4419907505101492,
            -1.0084715804938613,
            -0.7008323497530976,
            -0.31901765024691225
        ]
    ]
}
```

(continues on next page)

(continued from previous page)

```
r.move_joint(**joint_property)
```

### 3. Move linear

```
from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# moving the robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            -0.0007546,
            0.2713685,
            1.2664022,
            -0.0007550,
            1.6046191,
            -0.0007501,
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,
    "target_pose": [
        [
            0.521504613338733,
            -0.0005120121635832758,
            0.4429621801014548,
            3.140838623046875,
            -0.0007978816283866763,
            3.1415627002716064
        ],
        [
            0.5216067833501538,
            -0.21429644443731097,
            0.44311804388931564,
            3.140573024749756,
            -0.0008633044781163335,
            3.1414897441864014
        ],
        [
            0.30336607796720333,
```

(continues on next page)

(continued from previous page)

```

        -0.2139397968890111,
        0.44282379715713066,
        3.140761613845825,
        -0.0011759602930396795,
        3.1410810947418213
    ],
    [
        0.3033097863631349,
        0.19367482201823044,
        0.44268805519921206,
        -3.1399381160736084,
        -0.001531908637844026,
        3.141218662261963
    ],
    [
        0.5376223865405412,
        0.1909303305550618,
        0.44330028363322316,
        -3.1380574703216553,
        -0.0019246124429628253,
        3.1399550437927246
    ],
    [
        0.521504613338733,
        -0.0005120121635832758,
        0.4429621801014548,
        3.140838623046875,
        -0.0007978816283866763,
        3.1415627002716064
    ],
],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_linear(**linear_property)
r.stop()

```

#### 4. Move circular

```

from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        0.3223498, 0.170158, 1.3894271, -0.0039065, 1.5862455, -0.0013291
    ],
}

```

(continues on next page)

(continued from previous page)

```

        "current_joint_angles": r.robot_status("jointAngles")
    }
    r.move_joint(**joint_property)
    time.sleep(1)
    circular_property = {
        "speed": 0.25,
        "acceleration": 0.1,
        "target_pose": [
            [
                0.3744609827431085,
                -0.3391784988266481,
                0.23276604279256016,
                3.14119553565979,
                -0.00017731254047248513,
                -0.48800110816955566
            ],
            [
                0.37116786741831503,
                -0.19686307684994242,
                0.23300456855796453,
                3.141423225402832,
                -0.00020668463548645377,
                -0.48725831508636475
            ],
            [
                0.5190337951593321,
                -0.1969996948428492,
                0.23267853691809767,
                3.1414194107055664,
                -0.00017726201622281224,
                -0.48750609159469604
            ]
        ],
        "current_joint_angles": r.robot_status("jointAngles")
    }
    r.move_circular(**circular_property)
    r.stop()

```

## 5. Move composite

```

from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [

```

(continues on next page)

(continued from previous page)

```

        [0.9855635563580816, 0.3476807153487805, 1.1687323223466928, -0.
        0007014516639916519, 1.6256822043262882, -0.000683475326633617]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Composite
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "current_joint_angles": r.robot_status('jointAngles'),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.30323030824121955,
                    0.4573874594177961,
                    0.44309265860894687,
                    3.140892505645752,
                    -0.0005030535394325852,
                    -2.155384063720703
                ],
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,
                    -0.0014394369209185243,
                    -3.0334575176239014
                ]
            ]
        }
    },
    {
        "circular": {
            "targets": [
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,
                    -0.0014394369209185243,
                    -3.0334575176239014
                ],
                [
                    0.4708356936660009,
                    0.280180550729363,

```

(continues on next page)

(continued from previous page)

```

0.4431937440044086,
3.1407277584075928,
-0.002278585685417056,
-2.601893424987793
],
[
0.30323030824121955,
0.4573874594177961,
0.44309265860894687,
3.140892505645752,
-0.0005030535394325852,
-2.155384063720703
]
]
}
]
}

r.move_composite(**composite_motion_property)
r.stop()

```

## 6. Move trajectory

```

from neurapy.robot import Robot

r = Robot()
trajectory_motion_property = {
    "current_joint_angles": r.robot_status('jointAngles'),
    "timestamps": [
        0.01, 0.02, 0.03
    ],
    "target_joint": [
        [0.21, 0.2, 1.34, 0, 1.61, 0],
        [0.22, 0.21, 1.35, 0, 1.61, 0],
        [0.23, 0.22, 1.36, 0, 1.61, 0]
    ]
}

r.move_trajectory(**trajectory_motion_property)
r.stop()

```

## 7. Move recorded path

```

from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "is_motion": False,
    "file_location": "<path to file location>",
    "speed": 0.25,
    "use_recordings_filter": False,

```

(continues on next page)

(continued from previous page)

```

        "current_joint_angles": r.robot_status('jointAngles'),
    }
    r.record_path(**recorded_path_property)

```

## 5.3 Examples for Lara 10

### 1. Robot Object Initialization

```

from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)
print(r.current_tool)
print(r.connection)
print(r.version)

```

### 2. Move joint

```

from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        previous_joint_angles,
        [
            0.7740276502469322,
            0.13599234975308788,
            1.2223307505101257,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.30333999999999633,
            -0.5334353004938217,
            1.4419907505101492,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.059259210493824405,
            0.08133676975309524,

```

(continues on next page)

(continued from previous page)

```

        1.4419907505101492,
        -1.0084715804938613,
        -0.7008323497530976,
        -0.31901765024691225
    ]
]
}
r.move_joint(**joint_property)

```

### 3. Move linear

```

from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# moving the robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            -0.0007546,
            0.2713685,
            1.2664022,
            -0.0007550,
            1.6046191,
            -0.0007501,
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,
    "target_pose": [
        [
            0.521504613338733,
            -0.0005120121635832758,
            0.4429621801014548,
            3.140838623046875,
            -0.0007978816283866763,
            3.1415627002716064
        ],
        [
            0.5216067833501538,
            -0.21429644443731097,

```

(continues on next page)

(continued from previous page)

```

        0.44311804388931564,
        3.140573024749756,
        -0.0008633044781163335,
        3.1414897441864014
    ],
    [
        0.30336607796720333,
        -0.2139397968890111,
        0.44282379715713066,
        3.140761613845825,
        -0.0011759602930396795,
        3.1410810947418213
    ],
    [
        0.3033097863631349,
        0.19367482201823044,
        0.44268805519921206,
        -3.1399381160736084,
        -0.001531908637844026,
        3.141218662261963
    ],
    [
        0.5376223865405412,
        0.19093033055550618,
        0.44330028363322316,
        -3.1380574703216553,
        -0.0019246124429628253,
        3.1399550437927246
    ],
    [
        0.521504613338733,
        -0.0005120121635832758,
        0.4429621801014548,
        3.140838623046875,
        -0.0007978816283866763,
        3.1415627002716064
    ],
    ],
    "current_joint_angles":r.robot_status("jointAngles")
}
r.move_linear(**linear_property)
r.stop()

```

#### 4. Move circular

```

from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# Move to initial position

```

(continues on next page)

(continued from previous page)

```

joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [0.3223498, 0.170158, 1.3894271, -0.0039065, 1.5862455, -0.0013291]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_circular(**circular_property)
r.stop()

```

#### 5. Move composite

```

from neurapy.robot import Robot

r = Robot()
r.set_mode("Automatic")
time.sleep(1)

```

(continues on next page)

(continued from previous page)

```

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [0.9855635563580816, 0.3476807153487805, 1.1687323223466928, -0.
00007014516639916519, 1.6256822043262882, -0.000683475326633617]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Composite
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "current_joint_angles": r.robot_status('jointAngles'),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.30323030824121955,
                    0.4573874594177961,
                    0.44309265860894687,
                    3.140892505645752,
                    -0.0005030535394325852,
                    -2.155384063720703
                ],
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,
                    -0.0014394369209185243,
                    -3.0334575176239014
                ]
            ]
        }
    },
    {
        "circular": {
            "targets": [
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,

```

(continues on next page)

(continued from previous page)

```

        3.1414873600006104,
        -0.0014394369209185243,
        -3.0334575176239014
    ],
    [
        0.4708356936660009,
        0.280180550729363,
        0.4431937440044086,
        3.1407277584075928,
        -0.002278585685417056,
        -2.601893424987793
    ],
    [
        0.30323030824121955,
        0.4573874594177961,
        0.44309265860894687,
        3.140892505645752,
        -0.0005030535394325852,
        -2.155384063720703
    ]
]
}
}

r.move_composite(**composite_motion_property)
r.stop()

```

## 6. Move trajectory

```

from neurapy.robot import Robot

r = Robot()
trajectory_motion_property = {
    "current_joint_angles": r.robot_status('jointAngles'),
    "timestamps": [
        0.01, 0.02, 0.03
    ],
    "target_joint": [
        [0.21, 0.2, 1.34, 0, 1.61, 0],
        [0.22, 0.21, 1.35, 0, 1.61, 0],
        [0.23, 0.22, 1.36, 0, 1.61, 0]
    ]
}

r.move_trajectory(**trajectory_motion_property)
r.stop()

```

## 7. Move recorded path

```

from neurapy.robot import Robot

```

(continues on next page)

(continued from previous page)

```

r = Robot()
recorded_path_property = {
    "is_motion": False,
    "file_location": "<path to file location>",
    "speed": 0.25,
    "use_recordings_filter": False,
    "current_joint_angles": r.robot_status('jointAngles'),
}
r.record_path(**recorded_path_property)

```

## 5.4 Examples for Maira 7

### 1. Move linear

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.set_mode("Automatic")
time.sleep(1)

# Moving the robot to Initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            1.406,
            -0.8005,
            0.07339,
            -0.7950,
            -0.00769,
            -1.51563,
            -0.65509
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)

#Linear motion
linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,
    "target_pose": [
        [
            -0.14602202042158488,

```

(continues on next page)

(continued from previous page)

```

        -1.2039712892654788,
        0.5542529402524848,
        -3.1250412464141846,
        -0.05224483087658882,
        -1.030046820640564
    ],
    [
        0.32102659790371435,
        -1.2039027313771722,
        0.5543345835272986,
        -3.1251614093780518,
        -0.05228979140520096,
        -1.030221700668335
    ],
    [
        0.3214140596556481,
        -0.8018749844104675,
        0.55438618778188,
        -3.1251306533813477,
        -0.052478402853012085,
        -1.0297577381134033
    ],
    [
        -0.2883707227945903,
        -0.8016352570186388,
        0.5542651616343833,
        -3.125182628631592,
        -0.052266936749219894,
        -1.0304018259048462
    ],
    [
        -0.14602202042158488,
        -1.2039712892654788,
        0.5542529402524848,
        -3.1250412464141846,
        -0.05224483087658882,
        -1.030046820640564
    ]
],
    "current_joint_angles":r.robot_status("jointAngles")
}
r.move_linear(**linear_property)
r.stop()

```

## 2. Move circular

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.set_mode("Automatic")

```

(continues on next page)

(continued from previous page)

```

time.sleep(1)

#move robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            2.687,
            -0.4671,
            -0.1182,
            -1.2874,
            0.11341,
            -1.3622,
            -1.23634
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Circular
circular_property = {
    "speed": 1.0,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.8827123230328554,
            -0.4975248049417001,
            0.5826623081385038,
            3.135085105895996,
            -0.06406070291996002,
            0.655619740486145
        ],
        [
            -0.03171114438156373,
            -1.0127658173803937,
            0.58266951895531,
            3.1350698471069336,
            -0.064027339220047,
            -0.43322843313217163
        ],
        [
            -0.7324668416327569,
            -0.7001335665738433,
            0.5826737224363503,
            3.1350419521331787,
            -0.06396733969449997,
            -1.209897518157959
        ]
    ]
}

```

(continues on next page)

(continued from previous page)

```

    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_circular(**circular_property)
r.stop()

```

### 3. Move composite

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.set_mode("Automatic")
time.sleep(1)
#Move robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_safety": True,
    "target_joint": [
        [
            2.687546699107577,
            -0.46715931807787425,
            -0.11823711193030452,
            -1.2874158466547219,
            0.11341945351835118,
            -1.3622644468222112,
            -1.2363431996854033
        ]
    ],
    "current_joint_angles": r.robot_status("jointAngles")
}
r.move_joint(**joint_property)
time.sleep(1)

#Move Composite
composite_motion_property = {
    "speed": 0.432,
    "acceleration": 0.1,
    "current_joint_angles": r.robot_status('jointAngles'),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.8827123230328554,
                    -0.4975248049417001,
                    0.5826623081385038,
                    3.135085105895996,
                    -0.06406070291996002,

```

(continues on next page)

(continued from previous page)

```

0.655619740486145
],
[
-0.7324668416327569,
-0.7001335665738433,
0.5826737224363503,
3.1350419521331787,
-0.06396733969449997,
-1.209897518157959
]
]
}
},
{
  "circular": {
    "targets": [
      [
        -0.7324668416327569,
        -0.7001335665738433,
        0.5826737224363503,
        3.1350419521331787,
        -0.06396733969449997,
        -1.209897518157959
      ],
      [
        -0.03171114438156373,
        -1.0127658173803937,
        0.58266951895531,
        3.1350698471069336,
        -0.064027339220047,
        -0.43322843313217163
      ],
      [
        0.8827123230328554,
        -0.4975248049417001,
        0.5826623081385038,
        3.135085105895996,
        -0.06406070291996002,
        0.655619740486145
      ]
    ]
  }
},
]
}
r.move_composite(**composite_motion_property)
r.stop()

```

## SAMPLE APPLICATIONS

## 6.1 Pick and place for Maira

1. Example with points defined in GUI

**Warning:**

Add pickup point Pi0 and place point Pl0 in GUI. Also add points Pi1 and Pl1 10 cm above pickup and place point respectively.

```
from neurapy.robot import Robot
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.5)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "Pl1"])
r.set_override(0.5)
r.move_linear(["Pl1", "Pl0"])
r.release()
time.sleep(1)
r.move_linear(["Pl0", "Pl1"])
```

(continues on next page)

(continued from previous page)

```
r.set_override(1)
r.move_linear(["Pl1", "Home"])
```

## 2. Example using move fuctions with points defined in Neurapy

```
from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, r.
    ↪get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos, r.
    ↪get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos, pick_pos])
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos, offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos, place_pos])
r.release()
time.sleep(1)
r.move_linear(target_pose = [place_pos, offset_place_pos])

#default position
```

(continues on next page)

(continued from previous page)

```

r.move_joint('Home')

r.power_off()

```

### 3. Example with points saved using Neurapy

```

from neurapy.robot import Robot
import numpy as np
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point here

"""
A function to create four points according to the pickup and placing point.
"""
def save_point(target_end_effector_pose, target, reference_joint_angles = r.
    ↪get_point('Home', representation='Joint')):
    for i in range(2):

        if target == "pick":
            name = "Pi" + str(i)
        else:
            name = "Pl" + str(i)

        r.create_point(name = name, target_end_effector_pose = target_end_
    ↪effector_pose)
        target_end_effector_pose[2] = target_end_effector_pose[2] + 0.1
    return True

save_point(pick_pos, "pick")
save_point(place_pos, "place")

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.5)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)

```

(continues on next page)

(continued from previous page)

```

r.move_linear(["P11","P11"])
r.set_override(0.5)
r.move_linear(["P11","P10"])
r.release()
time.sleep(1)
r.move_linear(["P10","P11"])
r.set_override(1)
r.move_linear(["P11","Home"])

```

#### 4. Example with controlling gripper action through IOs

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, r.
↳get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,r.
↳get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos,pick_pos])
r.wait(port_name="DI_2", expected_value=1.0, wait_for_signal=True, wait_
↳time=0.0) #waits till digital input 2 is True
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos,offset_pick_pos])

```

(continues on next page)

(continued from previous page)

```

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos,place_pos])
r.release()
time.sleep(1)
r.set_digital_output(1,True) # set digital output 1 to true after release
r.move_linear(target_pose = [place_pos,offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

## 5. Example using planner

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi]
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi]
offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1
offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')

plan_id_1 = r.plan_move_linear(target_pose =[home,offset_pick_pos,pick_pos],
    ↪store_id=1)
plan_id_2 = r.plan_move_linear(target_pose =[pick_pos,offset_pick_pos,
    ↪offset_place_pos,place_pos],store_id=2)
plan_id_3 = r.plan_move_linear(target_pose =[place_pos,offset_place_pos,
    ↪home],store_id=3)

execute_motion = r.executor([plan_id_1]) #To execute the planned id
r.grasp()
time.sleep(1)
execute_motion = r.executor([plan_id_2])
r.release()
time.sleep(1)
execute_motion = r.executor([plan_id_3])

```

## 6. Example with pick and place function in 4 different position

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

```

(continues on next page)

(continued from previous page)

```

r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

def planner(pick_pos, place_pos):
    offset_pick_pos = copy.deepcopy(pick_pos)
    offset_pick_pos[2] = offset_pick_pos[2] + 0.1
    offset_place_pos = copy.deepcopy(place_pos)
    offset_place_pos[2] = offset_place_pos[2] + 0.1
    plan_id = [0, 0, 0]
    plan_id[0] = r.plan_move_linear(target_pose = [home, offset_pick_pos, pick_pos])
    plan_id[1] = r.plan_move_linear(target_pose = [pick_pos, offset_pick_pos, offset_place_pos, place_pos])
    plan_id[2] = r.plan_move_linear(target_pose = [place_pos, offset_place_pos, home])
    return plan_id

def execute_pp(plan_id):
    execute_motion = r.executor([plan_id[0]]) #To execute the planned id
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    return True

for i in range(len(pick_pos)):
    plan_id = planner(pick_pos[i], place_pos[i])
    execute_pp(plan_id)

```

#### 7. Example with simultaneous planning and execution

```

import threading
from neurapy.robot import Robot

```

(continues on next page)

(continued from previous page)

```

import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()

def planner(pick_pos, place_pos):
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_pos,
←pick_pos[i]],current_joint_angles= home_joint, store_id=base_number + (3←
←* i + 1))
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
←pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
←joint, store_id=base_number + 1 + (3 * i + 1))
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i], offset_
←place_pos, home],current_joint_angles= home_joint, store_id=base_number_
←+2 + (3 * i + 1))

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_id}\n")

```

(continues on next page)

(continued from previous page)

```

↪033[0m", flush=True)

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if plan_ids:
                plan_id = plan_ids.pop(0)
            else:
                if planning_thread.is_alive():
                    continue
                else:
                    break
        print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪flush=True)
        execute_motion = r.executor([plan_id[0]]) # To execute the planned
↪id
        print("Executed plan 1", flush=True)
        r.grasp()
        time.sleep(1)
        execute_motion = r.executor([plan_id[1]])
        print("Executed plan 2", flush=True)
        r.release()
        time.sleep(1)
        execute_motion = r.executor([plan_id[2]])
        print("Executed plan 3", flush=True)

        idx = idx + 1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

# Create and start execution thread
execute_plan()

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

8. Example with reusing IDs for repeated motion.

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()

```

(continues on next page)

(continued from previous page)

```

r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()
planning_complete = threading.Event()

def planner(pick_pos, place_pos):
    global planning_complete
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_pos,
←pick_pos[i]],current_joint_angles= home_joint, store_id=base_number + (3
←* i + 1),reusable = True)
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
←pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
←joint, store_id=base_number + 1 + (3 * i + 1),reusable = True)
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i], offset_
←place_pos, home],current_joint_angles= home_joint, store_id=base_number_
←+2 + (3 * i + 1),reusable = True)

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_id}\
←\033[0m", flush=True)
            planning_complete.set()

def execute_plan():
    idx = 0

```

(continues on next page)

(continued from previous page)

```

while True:
    with plan_ids_lock:
        if idx<=3:
            if plan_ids:
                plan_id = plan_ids[idx]
            else:
                continue
        else:
            break

    print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↵flush=True)
    execute_motion = r.executor([plan_id[0]]) # To execute the planned
↵id
    print("Executed motion 1", flush=True)
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    print("Executed motion 2", flush=True)
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    print("Executed motion 3", flush=True)

    idx = idx +1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↵pos))
planning_thread.start()

iter = 5 # number of loops,specify it as required

# Create and start execution thread
for i in range(iter):
    execute_plan()
    print(f'\033[92mExecuted the plan for {i+1} iteration.\033[0m')

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

## 6.2 Pick and place for Lara

1. Example with points defined in GUI

### Warning:

Add pickup point *Pi0* and place point *Pi0* in GUI. Also add points *Pi1* and *P11* 10 cm above pickup and place point respectively.

```
from neurapy.robot import Robot
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.2)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "P11"])
r.set_override(0.2)
r.move_linear(["P11", "P10"])
r.release()
time.sleep(1)
r.move_linear(["P10", "P11"])
r.set_override(1)
r.move_linear(["P11", "Home"])
```

2. Example using move fuctions with points defined in Neurapy

```
from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
```

(continues on next page)

(continued from previous page)

```

r.power_on()

#Defining points
pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, r.
    ↪get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,r.
    ↪get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos,pick_pos])
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos,offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos,place_pos])
r.release()
time.sleep(1)
r.move_linear(target_pose = [place_pos,offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

### 3. Example with points saved using Neurapy

```

from neurapy.robot import Robot
import numpy as np
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)

```

(continues on next page)

(continued from previous page)

```

r.power_on()

pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point here

"""
A function to create four points according to the pickup and placing point.
"""
def save_point(target_end_effector_pose, target, reference_joint_angles = r.
    ↪get_point('Home', representation='Joint')):
    for i in range(2):

        if target == "pick":
            name = "Pi"+ str(i)
        else:
            name = "Pl" + str(i)
        r.create_point(name = name, target_end_effector_pose = target_end_
    ↪effector_pose)
        target_end_effector_pose[2] = target_end_effector_pose[2] + 0.1
    return True

save_point(pick_pos, "pick")
save_point(place_pos, "place")

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.2)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "Pl1"])
r.set_override(0.2)
r.move_linear(["Pl1", "Pl0"])
r.release()
time.sleep(1)
r.move_linear(["Pl0", "Pl1"])
r.set_override(1)
r.move_linear(["Pl1", "Home"])

```

#### 4. Example with controlling gripper action through IOs

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

```

(continues on next page)

(continued from previous page)

```

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, r.
↳get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos, r.
↳get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos, pick_pos])
r.wait(port_name="DI_2", expected_value=1.0, wait_for_signal=True, wait_
↳time=0.0) #waits till digital input 2 is True
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos, offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos, place_pos])
r.release()
time.sleep(1)
r.set_digital_output(1, True) # set digital output 1 to true after release
r.move_linear(target_pose = [place_pos, offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

## 5. Example using planner

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi]
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi]
offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1
offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')

plan_id_1 = r.plan_move_linear(target_pose =[home,offset_pick_pos,pick_pos],
    ↪store_id=1)
plan_id_2 = r.plan_move_linear(target_pose =[pick_pos,offset_pick_pos,
    ↪offset_place_pos,place_pos],store_id=2)
plan_id_3 = r.plan_move_linear(target_pose =[place_pos,offset_place_pos,
    ↪home],store_id=3)

execute_motion = r.executor([plan_id_1]) #To execute the planned id
r.grasp()
time.sleep(1)
execute_motion = r.executor([plan_id_2])
r.release()
time.sleep(1)
execute_motion = r.executor([plan_id_3])

```

#### 6. Example with pick and place function in 4 different position

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],
    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],
    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

```

(continues on next page)

(continued from previous page)

```

]

def planner(pick_pos, place_pos):
    offset_pick_pos = copy.deepcopy(pick_pos)
    offset_pick_pos[2] = offset_pick_pos[2] + 0.1
    offset_place_pos = copy.deepcopy(place_pos)
    offset_place_pos[2] = offset_place_pos[2] + 0.1
    plan_id= [0,0,0]
    plan_id[0] = r.plan_move_linear(target_pose =[home,offset_pick_pos,pick_
↵pos])
    plan_id[1] = r.plan_move_linear(target_pose =[pick_pos,offset_pick_pos,
↵offset_place_pos,place_pos])
    plan_id[2] =r.plan_move_linear(target_pose =[place_pos,offset_place_pos,
↵home])
    return plan_id

def execute_pp(plan_id):
    execute_motion = r.executor([plan_id[0]]) #To execute the planned id
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    return True

for i in range(len(pick_pos)):
    plan_id = planner(pick_pos[i],place_pos[i])
    execute_pp(plan_id)

```

## 7. Example with simultaneous planning and execution

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],

```

(continues on next page)

(continued from previous page)

```

    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],
    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()

def planner(pick_pos, place_pos):
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_pos,
←pick_pos[i]],current_joint_angles= home_joint, store_id=base_number + (3
←* i + 1))
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
←pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
←joint, store_id=base_number + 1 + (3 * i + 1))
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i], offset_
←place_pos, home],current_joint_angles= home_joint, store_id=base_number
←+2 + (3 * i + 1))

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_id}\
←\033[0m", flush=True)

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if plan_ids:
                plan_id = plan_ids.pop(0)
            else:
                if planning_thread.is_alive():
                    continue
                else:
                    break
            print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
←flush=True)
            execute_motion = r.executor([plan_id[0]]) # To execute the planned

```

(continues on next page)

(continued from previous page)

```

↪id
    print("Executed plan 1", flush=True)
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    print("Executed plan 2", flush=True)
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    print("Executed plan 3", flush=True)

    idx = idx + 1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

# Create and start execution thread
execute_plan()

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

8. Example with reusing IDs for repeated motion.

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],
    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],

```

(continues on next page)

(continued from previous page)

```

    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()
planning_complete = threading.Event()

def planner(pick_pos, place_pos):
    global planning_complete
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_pos,
↪ pick_pos[i]], current_joint_angles= home_joint, store_id=base_number + (3
↪ * i + 1), reusable = True)
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
↪ pick_pos, offset_place_pos, place_pos[i]], current_joint_angles= home_
↪ joint, store_id=base_number + 1 + (3 * i + 1), reusable = True)
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i], offset_
↪ place_pos, home], current_joint_angles= home_joint, store_id=base_number
↪ + 2 + (3 * i + 1), reusable = True)

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_id}\
↪ \033[0m", flush=True)
        planning_complete.set()

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if idx <= 3:
                if plan_ids:
                    plan_id = plan_ids[idx]
                else:
                    continue
            else:
                break

        print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪ flush=True)
        execute_motion = r.executor([plan_id[0]]) # To execute the planned
↪ id
        print("Executed motion 1", flush=True)
        r.grasp()

```

(continues on next page)

(continued from previous page)

```

        time.sleep(1)
        execute_motion = r.executor([plan_id[1]])
        print("Executed motion 2", flush=True)
        r.release()
        time.sleep(1)
        execute_motion = r.executor([plan_id[2]])
        print("Executed motion 3", flush=True)

        idx = idx + 1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

iter = 5 # number of loops, specify it as required

# Create and start execution thread
for i in range(iter):
    execute_plan()
    print(f'\033[92mExecuted the plan for {i+1} iteration.\033[0m')

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

## 6.3 External Source Applications Examples

### 1. Creating external source object

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↪process/Robotiq2f85.json", drymode = False)

```

### 2. Connect to external source

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↪process/Robotiq2f85.json", drymode = False)
# When connectin the "init" sequence will also be executed
robotiq_gripper.connect()

```

### 3. Disconnect external source

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
robotiq_gripper.connect()
robotiq_gripper.disconnect()
```

4. Check if the external source is connected

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
robotiq_gripper.connect()
# Check if the external source is connected.
if(robotiq_gripper.is_connected()):
    print("Robotiq2F85 is connected")
else:
    print("Robotiq2F85 is not connected")
```

5. Execute a process (function) using external source

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
robotiq_gripper.connect()
# Execute the grip process (function) with parameters. This function_
↳executes the grip process configuration.
robotiq_gripper.execute_process(process_name="grip", parameters = {"width
↳":65535,"force":255,"speed":255})
```

6. Reset error in an external source.

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
robotiq_gripper.connect()
# Reset the error in the external source. This function executes the "reset_
↳error" process configuration defined in the json file.
robotiq_gripper.execute_error_reset_process()
```

7. Execute stop function set for the external source

```

from neurapy.robot import Robot
import time
import time
r = Robot()
# Create the external source application object for the welding source
# dry_mode set true will skip executing the process_on function which is
↳used to send the arc on signal to the welding source.
welding_source = r.create_external_source_app(file_path="/home/hrg/data/
↳process/iROB401.json", drymode = True)
# Connect to the device
welding_source.connect()
# set the process data in the power source.
welding_source.execute_process(process_name="job", parameters = {"job":1})
time.sleep(1)
#Start arc on (Signal not sent as dry_mode is set to True.)
welding_source.process_on()
time.sleep(2)
#Turn off arc
welding_source.process_off()

```

8. Execute start function set for the external source.

```

from neurapy.robot import Robot
import time
r = Robot()
# Create the external source application object for the welding source
welding_source = r.create_external_source_app(file_path="/home/hrg/data/
↳process/iROB401.json", drymode = True)
# Connect to the device
welding_source.connect()
# set the process data in the power source.
welding_source.execute_process(process_name="job", parameters = {"job":1})
time.sleep(1)
#Start arc on, (Signal not sent as dry_mode is set to True)
welding_source.process_on()

```

9. Stop all connected external sources

```

from neurapy.robot import Robot
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
# Connect to the device
robotiq_gripper.connect()
#This Function executes the "stop" process for all the external sources
↳connected .
#It is defined in the process access section of the json file.
robotiq_gripper.process_off_all()

```

10. Set dry mode for an external source

```

from neurapy.robot import Robot

```

(continues on next page)

(continued from previous page)

```

import time
r = Robot()
# Create the external source application object for the welding source
welding_source = r.create_external_source_app(file_path="/home/hrg/data/
↳process/iROB401.json", drymode = True)
# Connect to the device
welding_source.connect()
# Set the welding source in dry_mode state
welding_source.set_dry_mode(dry_mode=True)
# Set the process data in the power source.Commands will be send to the
↳welding source.
welding_source.execute_process(process_name="job", parameters = {"job":1})
time.sleep(1)
#Start arc on, (Signal not sent as dry_mode is set to True)
welding_source.process_on()

```

#### 11. ModbusRTU Gripper example for external source

```

from neurapy.robot import Robot
import time
r = Robot()
# Create the external source application object for the Robotiq Gripper
robotiq_gripper = r.create_external_source_app(file_path="/home/hrg/data/
↳process/Robotiq2f85.json", drymode = False)
# Connect to the device
robotiq_gripper.connect()
if robotiq_gripper.is_connected():
    # gripper open
    robotiq_gripper.execute_process(process_name="grip", parameters = {
↳"width":0,"force":255,"speed":255})
    time.sleep(1)
    # gripper close
    robotiq_gripper.execute_process(process_name="grip", parameters = {
↳"width":65535,"force":255,"speed":255})

```

#### 12. ModbusTCP lift exmaple for external source

```

from neurapy.robot import Robot
import time
r = Robot()
# Create the external source application for the Linear lift external axis
↳using Linak.
lift_obj = r.create_external_source_app(file_path="/home/hrg/data/process/
↳LC3_IC_Lift.json", drymode = False)
# Connect to the device
lift_obj.connect()
# linear lift moves up
lift_obj.execute_process(process_name="GoToPosition", parameters = {"current
↳":251,"speed":"80","position":5000})
time.sleep(5)
# linear lift moves down
lift_obj.execute_process(process_name="GoToPosition", parameters = {"current
↳":251,"speed":"80","position":1000})

```

## 13. EthernetIP welding source example for external source

```
from neurapy.robot import Robot
import time
r = Robot()
# Create the external source application object for the welding source
welding_source = r.create_external_source_app(file_path="/home/hrg/data/
↪process/iROB401.json", drymode = True)
# Connect to the device
welding_source.connect()
# set the process data in the power source.
welding_source.execute_process(process_name="job", parameters = {"job":1})
time.sleep(1)
# Start arc on , (Signal not sent as dry_mode is set to True)
welding_source.process_on()
time.sleep(2)
#Turn off arc
welding_source.process_off()
```



## CHAPTER SEVEN

### TCP-FLANGE CONNECTION PIN OUTS FOR LARA



#### 6.3.4.1. TCP Flange connection pin-out

External flange connections pin-out (12 pole M12 connector/ IEC 61076-2-101)

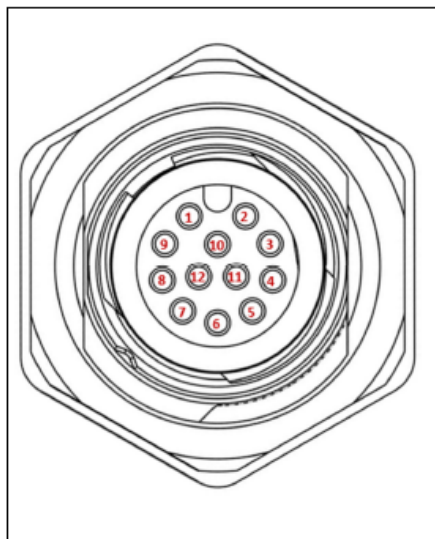


Fig. 25: IO ports, pin assignment of M12 connector (IEC 61076-2-101)

Pin-out of M12 connector for external connection is defined as follows:

| Pin | Line          | Cable color             | Definition                        |
|-----|---------------|-------------------------|-----------------------------------|
| 1   | Input_0       | Brown                   | Digital input 0                   |
| 2   | Input_1       | Blue                    | Digital input 1                   |
| 3   | Input_2       | White                   | Digital input 2                   |
| 4   | Output_0      | Green                   | Digital output 0                  |
| 5   | Output_1      | Powder                  | Digital output 1                  |
| 6   | Output_2      | Yellow                  | Digital output 2                  |
| 7   | RS485_A (TX+) | Black                   | RS484+ communication (Modbus RTU) |
| 8   | RS485_B (TX-) | Gray                    | RS484- communication (Modbus RTU) |
| 9   | AI 0          | Red                     | Analog input 0                    |
| 10  | AI 1          | Purple                  | Analog input 1                    |
| 11  | 24 V          | Ash powder (Grey/ Pink) | Power supply (24V)                |
| 12  | GND           | Red / Blue              | Common Ground (0V)                |

Table 12: External wiring for M12 TCP Connector

## TCP-FLANGE CONNECTION PIN OUTS FOR MAIRA



### External flange connections

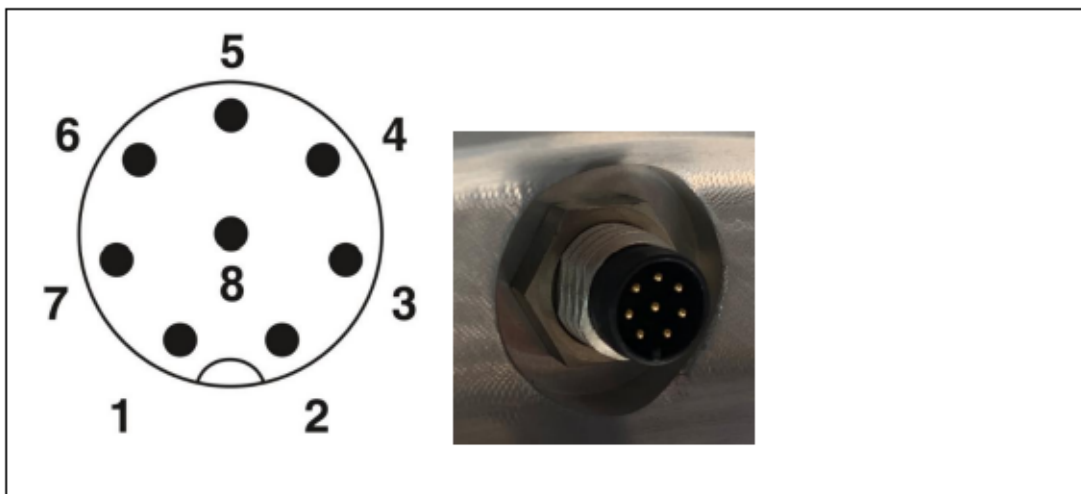


Fig. 29: M8 connectors, pin assignment (SACC-DSI-M8MS-8CON-M8/0,5)

IO external flange connections are defined as follows:

| Pin           | Line color | Definition | Notes    |
|---------------|------------|------------|----------|
| <b>Port1</b>  |            |            |          |
| 1             | White      | TX+        |          |
| 2             | Brown      | RX-        |          |
| 3             | Green      | TX-        |          |
| 4             | Yellow     | RX+        |          |
| 5             | Gray       | +24V       |          |
| 6             | Pink       |            | Not used |
| 7             | Blue       |            | Not used |
| 8             | Red        | GND        |          |
| <b>Port 2</b> |            |            |          |
| 1             | White      | DI_0       |          |
| 2             | Brown      | DI_1       |          |
| 3             | Green      | DO_0       |          |
| 4             | Yellow     | DO_1       |          |
| 5             | Gray       | +24V       |          |
| 6             | Pink       | AI_0       |          |
| 7             | Blue       | AI_1       |          |
| 8             | Red        | GND        |          |
| <b>Port 3</b> |            |            |          |
| 1             | White      | RS485_A    |          |
| 2             | Brown      | RS485_B    |          |
| 3             | Green      |            | Not used |
| 4             | Yellow     |            | Not used |
| 5             | Gray       | +24V       |          |
| 6             | Pink       |            | Not used |
| 7             | Blue       |            | Not used |
| 8             | Red        | GND        |          |

## PYTHON MODULE INDEX

### n

`neurapy.robot`, [40](#)

## A

activate\_ethercat\_interface() (neurapy.robot.Robot method), 41  
 activate\_ethernetIP\_interface() (neurapy.robot.Robot method), 41  
 activate\_modbus\_interface() (neurapy.robot.Robot method), 41  
 activate\_opcua\_interface() (neurapy.robot.Robot method), 41  
 activate\_profinet\_interface() (neurapy.robot.Robot method), 42  
 activate\_ros\_interface() (neurapy.robot.Robot method), 42  
 activate\_servo\_interface() (neurapy.robot.Robot method), 42  
 associate\_pose\_to\_new\_parent\_frame() (neurapy.robot.Robot method), 43

## C

compute\_forward\_kinematics() (neurapy.robot.Robot method), 43  
 compute\_inverse\_kinematics() (neurapy.robot.Robot method), 43  
 convert\_euler\_to\_quaternion\_pose() (neurapy.robot.Robot method), 44  
 convert\_quaternion\_to\_euler\_pose() (neurapy.robot.Robot method), 45  
 create\_external\_source\_app() (neurapy.robot.Robot method), 45  
 create\_point() (neurapy.robot.Robot method), 46  
 create\_tool() (neurapy.robot.Robot method), 46

## D

deactivate\_ethercat\_interface() (neurapy.robot.Robot method), 49  
 deactivate\_ethernetIP\_interface() (neurapy.robot.Robot method), 49  
 deactivate\_modbus\_interface() (neurapy.robot.Robot method), 49  
 deactivate\_opcua\_interface() (neurapy.robot.Robot method), 50

deactivate\_profinet\_interface() (neurapy.robot.Robot method), 50  
 deactivate\_ros\_interface() (neurapy.robot.Robot method), 50  
 deactivate\_servo\_interface() (neurapy.robot.Robot method), 50  
 delete\_pose\_in\_DB() (neurapy.robot.Robot method), 51  
 disable\_collision\_detection() (neurapy.robot.Robot method), 51  
 disable\_friction\_mode() (neurapy.robot.Robot method), 51  
 disable\_reflex() (neurapy.robot.Robot method), 52

## E

enable\_collision\_detection() (neurapy.robot.Robot method), 52  
 enable\_friction\_mode() (neurapy.robot.Robot method), 52  
 enable\_reflex() (neurapy.robot.Robot method), 53  
 encoder2rad() (neurapy.robot.Robot method), 53  
 execute\_closest\_orthogonal\_tcp\_align() (neurapy.robot.Robot method), 53  
 execute\_exact\_tcp\_align() (neurapy.robot.Robot method), 54  
 execute\_program() (neurapy.robot.Robot method), 54  
 execute\_z\_axis\_tcp\_align() (neurapy.robot.Robot method), 55  
 executor() (neurapy.robot.Robot method), 55

## G

get\_analog\_input() (neurapy.robot.Robot method), 56  
 get\_analog\_output() (neurapy.robot.Robot method), 56  
 get\_current\_joint\_angles() (neurapy.robot.Robot method), 56  
 get\_current\_joint\_angles\_with\_timestamp() (neurapy.robot.Robot method), 57  
 get\_current\_joint\_torques() (neurapy.robot.Robot method), 57

|                                                                                                         |                                                                                                          |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>get_current_joint_torques_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 57    | <code>get_motor_side_encoder_values_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 66 |
| <code>get_current_joint_velocities()</code><br>( <i>neurapy.robot.Robot method</i> ), 57                | <code>get_override()</code> ( <i>neurapy.robot.Robot method</i> ), 66                                    |
| <code>get_current_joint_velocities_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 58 | <code>get_point()</code> ( <i>neurapy.robot.Robot method</i> ), 66                                       |
| <code>get_current_load_side_encoder_values()</code> ( <i>neurapy.robot.Robot method</i> ), 58           | <code>get_point_names()</code> ( <i>neurapy.robot.Robot method</i> ), 67                                 |
| <code>get_current_motor_side_encoder_values()</code> ( <i>neurapy.robot.Robot method</i> ), 58          | <code>get_pose_id_from_name()</code> ( <i>neurapy.robot.Robot method</i> ), 67                           |
| <code>get_current_tool_cogs()</code> ( <i>neurapy.robot.Robot method</i> ), 58                          | <code>get_pose_in_parent_frame_from_DB()</code> ( <i>neurapy.robot.Robot method</i> ), 67                |
| <code>get_current_tool_inertias()</code> ( <i>neurapy.robot.Robot method</i> ), 59                      | <code>get_pose_in_reference_frame_from_DB()</code> ( <i>neurapy.robot.Robot method</i> ), 67             |
| <code>get_current_tool_mass()</code> ( <i>neurapy.robot.Robot method</i> ), 59                          | <code>get_pose_in_world_frame_from_DB()</code> ( <i>neurapy.robot.Robot method</i> ), 68                 |
| <code>get_current_tool_properties()</code> ( <i>neurapy.robot.Robot method</i> ), 59                    | <code>get_program_names()</code> ( <i>neurapy.robot.Robot method</i> ), 68                               |
| <code>get_current_tool_rpy_offsets()</code> ( <i>neurapy.robot.Robot method</i> ), 60                   | <code>get_reference_frame()</code> ( <i>neurapy.robot.Robot method</i> ), 68                             |
| <code>get_current_tool_translation_offsets()</code> ( <i>neurapy.robot.Robot method</i> ), 60           | <code>get_reference_frame_with_offset()</code> ( <i>neurapy.robot.Robot method</i> ), 69                 |
| <code>get_diagnostics()</code> ( <i>neurapy.robot.Robot method</i> ), 60                                | <code>get_register_bytes()</code> ( <i>neurapy.robot.Robot method</i> ), 69                              |
| <code>get_digital_input()</code> ( <i>neurapy.robot.Robot method</i> ), 60                              | <code>get_register_values()</code> ( <i>neurapy.robot.Robot method</i> ), 70                             |
| <code>get_digital_output()</code> ( <i>neurapy.robot.Robot method</i> ), 61                             | <code>get_safety()</code> ( <i>neurapy.robot.Robot method</i> ), 70                                      |
| <code>get_doc()</code> ( <i>neurapy.robot.Robot method</i> ), 61                                        | <code>get_selected_fts_data()</code> ( <i>neurapy.robot.Robot method</i> ), 70                           |
| <code>get_encoder_offsets()</code> ( <i>neurapy.robot.Robot method</i> ), 61                            | <code>get_selected_tool_data()</code> ( <i>neurapy.robot.Robot method</i> ), 71                          |
| <code>get_errors()</code> ( <i>neurapy.robot.Robot method</i> ), 62                                     | <code>get_selected_tool_name()</code> ( <i>neurapy.robot.Robot method</i> ), 71                          |
| <code>get_ext_digital_input()</code> ( <i>neurapy.robot.Robot method</i> ), 62                          | <code>get_servo_trajectory_scaling_factor()</code> ( <i>neurapy.robot.Robot method</i> ), 71             |
| <code>get_ext_digital_output()</code> ( <i>neurapy.robot.Robot method</i> ), 62                         | <code>get_tcp_force()</code> ( <i>neurapy.robot.Robot method</i> ), 72                                   |
| <code>get_flange_pose()</code> ( <i>neurapy.robot.Robot method</i> ), 63                                | <code>get_tcp_pose()</code> ( <i>neurapy.robot.Robot method</i> ), 73                                    |
| <code>get_flange_pose_quaternion()</code> ( <i>neurapy.robot.Robot method</i> ), 63                     | <code>get_tcp_pose_quaternion()</code> ( <i>neurapy.robot.Robot method</i> ), 73                         |
| <code>get_flange_pose_quaternion_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 63   | <code>get_tcp_pose_quaternion_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 73       |
| <code>get_flange_pose_with_timestamp()</code> ( <i>neurapy.robot.Robot method</i> ), 63                 | <code>get_tcp_pose_with_timestamp()</code> ( <i>neurapy.robot.Robot method</i> ), 73                     |
| <code>get_io_configuration()</code> ( <i>neurapy.robot.Robot method</i> ), 64                           | <code>get_tcp_speed()</code> ( <i>neurapy.robot.Robot method</i> ), 74                                   |
| <code>get_io_status()</code> ( <i>neurapy.robot.Robot method</i> ), 64                                  | <code>get_tcp_torque()</code> ( <i>neurapy.robot.Robot method</i> ), 74                                  |
| <code>get_joint_acceleration()</code> ( <i>neurapy.robot.Robot method</i> ), 64                         | <code>get_tcp_wrench()</code> ( <i>neurapy.robot.Robot method</i> ), 74                                  |
| <code>get_joint_speed()</code> ( <i>neurapy.robot.Robot method</i> ), 64                                | <code>get_tool_analog_input()</code> ( <i>neurapy.robot.Robot method</i> ), 74                           |
| <code>get_linear_acceleration()</code> ( <i>neurapy.robot.Robot method</i> ), 65                        | <code>get_tool_digital_input()</code> ( <i>neurapy.robot.Robot method</i> ), 75                          |
| <code>get_linear_speed()</code> ( <i>neurapy.robot.Robot method</i> ), 65                               | <code>get_tool_digital_output()</code> ( <i>neurapy.robot.Robot method</i> ), 75                         |
| <code>get_load_side_encoder_values_with_timestamp()</code><br>( <i>neurapy.robot.Robot method</i> ), 65 | <code>get_tools()</code> ( <i>neurapy.robot.Robot method</i> ), 75                                       |
|                                                                                                         | <code>get_warnings()</code> ( <i>neurapy.robot.Robot method</i> ), 76                                    |
|                                                                                                         | <code>grasp()</code> ( <i>neurapy.robot.Robot method</i> ), 76                                           |
|                                                                                                         | <code>gripper()</code> ( <i>neurapy.robot.Robot method</i> ), 76                                         |

## I

[init\\_program\(\)](#) (*neurapy.robot.Robot method*), 76  
[io\(\)](#) (*neurapy.robot.Robot method*), 77  
[is\\_collision\\_enabled\(\)](#) (*neurapy.robot.Robot method*), 77  
[is\\_free\\_drive\\_mode\\_enabled\(\)](#) (*neurapy.robot.Robot method*), 78  
[is\\_reflex\\_enabled\(\)](#) (*neurapy.robot.Robot method*), 78  
[is\\_robot\\_in\\_automatic\\_mode\(\)](#) (*neurapy.robot.Robot method*), 78  
[is\\_robot\\_in\\_collision\(\)](#) (*neurapy.robot.Robot method*), 78  
[is\\_robot\\_in\\_semi\\_automatic\\_mode\(\)](#) (*neurapy.robot.Robot method*), 79  
[is\\_robot\\_in\\_simulation\(\)](#) (*neurapy.robot.Robot method*), 79  
[is\\_robot\\_in\\_teach\\_mode\(\)](#) (*neurapy.robot.Robot method*), 79

## J

[jog\(\)](#) (*neurapy.robot.Robot method*), 80

## L

[list\\_methods\(\)](#) (*neurapy.robot.Robot method*), 80  
[list\\_recorded\\_path\(\)](#) (*neurapy.robot.Robot method*), 81  
[lock\\_cartesian\\_axes\(\)](#) (*neurapy.robot.Robot method*), 81  
[lock\\_joint\\_axes\(\)](#) (*neurapy.robot.Robot method*), 81

## M

[module](#)  
     [neurapy.robot](#), 40  
[motion\\_status\(\)](#) (*neurapy.robot.Robot method*), 82  
[move\\_circular\(\)](#) (*neurapy.robot.Robot method*), 82  
[move\\_composite\(\)](#) (*neurapy.robot.Robot method*), 87  
[move\\_joint\(\)](#) (*neurapy.robot.Robot method*), 94  
[move\\_joint\\_relative\(\)](#) (*neurapy.robot.Robot method*), 97  
[move\\_linear\(\)](#) (*neurapy.robot.Robot method*), 102  
[move\\_linear\\_relative\(\)](#) (*neurapy.robot.Robot method*), 109  
[move\\_pose\\_to\\_new\\_parent\\_frame\(\)](#) (*neurapy.robot.Robot method*), 113  
[move\\_recorded\\_path\(\)](#) (*neurapy.robot.Robot method*), 114  
[movelinear\\_online\(\)](#) (*neurapy.robot.Robot method*), 115

## N

[neurapy.robot](#)  
     [module](#), 40

[notify\\_error\(\)](#) (*neurapy.robot.Robot method*), 116  
[notify\\_info\(\)](#) (*neurapy.robot.Robot method*), 116  
[notify\\_warning\(\)](#) (*neurapy.robot.Robot method*), 116

## P

[pause\(\)](#) (*neurapy.robot.Robot method*), 117  
[plan\\_move\\_circular\(\)](#) (*neurapy.robot.Robot method*), 117  
[plan\\_move\\_composite\(\)](#) (*neurapy.robot.Robot method*), 121  
[plan\\_move\\_joint\(\)](#) (*neurapy.robot.Robot method*), 129  
[plan\\_move\\_linear\(\)](#) (*neurapy.robot.Robot method*), 132  
[plan\\_move\\_recorded\\_path\(\)](#) (*neurapy.robot.Robot method*), 138  
[plan\\_realtime\\_wait\(\)](#) (*neurapy.robot.Robot method*), 139  
[power\\_off\(\)](#) (*neurapy.robot.Robot method*), 140  
[power\\_on\(\)](#) (*neurapy.robot.Robot method*), 140  
[program\\_status\(\)](#) (*neurapy.robot.Robot method*), 140

## Q

[quaternion\\_to\\_rpy\(\)](#) (*neurapy.robot.Robot method*), 141

## R

[rad2encoder\(\)](#) (*neurapy.robot.Robot method*), 141  
[read\\_safeio\(\)](#) (*neurapy.robot.Robot method*), 142  
[realtime\\_wait\(\)](#) (*neurapy.robot.Robot method*), 142  
[release\(\)](#) (*neurapy.robot.Robot method*), 142  
[remove\\_load\(\)](#) (*neurapy.robot.Robot method*), 143  
[replicate\\_list\\_poses\\_in\\_new\\_parent\\_frame\(\)](#) (*neurapy.robot.Robot method*), 143  
[replicate\\_pose\\_in\\_new\\_parent\\_frame\(\)](#) (*neurapy.robot.Robot method*), 143  
[replicate\\_pose\\_in\\_new\\_parent\\_frame\\_and\\_delete\\_original\(\)](#) (*neurapy.robot.Robot method*), 143  
[reset\\_axes\\_lock\(\)](#) (*neurapy.robot.Robot method*), 144  
[reset\\_collision\(\)](#) (*neurapy.robot.Robot method*), 144  
[reset\\_control\(\)](#) (*neurapy.robot.Robot method*), 144  
[reset\\_errors\(\)](#) (*neurapy.robot.Robot method*), 145  
[reset\\_led\\_mode\(\)](#) (*neurapy.robot.Robot method*), 145  
[reset\\_warnings\(\)](#) (*neurapy.robot.Robot method*), 145  
[Robot](#) (*class in neurapy.robot*), 40  
[rpy\\_to\\_quaternion\(\)](#) (*neurapy.robot.Robot method*), 146

## S

[save\\_new\\_pose\\_in\\_DB\(\)](#) (*neurapy.robot.Robot method*), 146

save\_or\_update\_pose\_in\_DB() (*neurapy.robot.Robot method*), 146  
 servo\_j() (*neurapy.robot.Robot method*), 147  
 servo\_x() (*neurapy.robot.Robot method*), 148  
 set\_analog\_output() (*neurapy.robot.Robot method*), 150  
 set\_current\_position\_as\_zero() (*neurapy.robot.Robot method*), 150  
 set\_dblout\_register() (*neurapy.robot.Robot method*), 151  
 set\_digital\_output() (*neurapy.robot.Robot method*), 151  
 set\_encoder\_offsets() (*neurapy.robot.Robot method*), 152  
 set\_ext\_digital\_output() (*neurapy.robot.Robot method*), 152  
 set\_intout\_register() (*neurapy.robot.Robot method*), 153  
 set\_joint\_acceleration() (*neurapy.robot.Robot method*), 153  
 set\_joint\_speed() (*neurapy.robot.Robot method*), 154  
 set\_led\_mode() (*neurapy.robot.Robot method*), 154  
 set\_linear\_acceleration() (*neurapy.robot.Robot method*), 155  
 set\_linear\_speed() (*neurapy.robot.Robot method*), 155  
 set\_load() (*neurapy.robot.Robot method*), 155  
 set\_opcua\_msg() (*neurapy.robot.Robot method*), 156  
 set\_override() (*neurapy.robot.Robot method*), 156  
 set\_safety() (*neurapy.robot.Robot method*), 157  
 set\_tool() (*neurapy.robot.Robot method*), 157  
 set\_tool\_digital\_output() (*neurapy.robot.Robot method*), 157  
 set\_tool\_digital\_outputs() (*neurapy.robot.Robot method*), 157  
 stop() (*neurapy.robot.Robot method*), 158  
 stop\_movelinear\_online() (*neurapy.robot.Robot method*), 158  
 switch\_to\_automatic\_mode() (*neurapy.robot.Robot method*), 159  
 switch\_to\_real() (*neurapy.robot.Robot method*), 159  
 switch\_to\_semi\_automatic\_mode() (*neurapy.robot.Robot method*), 159  
 switch\_to\_simulation() (*neurapy.robot.Robot method*), 160  
 switch\_to\_teach\_mode() (*neurapy.robot.Robot method*), 160  
 turn\_off\_free\_drive\_mode() (*neurapy.robot.Robot method*), 161  
 turn\_off\_jog() (*neurapy.robot.Robot method*), 161  
 turn\_on\_free\_drive\_mode() (*neurapy.robot.Robot method*), 161  
 turn\_on\_jog() (*neurapy.robot.Robot method*), 162

## U

unpause() (*neurapy.robot.Robot method*), 162  
 update\_current\_tool\_parameters() (*neurapy.robot.Robot method*), 162  
 update\_pose\_in\_DB() (*neurapy.robot.Robot method*), 164  
 update\_tool\_parameters() (*neurapy.robot.Robot method*), 164

## W

wait() (*neurapy.robot.Robot method*), 166  
 wait\_for\_analog\_input() (*neurapy.robot.Robot method*), 166  
 wait\_for\_digital\_input() (*neurapy.robot.Robot method*), 167  
 wait\_for\_digital\_input\_timer\_off\_delay() (*neurapy.robot.Robot method*), 167  
 wait\_for\_digital\_input\_timer\_on\_delay() (*neurapy.robot.Robot method*), 168  
 wait\_for\_tool\_analog\_input() (*neurapy.robot.Robot method*), 168  
 wait\_for\_tool\_digital\_input() (*neurapy.robot.Robot method*), 168  
 wait\_for\_tool\_digital\_input\_timer\_off\_delay() (*neurapy.robot.Robot method*), 169  
 wait\_for\_tool\_digital\_input\_timer\_on\_delay() (*neurapy.robot.Robot method*), 169  
 wait\_motion\_finished() (*neurapy.robot.Robot method*), 170

## T

tare\_fts() (*neurapy.robot.Robot method*), 160  
 transform\_pose\_to\_reference() (*neurapy.robot.Robot method*), 160